

HAWASSA UNIVERSITY

HAWASSA INSTITUTE OF TECHNOLOGY

DEPARTMENT OF ELECTRICAL AND COMPUTER ENGINEERING

DEVELOPMENT OF AN INTEGRATED SPEECH TECHNOLOGY SYSTEM FOR AFAAN OROMO

BY

Admassu Ashebir, Hunde Dejene, and Nimona Asefa

ADVISOR

Mr. Musa Hayato(MSc)

A THESIS SUBMITTED TO THE FACULTY OF ELECTRICAL ENGINEERING OF HAWASSA INSTITUTE OF TECHNOLOGY IN PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR THE DEGREE OF BACHELOR OF SCIENCE IN ELECTRICAL AND COMPUTER ENGINEERING

Hawassa, Ethiopia

May 2026

HAWASSA UNIVERSITY

HAWASSA INSTITUTE OF TECHNOLOGY

DEPARTMENT OF ELECTRICAL AND COMPUTER ENGINEERING

This thesis, entitled "***Development Of An Integrated Speech Technology System For Afaan Oromoo***", prepared by Admassu Ashebir, Hunde Dejene, and Nimona Asefa, is approved for graduation in partial fulfillment of the requirements for the degree of Bachelor of Science in Electrical and Computer Engineering

(Computer Engineering Stream).

Board of Examiners:

Advisor: _____Signature:_____ Date:_____

Examiner: _____Signature:_____ Date:_____

Examiner: _____Signature:_____ Date:_____

Examiner: _____Signature:_____ Date:_____

Examiner: _____Signature:_____ Date:_____

DECLARATION

We, the undersigned, declare that this thesis is our original work, has not been presented for a degree in this or any other university, and that all sources of materials used for the thesis have been duly acknowledged.

Research Submitted By

Names of Students:

1. _____Signature: _____ Date: _____
2. _____Signature: _____Date: _____
3. _____Signature: _____Date: _____

This thesis has been submitted for examination with my approval as an advisor.

Name of Advisor: _____Signature:_____Date: _____

ABSTRACT

Cushitic languages such as Afaan Oromo spoken by over 50 million people in the Horn of Africa remain critically underrepresented in speech technology. While Automatic Speech Recognition (ASR) research for Afaan Oromo has progressed, the absence of Text-to-Speech (TTS) synthesis leaves existing systems fundamentally unidirectional. This thesis addresses this gap through a comparative empirical evaluation of three neural TTS architectures F5-TTS (flow-matching diffusion), VITS (variational inference with adversarial learning), and Fish Speech (codec-based language model) across ten experimental configurations trained under Kaggle T4 GPU constraints.

The results demonstrate that architecture selection is more consequential than dataset scale for low-resource TTS: F5-TTS fails to adapt due to dominant English/Mandarin phonetic priors; unfrozen VITS training collapses into GAN-induced artifacts; whereas Fish Speech fine-tuned via Low-Rank Adaptation (LoRA r=32) combined with Retrieval-based Voice Conversion (RVC v2) achieves the highest naturalness and intelligibility, with a Mean Opinion Score (MOS) of 4.35 from a panel of ten native speakers. Concurrently, the Whisper-Small ASR model was progressively fine-tuned on a corpus expanded to 101,518 pairs (~141 hours), achieving a Word Error Rate of 1.5% on the validation set.

The final system integrates ASR, TTS, and on-device Google ML Kit OCR into a unified FastAPI backend and cross-platform Flutter mobile application, deployed alongside four web applications on Hugging Face Spaces. This deployment initiates a *Virtuous Cycle* feedback loop in which user speech contributions continuously expand the training corpus, establishing a sustainable, open-source data infrastructure for low-resource Cushitic speech technology.

Keywords: *Afaan Oromoo, Text-to-Speech Synthesis, Automatic Speech Recognition, Variational Inference (VITS), Codec Language Model (Fish Speech), Low-Rank Adaptation, Retrieval-based Voice Conversion (RVC).*

DEDICATION

This work is proudly dedicated to the Oromoo people, whose rich heritage, enduring resilience, and unwavering spirit have been a constant source of inspiration. We honor the community that nurtured us and hope this project serves as a meaningful contribution to the advancement, preservation, and digital growth of our beautiful language and culture.

ACKNOWLEDGEMENTS

First and foremost, we would like to acknowledge the grace of the Almighty and our Savior, Jesus Christ, whose divine protection, strength, and guidance have sustained us through every step of our academic journey and daily lives.

We would like to express our deepest gratitude to our advisor, Mr. Musa Hayato, for his invaluable guidance, constructive criticism, and relentless encouragement throughout this research. His scholarly insights and high standards of research have significantly shaped this thesis.

Special thanks go to the open-source community, particularly the developers of Fish Speech, VITS, Applio, and OpenAI's Whisper, whose repositories and pretrained models made this study possible under limited compute constraints. We also extend our sincere appreciation to the native speakers of Afaan Oromo who participated in recording and validating data for the speech corpus.

Finally, we express our heartfelt gratitude to our families, friends and classmates for their moral support, understanding, contribution and patience during our undergraduate studies.

Table of Contents

DECLARATION ii

ABSTRACT iii

DEDICATION iv

ACKNOWLEDGEMENTS v

List of Tables xiv

Appendix Tables xvi

List of Figures xviii

LIST OF ABBREVIATIONS AND ACRONYMS xx

CHAPTER 1: INTRODUCTION 1

1.1 Background of the Study 1

1.2 Statement of the Problem 3

1.2.1 The Unidirectional Communication Constraint 3

1.2.2 The ASR Accuracy Plateau 3

1.2.3 Dialectal Erasure and Linguistic Homogenization 4

1.2.4 Absence of Comparative Architecture Studies and Integrated Deployment 4

1.2.5 Inefficient and Unsustainable Data Pipelines 4

1.3 Research Questions 5

1.4 Objectives of the Study 5

1.4.1 General Objective 5

1.4.2 Specific Objectives 6

1.5 Scope and Limitations 6

1.5.1 Scope of the Project 6

1.5.2 Limitations 7

1.6 Significance of the Study 8

1.6.1 Academic and Methodological Significance 8
1.6.2 Social and Cultural Impact 9
1.6.3 Technological and Economic Contribution 9
1.7 Organization of the Thesis 9
CHAPTER 2: LITERATURE REVIEW 11
2.1 Evolution of Automatic Speech Recognition 11
2.1.1 Statistical Foundations: HMM-GMM Systems 11
2.1.2 The Hybrid Revolution: HMM-DNN Architectures 12
2.1.3 End-to-End Neural Models: The CTC and Attention Era 12
2.1.4 Self-Supervised Learning: Wav2Vec 2.0 and HuBERT 13
2.1.5 Weakly Supervised Large-Scale Models: OpenAI Whisper 14
2.1.6 ASR for Low-Resource Languages: Specific Challenges 15
2.2 State of Afaan Oromo ASR Research 15
2.2.1 Academic Contributions 15
2.2.2 Commercial and Platform-Based Systems 16
2.3 Evolution of Text-to-Speech Synthesis 17
2.3.1 Concatenative and Unit-Selection Synthesis 18
2.3.2 Statistical Parametric Synthesis 18
2.3.3 Neural TTS: Tacotron, WaveNet, and the Deep Learning Revolution 19
2.3.4 VITS: Conditional Variational Autoencoder with Adversarial Learning 20
2.3.5 Transformer-Based TTS and F5-TTS 20
2.3.6 Codec-Based Neural TTS: Fish Speech 21
2.3.7 Zero-Shot Voice Cloning 22
2.4 State of Afaan Oromo Speech Synthesis Research 23
2.4.1 State-of-the-Art: Transformer Neural TTS 23

2.4.2 Gap Analysis: What Remains Unaddressed 23
2.5 Optical Character Recognition (OCR) 24
2.5.1 Traditional OCR: Tesseract 24
2.5.2 Deep Learning OCR: PaddleOCR and Google ML Kit 24
2.5.3 OCR for Latin-Script African Languages 25
2.6 Transfer Learning and Fine-Tuning Strategies for Low-Resource Languages 25
2.6.1 Progressive Multilingual Transfer Learning 26
2.6.2 Parameter-Efficient Fine-Tuning: Low-Rank Adaptation (LoRA) 26
2.6.3 Frozen-Component Fine-Tuning Strategies 27
2.6.4 Vocabulary Expansion and Tokenizer Adaptation 28
2.7 Research Gap and Motivation 29
2.8 Summary of Literature Review 31
CHAPTER 3: METHODOLOGY 33
3.1 Research Design 33
3.2 System Architecture Overview 33
3.3 Dataset Preparation 35
3.3.1 TTS Dataset: Single-Speaker Corpus 35
3.3.2 TTS Dataset: Multi-Speaker Corpus 36
3.3.3 ASR Dataset 37
3.3.4 Afaan Oromoo Text Normalization Module 38
3.3.5 Audio Preprocessing 39
3.4 TTS Model Development: Experimental Configurations 40
3.4.1 Experimental Design 40
3.4.2 F5-TTS Fine-Tuning (Experiments A and B) 41
3.4.3 VITS Fine-Tuning (Experiments C, D, and E) 43

3.4.4 Fish Speech Fine-Tuning (Experiments F, G, H, and I) 44
3.5 ASR Model Development 47
3.5.1 Model 1: Semester 1 Baseline (Whisper_Merged_Final) 47
3.5.2 Model 2: Clean-Audio Fine-Tuning (Whisper_Sem2_Final) 47
3.5.3 Model 3: Augmented Fine-Tuning (Whisper_R3_Final) 48
3.5.4 Evaluation Metric: Word Error Rate 48
3.6 OCR Integration 48
3.7 System Integration and Deployment 49
3.7.1 The Virtuous Cycle Data Ecosystem 49
3.7.2 Backend Architecture (FastAPI) 49
3.7.3 Mobile Application (Flutter) 50
3.7.4 Web Application Deployment 50
3.7.5 Docker Deployment Architecture 51
3.8 Ethical Considerations 51
3.9 Evaluation Framework 52
3.9.1 Objective Metrics 52
3.9.2 Subjective Assessment 52
3.9.3 Comparative Analysis Framework 53
CHAPTER 4: EXPERIMENTS AND RESULTS 54
4.1 Experimental Environment 54
4.2 F5-TTS Results Complete Failure 55
4.2.1 Custom Vocabulary Design 55
4.2.2 Training Infrastructure 55
4.2.3 Experiment A: Multi-Speaker (68 Hours) 56
4.2.4 Experiment B: Single-Speaker (17 Hours) 57

4.2.5 Comprehensive F5-TTS Failure Analysis 58
4.3 VITS Results Three Experiments 59
4.3.1 Experiment C: Vocabulary Expansion (96 Tokens) 59
4.3.2 Experiment D: Full Fine-Tuning (Decoder Unfrozen) 60
4.3.3 Experiment E: Frozen Decoder Fine-Tuning 62
4.3.4 VITS Comparative Analysis 63
4.4 Fish Speech Results Four Experiments 64
4.4.1 Experiment F: LoRA r=8, Single Speaker 64
4.4.2 Experiment G: LoRA r=32, Single Speaker 65
4.4.3 Experiment H: LoRA r=32, Multi-Speaker 66
4.4.4 Experiment I: Fish Speech + RVC Pipeline (Best Results) 69
4.4.5 Experiment J: VITS + RVC Pipeline (Baseline Comparison) 71
4.4.6 Perceptual Evaluation (MOS) Results 72
4.5 ASR Results 73
4.5.1 Three-Stage Model Progression 73
4.5.2 Augmentation Impact Analysis 74
4.5.3 Generalization vs. Test-Set Performance 74
4.6 Comprehensive Comparative Analysis 75
4.6.1 Master Comparison Table 75
4.6.2 Key Insight: Architecture Selection vs. Dataset Size 78
4.6.3 Comparison with Prior Work 79
4.7 Deployment Results 79
4.7.1 Web Applications 79
4.7.2 Mobile Application 80
CHAPTER 5: DISCUSSION AND ANALYSIS 82

5.1 Interpretation of TTS Results 82
5.1.1 Why F5-TTS Failed for Afaan Oromo 82
5.1.2 Why VITS Vocabulary Expansion Caused Catastrophic Failure 84
5.1.3 Why VITS Decoder Unfreezing Caused Buzzing 85
5.1.4 Why Frozen Decoder Fine-Tuning Partially Succeeded 86
5.1.5 Why Fish Speech Achieved Best Results 88
5.1.6 Native Voice Cloning Limitations and the RVC Alternative 88
5.2 The Architecture vs. Dataset Size Debate 89
5.3 Comparison with Prior Afaan Oromo TTS Work 90
5.4 ASR Discussion 91
5.5 Challenges and Lessons Learned 92
5.5.1 Computational Constraints 92
5.5.2 Data Quality vs. Quantity 92
5.5.3 Kaggle-Specific Bugs 92
5.5.4 Production Deployment Challenges 93
5.6 Threats to Validity 93
CHAPTER 6: CONCLUSION AND FUTURE WORK 95
6.1 Summary of Contributions 95
6.2 Key Findings 96
6.3 Limitations 97
6.4 Future Work 97
6.4.1 Dataset Curation and Expansion 98
6.4.2 Real-Time Streaming Inference 98
6.4.3 On-Device Inference via Model Quantization 98
6.4.4 Formal Perceptual Evaluation 98

6.4.5 Speaker Diarization and Multi-Turn Dialogue 99
6.4.6 Expansion to Additional Cushitic Languages 99
6.4.7 iOS Deployment and Cross-Platform Expansion 99
6.4.8 Emotional and Expressive Speech Synthesis 99
6.5 Closing Statement 100
REFERENCES 101
APPENDIX A: DETAILED IMPLEMENTATION SUPPLEMENT 105
S3.1 Complete Data Processing Pipeline 105
S3.1.1 Robust CSV Parser Implementation (Cell 3) 105
S3.1.2 Five-Strategy Prefix Leak Detection and Correction (V2 Pipeline) 106
S3.1.3 Audio Validation and Normalization 107
S3.1.4 Dataset Format Per Architecture 108
S3.2 Comprehensive Afaan Oromoo Text Normalization Module Full Specification 108
S3.2.1 Number-to-Word Conversion (*num_to_oromo*) 109
S3.2.2 Ordinal Numbers (`num_to_oromo_ordinal`) 110
S3.2.3 Date Normalization 111
S3.2.4 Currency Normalization 111
S3.2.5 Other Normalizations 112
S3.2.6 Abbreviation Expansion (60+ entries) 112
S3.3 VITS Implementation: Complete Cell-by-Cell Pipeline 113
S3.3.1 VITS V1 Pipeline (Unfrozen Decoder Experiment D) 113
S3.3.2 VITS V2 Pipeline (Frozen Decoder Experiment E) 113
S3.3.3 Complete VITS Training Patches (A–K) 114
S3.3.4 V2 Anti-Collapse Training Configuration 114
S3.3.5 Weight-Norm Inference Pipeline (Cell 8) 115

S3.3.6 Chunked Generation for Long Text (V2 Cell 8b) 116
S3.3.7 Discriminator Collapse Pattern (V2 Discovery) 117
S3.4 Fish Speech Implementation: Extended Details 118
S3.4.1 LoRA Merge and Interpolation 118
S3.4.2 Fish Speech Dataset Preparation 118
S3.4.3 Per-Speaker RVC Preprocessing Scripts 119
S3.4.4 The fix_f0.py Custom Script Full Implementation 119
S3.5 Deployment Details: Complete Bug Registry 121
S3.5.1 Dependency and Environment Bugs (9 resolved) 121
S3.5.2 Inference and Audio Quality Bugs (7 resolved) 122
S3.5.3 RVC Parameter Tuning Journey 123
S3.5.4 Fish Speech + RVC Web Application File Structure 124
S3.5.5 Iterative Deployment Timeline (15+ iterations) 125
S3.5.6 Gradio Web Interface Features 125
S3.6 ASR Implementation: Extended Details 126
S3.6.1 Data Augmentation Implementation 126
S3.6.2 Shard-Based Feature Extraction 126
S3.6.3 Decoder Configuration for Oromo 127

List of Tables

Table No.	Title	Page
Table 2.1	Summary of Key Literature Works Reviewed in Speech Synthesis, Recognition, and OCR Domains	32
Table 3.1	Deployment Interfaces and Locations of the Integrated Speech and OCR System	35
Table 3.2	Statistical and Formatting Properties of the Primary Single-Speaker TTS Dataset	35
Table 3.3	Prefix Leak Detection Strategies and Criteria for the TTS Dataset Cleanup	36
Table 3.4	Properties and Composition of the Sourced Afaan Oromo ASR Dataset	37
Table 3.5	Data Augmentation Strategies Applied to the ASR Dataset	37
Table 3.6	Post-Augmentation Statistics for the Fine-Tuned ASR Dataset	38
Table 3.7	Summary of All TTS Experimental Configurations	40
Table 3.8	Hyperparameter and Training Configuration for the F5-TTS Experiments	42
Table 3.9	Hyperparameters for LoRA Rank-8 (r=8) Fine-Tuning of Fish Speech	44
Table 3.10	Hyperparameters for LoRA Rank-32 (r=32) Fine-Tuning of Fish Speech	44
Table 3.11	Properties and Training Parameters of the Five Target RVC Models	46
Table 3.12	Recommended Production Parameters and Rationale for RVC Inference	46
Table 3.13	Fine-Tuning Hyperparameters for Whisper ASR Model 2 (Clean Dataset)	47
Table 3.14	Fine-Tuning Hyperparameters for Whisper ASR Model 3 (Augmented Dataset)	48
Table 3.15	API Endpoint Definitions and HTTP Methods for the FastAPI Backend	50
Table 3.16	Crucial Dependency Version Pinning and Rationale for the Synthesis Environment	51
Table 3.17	Standard Afaan Oromo Test Sentences Used for Comparative Evaluation	53
Table 4.1	Hardware and Software Environment Specifications	54
Table 4.2	Character Distribution and Tokens in the Custom 87-Token Afaan Oromo Vocabulary	55

Table 4.3	Summary of Monkey Patches Required for F5-TTS Kaggle Environment Adaptation	56
Table 4.4	Model Hyperparameters for VITS Fine-Tuning on Single-Speaker Dataset	61
Table 4.5	Stable Training Patches (A–K) Applied to the VITS Pipeline	61
Table 4.6	Comprehensive VITS Experiment Comparison Across Key Criteria	63
Table 4.7	Fish Speech Multi-Speaker Checkpoint Evaluation (Experiment H)	67
Table 4.8	Comparative Training Dynamics: Single-Speaker (Exp. G) vs. Multi-Speaker (Exp. H)	69
Table 4.9	RVC v2 Model Training Results on 5 Speakers	70
Table 4.10	Production Bugs and Quick Fixes Identified During App Deployment	70
Table 4.11	Aggregated Mean Opinion Score (MOS) Evaluation by 10 Native Speakers	72
Table 4.12	Fine-Tuning Progression of the Whisper-Small ASR Model	73
Table 4.13	Dataset Augmentation and Corpus Expansion Statistics	74
Table 4.14	WER Comparison on Unseen Public Dataset (1,610 samples)	76
Table 4.15	WER Comparison on Development Set (Studio Quality, 2,387 samples)	76
Table 4.16	Complete Architecture Comparison Across All Ten Synthesis Experiments	77
Table 4.17	Detailed Comparison with Prior Afaan Oromo Text-to-Speech Systems	79
Table 4.18	List of Web Applications Deployed on Hugging Face Spaces	79
Table 4.19	Inference Performance and Operations Latency on CPU-Only Deployment Spaces	80

Appendix Tables

Table No.	Title	Page
Appendix Table 1	Base Digits (0–10) and Their Written Form in Afaan Oromo	109
Appendix Table 2	Compound and Cardinal Written Forms of Tens (20–90) in Afaan Oromo	109
Appendix Table 3	Scale Formats and Written Examples of Hundreds and Higher Numerals in Afaan Oromo	110
Appendix Table 4	Cardinal and Ordinal Conversion Patterns in Afaan Oromo	110
Appendix Table 5	English Month Equivalents and Their Standard Names in Afaan Oromo	111
Appendix Table 6	Currency Symbols and Their Written Conversion Rules in Afaan Oromo	111
Appendix Table 7	Special Categories and Expansion Rules for Oromo Text Normalization	112
Appendix Table 8	Standard Abbreviation Expansion Dictionary for Afaan Oromo Text Normalization	112
Appendix Table 9	Notebook Cell Mapping for VITS V1 Unfrozen Decoder Pipeline	113
Appendix Table 10	Notebook Cell Mapping and Environment Upgrades for VITS V2 Pipeline	113
Appendix Table 11	Complete Stable Patches (A–K) Implemented in the VITS Training Loop	114
Appendix Table 12	Hyperparameter Tuning Iterations and Rationale for VITS V2 Anti-Collapse Training	115
Appendix Table 13	Phase-Wise Analysis of VITS V2 Discriminator Collapse and Loss Behavior	117
Appendix Table 14	Per-Speaker Audio Preprocessing and Training Dataset Distribution in RVC	119

Appendix Table 15	Dependency and Environment Integration Bugs Registry and Resolutions	121
Appendix Table 16	Inference Latency and Synthesis Audio Quality Bugs Registry and Resolutions	122
Appendix Table 17	RVC Inference Parameter Hyperparameter Tuning and Empirical Findings	123
Appendix Table 18	Continuous Deployment Iterations and Deployment Stages for the TTS Application	125

List of Figures

Figure No.	Title	Page
Figure 1	Complete System Architecture Diagram showing the integrated pipelines (ASR→Text flow, TTS flow, and Image→OCR→TTS flow), FastAPI backend, Flutter app, and web applications, with arrows representing the "Virtuous Cycle" feedback loop	34
Figure 2	Comparative quality assessment across all ten experimental configurations, visualizing intelligibility, naturalness, and overall, MOS ratings	41
Figure 3	RVC v2 voice conversion spectrograms showing source Fish Speech output (left), target speaker reference (middle), and RVC-converted output (right)	45
Figure 4	Evaluation Framework Flowchart visualizing the evaluation pipeline from audio generation to objective metrics and subjective assessment, leading to comparative ranking and final architecture selection	53
Figure 5	Visualization of F5-TTS Multi-Speaker Output Waveform (top) and Spectrogram (bottom), showing a random noise-like pattern and uniform energy distribution without formant bands or harmonic structure	57
Figure 6	Spectrogram comparison between Reference Oromo Speech (left) and F5-TTS Single-Speaker Output (right), illustrating how the synthesized output fails to form Oromo formant distributions	58
Figure 7	Fish Speech LoRA r=8 Training Loss Curve, showing steady decrease from approximately 4.0 to 1.5 over 3,000 steps, followed by an early plateau indicating underfitting due to rank capacity limitations	65
Figure 8	Comparison of Training Loss Curves between LoRA r=8 and r=32, showing the r=32 configuration continues to decrease steadily to approximately 0.9 by step 7,000, breaking through the representation plateau	66
Figure 9	Whisper ASR Progressive Fine-Tuning Results across three training rounds: Round 1 (Semester 1 baseline, WER 55.94%), Round 2 (clean data fine-tune, WER 40.36%), and Round 3 (augmented data fine-tune, WER 38.82% on unseen data, 1.5% on internal validation set)	75
Figure 10	Conceptual User Interface Flow of the Flutter Mobile Application, demonstrating (1) the Speech-to-Text tab with live recording waveforms and transcripts, (2) the Text-to-Speech tab with dialect and speaker controls, and (3) the OCR tab displaying extracted document text with one-tap synthesis routing	81

Figure 11	Waveform comparison between reference Oromo speech (left) and VITS unfrozen decoder output (right) showing narrow-bandwidth periodic buzzing characteristic of GAN mode collapse, with no natural speech envelope or phonemic structure	86
Figure 12	Spectrogram comparison between reference Oromo speech (left) and VITS frozen decoder output (right), showing preserved formant structure and harmonic bands from the pre-trained MMS model	87

LIST OF ABBREVIATIONS AND ACRONYMS

API	Application Programming Interface
AR	Autoregressive
ASR	Automatic Speech Recognition
BOS	Beginning of Sequence
CFM	Conditional Flow Matching
CNN	Convolutional Neural Network
CTC	Connectionist Temporal Classification
DiT	Diffusion Transformer
DNN	Deep Neural Network
ELBO	Evidence Lower Bound
EMA	Exponential Moving Average
F0	Fundamental Frequency (Pitch)
FAISS	Facebook AI Similarity Search
FFN	Feed-Forward Network
FSQ	Finite Scalar Quantization
GAN	Generative Adversarial Network
GMM	Gaussian Mixture Model
HF	Hugging Face
HMM	Hidden Markov Model
IPA	International Phonetic Alphabet
KL	Kullback-Leibler (Divergence)
LLM	Large Language Model
LoRA	Low-Rank Adaptation
MAS	Monotonic Alignment Search
MFCC	Mel-Frequency Cepstral Coefficients
MMS	Massively Multilingual Speech (Meta)
MOS	Mean Opinion Score
MPD	Multi-Period Discriminator
NLP	Natural Language Processing
OCR	Optical Character Recognition

ODE	Ordinary Differential Equation
OOM	Out Of Memory
PEFT	Parameter-Efficient Fine-Tuning
RMVPE	Robust Model for Pitch Estimation using Vocal Envelope
RVC	Retrieval-based Voice Conversion
SSL	Self-Supervised Learning
T2S	Text-to-Semantic
TTS	Text-to-Speech
VAD	Voice Activity Detection
VAE	Variational Autoencoder
VITS	Variational Inference with adversarial learning for end-to-end Text-to-Speech
VQGAN	Vector Quantized Generative Adversarial Network
VRAM	Video Random Access Memory
WAV	Waveform Audio File Format
WER	Word Error Rate

CHAPTER 1: INTRODUCTION

1.1 Background of the Study

The rapid proliferation of digital technologies has fundamentally transformed human communication, information access, and socioeconomic participation across the globe. Voice-enabled interfaces encompassing virtual assistants, automated customer service platforms, real-time transcription tools, and screen readers have emerged as indispensable conduits through which individuals interact with digital ecosystems. The global speech and voice recognition market, valued at approximately USD 12.62 billion in 2023, is projected to reach USD 84.97 billion by 2033 [1]. However, the overwhelming majority of commercial speech systems are engineered for a narrow set of high-resource languages predominantly English, Mandarin, Spanish, and a handful of European languages collectively representing fewer than one hundred of the world's estimated seven thousand living languages [2]. This disparity engenders a pervasive "digital language divide," wherein speakers of technologically underserved languages are systematically excluded from the benefits of the digital revolution.

Nowhere is this divide more acute than across the African continent, which is home to over two thousand distinct languages yet remains critically underrepresented in the global NLP research landscape. A comprehensive survey by Masakhane revealed that as of 2023, fewer than 5% of African languages had any publicly available speech dataset of sufficient quality and volume to support modern deep learning approaches [3]. This scarcity of training corpora, annotated speech data, and benchmark evaluation sets defines the "low-resource" classification that encompasses the vast majority of African linguistic heritage.

Afaan Oromo occupies a particularly consequential position within this landscape. As the most widely spoken Cushitic language in the Afroasiatic family, it serves as the primary language of communication for over 50 million people across the Oromia Regional State of Ethiopia, as well as parts of Kenya, Somalia, and diaspora communities worldwide [4]. In 2020, Afaan Oromo was adopted as a federal working language of Ethiopia alongside Amharic [5], further amplifying the imperative for robust digital language tools. Despite

this demographic and political significance, Afaan Oromo remains classified as a low-resource language in computational linguistics.

The field of Automatic Speech Recognition (ASR) has undergone transformative evolution in recent years. Self-supervised models such as Wav2Vec 2.0 [6] and large-scale weakly-supervised models such as OpenAI's Whisper [7] trained on approximately 680,000 hours of multilingual data have demonstrably lowered the barrier to entry for developing speech recognition systems for previously neglected languages. Building on this foundation, the first phase of this research successfully established a baseline ASR system for Afaan Oromo by fine-tuning the Whisper-Small architecture on a curated speech corpus, achieving a Word Error Rate (WER) of approximately 25.04% on clean audio a meaningful advance over prior academic baselines exceeding 40% WER [8].

However, a fundamental asymmetry persists: existing Afaan Oromo speech research is typically segmented, focusing exclusively on either ASR or TTS in isolation, without unified deployment strategies. A truly inclusive speech platform demands both the ability to recognize speech and to synthesize it. Modern neural TTS architectures including VITS [9], SpeechT5 [10], F5-TTS [11], and Fish Speech [12] offer unprecedented opportunities for high-quality synthesis in data-constrained environments, particularly when combined with parameter-efficient fine-tuning strategies such as Low-Rank Adaptation (LoRA) [13]. Furthermore, zero-shot voice cloning technologies enable the preservation of Afaan Oromo's rich dialectal diversity across the Borena, Shewa, Wollega, Hararghe, and Jimma zones.

It is within this confluence of technological opportunity and unmet linguistic need that the present research is situated. This thesis presents a comprehensive comparative evaluation of neural TTS architectures specifically F5-TTS, VITS, and Fish Speech for low-resource Afaan Oromo, culminating in an open-source integrated speech ecosystem that combines the optimal TTS model with progressive ASR, Optical Character Recognition (OCR), and a community-driven data collection platform.

1.2 Statement of the Problem

Despite the foundational progress achieved during the first phase of this research, five critical and interconnected gaps impede the realization of a fully functional speech system for Afaan Oromo. These gaps span the domains of system capability, model performance, linguistic representation, data sustainability, and architectural integration.

1.2.1 The Unidirectional Communication Constraint

The existing ASR model converts speech to text but provides no mechanism for vocalizing information back to the user. This unidirectionality critically undermines the system's utility for its most vulnerable target users: visually impaired individuals who cannot perceive text-based outputs, populations with limited literacy for whom written text holds no functional value, and oral-literate communities who preferentially engage with information in its spoken form. Without a TTS component, the system cannot function as a digital assistant, an accessible reading aid, or a conversational interface applications that have become integral to contemporary digital participation. This absence constitutes a structural barrier to equitable information access for the Afaan Oromo-speaking population.

1.2.2 The ASR Accuracy Plateau

The baseline ASR model has reached a performance plateau at approximately 25% WER on clean audio. While acceptable for casual transcription, this falls short of the precision demanded by high-stakes application domains. In healthcare settings, a misrecognized medical term could precipitate clinical errors. In legal and administrative contexts, transcription inaccuracies could compromise the integrity of official records. Error analysis reveals particular degradation on complex syntactic constructions, domain-specific terminology, and code-switching between Afaan Oromo, Amharic, and English phenomena ubiquitous in natural speech. Overcoming this plateau requires strategically curated, domain-specific data that targets these identified weaknesses through principled collection methodologies.

1.2.3 Dialectal Erasure and Linguistic Homogenization

Afaan Oromo encompasses rich regional varieties including Borena, Shewa, Wollega, Hararghe, Arsi, and Guji dialects each with distinctive phonological patterns, prosodic contours, and lexical preferences. These dialectal variations are not merely academic curiosities; they are integral components of regional identity and cultural heritage. Standard TTS implementations generate speech in a single artificial "standard" voice that fails to reflect this diversity, risking alienation of users who do not hear their own linguistic identity in the system's output, thereby reducing trust, adoption, and perceived cultural relevance.

1.2.4 Absence of Comparative Architecture Studies and Integrated Deployment

Existing Afaan Oromo speech technology efforts have proceeded in isolation, with no rigorous comparative evaluation of how modern competing neural architectures flow-matching, codec-based, and variational approaches perform under low-resource constraints. To the best of the authors' knowledge, no open-source system currently exists that identifies the optimal TTS architecture through empirical testing and integrates it with ASR into a unified deployable application. Existing commercial solutions remain proprietary, opaque, and unavailable for offline use a critical limitation in regions with intermittent connectivity.

1.2.5 Inefficient and Unsustainable Data Pipelines

The long-term viability of any speech system depends on the continuous availability of high-quality training data. The first phase introduced an improved data collection platform, but it still operates unidirectionally collecting speech for ASR training without leveraging TTS to create a validation loop. No interactive ecosystem exists in which users simultaneously contribute to and benefit from the system, creating a self-reinforcing cycle of data quality improvement.

These five problems are structurally interrelated. The absence of TTS (Problem 1) precludes a unified validation ecosystem (Problem 5). Lack of dialectal awareness (Problem 3) constrains ASR improvement (Problem 2). The absence of integrated architecture (Problem 4) prevents any individual solution from delivering tangible user

value. This thesis addresses all five through a unified, architecturally coherent approach grounded in rigorous empirical comparison.

1.3 Research Questions

This research is guided by the following five research questions:

RQ1: Which neural TTS architecture among F5-TTS, VITS (including vocabulary-expanded and decoder-frozen variants), and Fish Speech yields the highest intelligibility and naturalness for Afaan Oromo speech synthesis when fine-tuned under low-resource constraints on a single T4 GPU?

RQ2: How does dataset composition specifically multi-speaker versus single-speaker training corpora affect TTS fine-tuning convergence, output quality, and phonetic accuracy for Afaan Oromo?

RQ3: Can a progressive fine-tuning strategy employing LoRA rank escalation transitioning from rank 8 to rank 32 effectively overcome training convergence plateaus in adapter-based TTS models for low-resource languages?

RQ4: To what extent can the existing Whisper-Small ASR model for Afaan Oromo be improved through targeted fine-tuning on additional domain-specific data, and what is the resulting impact on Word Error Rate?

RQ5**:** How can ASR, TTS, and OCR components be effectively integrated into a unified, open-source mobile application architecture that delivers a seamless user experience while remaining deployable on zero-cost or low-cost infrastructure?

1.4 Objectives of the Study

1.4.1 General Objective

The general objective of this thesis is to conduct a comprehensive comparative evaluation of neural TTS architectures for low-resource Afaan Oromo and integrate the optimal model with progressive ASR and OCR into a fully open-source, deployable speech platform that demonstrates dialect-adaptable voice cloning capacities.

1.4.2 Specific Objectives

To realize the general objective, the following specific objectives are pursued:

- Comparative TTS Architecture Evaluation: Rigorously evaluate F5-TTS (multi-speaker and single-speaker), VITS (vocabulary-expanded, standard, and frozen-decoder variants), and Fish Speech (progressive LoRA rank escalation from r=8 to r=32) across dimensions of intelligibility, naturalness, phonetic accuracy, and training convergence stability.
- ASR Model Enhancement: Improve the WER of the existing Afaan Oromo ASR model below 25% on clean audio through targeted fine-tuning of the Whisper-Small architecture on additional curated domain-specific data.
- Dialect-Aware TTS Development: Develop a TTS engine capable of generating natural-sounding Afaan Oromo speech with zero-shot voice cloning capacity to support major regional dialectal varieties, including Borena, Shewa, and Wollega.
- OCR-to-Speech Integration: Implement an OCR pipeline that extracts text from images and routes it to the TTS engine, enabling a "Scan-to-Speech" accessibility feature for physical Afaan Oromo text.
- End-to-End System Integration: Integrate ASR, TTS, and OCR into a unified FastAPI microservices backend and cross-platform Flutter mobile application.
- Interactive Data Ecosystem: Enhance the existing data collection platform into a "virtuous cycle" ecosystem in which contributors record speech, validate ASR transcriptions, and verify TTS outputs, creating a self-sustaining loop of improving datasets.
- Systematic Performance Evaluation: Rigorously evaluate the integrated system using WER for ASR, Mean Opinion Score for TTS, and qualitative user acceptance testing for usability and dialectal authenticity.

1.5 Scope and Limitations

1.5.1 Scope of the Project

This thesis encompasses the end-to-end development and evaluation of an integrated speech ecosystem for Afaan Oromo. The specific boundaries of the work are delineated as follows:

▪ TTS Architecture Exploration: Three neural TTS architectures investigated across ten experimental configurations varying in dataset composition, fine-tuning strategy (vocabulary-expanded full fine-tuning, frozen-decoder fine-tuning, and LoRA adaptation), and LoRA rank settings (r=8 to r=32).

▪ ASR Enhancement: The existing Whisper-Small ASR model further fine-tuned on additional curated speech data augmented via speed perturbation and additive noise.

▪ Dataset Utilization: A 68-hour multi-speaker dataset aggregated from diverse speakers and recording conditions, and a 17-hour high-quality single-speaker dataset recorded under controlled conditions

▪ Voice Conversion: Retrieval-based Voice Conversion (RVC v2) applied as a post processing pipeline to achieve multi-speaker, dialect-aware synthesis from 30 minutes of reference audio per target speaker.

▪ System Integration: A Python FastAPI backend with RESTful API endpoints and a Flutter-based Android mobile application extended from the first-phase prototype to include TTS and OCR functionality. Full integration of all model endpoints into the mobile application represents ongoing work beyond the current project timeline.

▪ OCR Implementation: A basic OCR pipeline utilizing Tesseract or Google ML Kit for text extraction from images.

▪ Data Ecosystem Enhancement: The existing web-based data collection portal extended to support ASR and TTS validation workflows.

1.5.2 Limitations

While this thesis aspires to a production-grade outcome, the work operates within specific constraints that bound the generalizability and performance of the proposed system:

▪ Computational Constraints: All model training and inference are conducted on freely available GPU resources specifically, Google Colab Free Tier and Kaggle Notebooks (NVIDIA T4, 16GB VRAM). These constraints necessitate the use of smaller model architectures, impose limitations on maximum batch size, and require aggressive checkpoint management to accommodate session timeouts.

▪ Dataset Scale Limitations: The training corpora, while substantial for a low-resource language, are orders of magnitude smaller than the datasets used to train the base models being fine-tuned. The 17-hour single-speaker corpus and the 68-hour multi-

speaker corpus may be insufficient to fully capture the phonetic and prosodic complexity of Afaan Oromo across its diverse dialectal spectrum.

▪ Real-Time Processing Constraints: The system employs a chunk-based processing paradigm and does not support true millisecond-level real-time streaming, owing to the stateless nature of the free-tier hosting environment and the inherent latency of sequential ASR-to-TTS processing chain.

▪ Dialect Representation and Dataset Imbalance: While the system architecture supports dialect-adaptable voice synthesis via RVC, the multi-speaker training corpus is severely imbalanced the Central Shewa dialect constitutes more than half of the dataset, followed by Wollega, while Borena is highly underrepresented owing to the practical constraints of field data collection in this region. This work serves as an empirical proof of concept; future studies must curate a larger, more balanced multi-speaker dataset to achieve complete dialectal awareness.

▪ Code-Switching Robustness: The models are primarily optimized for monolingual Afaan Oromo input. Performance may degrade when users engage in extensive intra-sentential code-switching with Amharic or English, a common sociolinguistic practice among urban Oromo speakers.

▪ Evaluation Depth: Subjective evaluation is constrained by the availability of native Afaan Oromo speakers within the academic institution and is not a statistically representative survey of the broader Oromo-speaking population.

1.6 Significance of the Study

1.6.1 Academic and Methodological Significance

This thesis provides a reproducible methodological blueprint for low-resource TTS evaluation and system integration, adaptable to any of the hundreds of African and Asian languages currently lacking speech technology support. It constitutes, to the best of the authors' knowledge, the first systematic comparison of F5-TTS, VITS, and Fish Speech for Afaan Oromo under controlled experimental conditions providing evidence-based guidance for architecture selection in low-resource TTS development. Negative results, such as the failure of vocabulary expansion and multi-speaker training under these constraints, are themselves valuable scientific contributions. The research additionally contributes novel fine-tuning strategies: empirical validation of a frozen-decoder

approach for VITS and progressive LoRA rank escalation for Fish Speech, both generalizable to other low-resource language scenarios.

1.6.2 Social and Cultural Impact

The integration of TTS extends the system beyond text-based transcription to encompass spoken output, with direct implications for visually impaired individuals, populations with limited literacy, and elderly community members. By supporting regional dialectal variation through voice cloning, the system actively resists technological homogenization of language, affirming the linguistic identity of users across Borena, Shewa, and Wollega regions. The OCR-to-Speech pipeline further extends utility beyond the digital domain, enabling users to photograph physical documents and hear them read aloud in natural Afaan Oromo speech.

1.6.3 Technological and Economic Contribution

The entire architecture is designed for deployment on free-tier resources, demonstrating that meaningful AI capabilities can be delivered without capital expenditure empowering local institutions, startups, and civil society organizations to adopt advanced speech AI without dependency on expensive commercial APIs. All models, datasets, code, and evaluation protocols are released as open-source contributions, enabling future researchers to build upon this foundation rather than reinventing it. The proposed *Virtuous Cycle* data collection ecosystem represents a paradigm shift from static data collection to a dynamic, self-sustaining model in which every user interaction generates validated data that incrementally improves the system a direct response to the chronic data scarcity that has historically constrained speech technology for low-resource languages.

1.7 Organization of the Thesis

The remainder of this thesis is organized as follows:

Chapter 2 reviews the technical foundations of ASR and TTS, surveying the evolution from statistical to modern neural approaches, the state of Afaan Oromo speech technology, and the research gap this thesis addresses.

Chapter 3 details the research methodology, including dataset preparation, all ten TTS experimental configurations, ASR enhancement strategy, OCR integration, and the evaluation framework.

Chapter 4 reports empirical findings across all experiments, including both successful configurations and negative results.

Chapter 5 provides interpretive analysis of the results, explaining observed performance differences and discussing threats to validity.

Chapter 6 synthesizes contributions, acknowledges limitations, and outlines directions for future work.

CHAPTER 2: LITERATURE REVIEW

2.1 Evolution of Automatic Speech Recognition

Automatic Speech Recognition (ASR) the computational process of converting spoken language into text has undergone successive paradigm shifts over the past six decades, each driven by advances in statistical modeling, neural architecture design, and the availability of large-scale training corpora. Understanding this trajectory is essential for contextualizing the Whisper-based approach adopted in this thesis and for appreciating the specific challenges that low-resource languages such as Afaan Oromo continue to pose.

2.1.1 Statistical Foundations: HMM-GMM Systems

The dominant ASR paradigm from the 1970s through the early 2000s was grounded in the mathematical framework of Hidden Markov Models (HMMs) combined with Gaussian Mixture Models (GMMs) for acoustic modeling [14]. In this framework, the acoustic signal is modeled as a sequence of states in a Markov chain, where each state emits an observation (typically a short-time spectral feature vector such as Mel-Frequency Cepstral Coefficients, MFCCs) according to a GMM probability distribution. The recognition problem is then cast as finding the most likely word sequence given the acoustic observations, solved using the Viterbi algorithm for state decoding.

HMM-GMM systems were paired with n-gram language models that captured the statistical probability of word sequences, and a pronunciation lexicon that mapped words to phoneme sequences. This tripartite architecture acoustic model, language model, and lexicon remained the standard for decades and powered the first generation of commercial speech products. Systems such as CMU Sphinx [15] and HTK [16] established the engineering foundations upon which the field was built.

However, HMM-GMM systems exhibited fundamental limitations when applied to morphologically complex or tonally distinct languages, and their performance degraded sharply in the presence of speaker variability, background noise, or accented speech. For low-resource languages such as Afaan Oromo, the requirement for phonetically balanced,

carefully annotated training corpora and hand-crafted pronunciation lexicons posed prohibitive barriers to system development.

2.1.2 The Hybrid Revolution: HMM-DNN Architectures

A critical inflection point occurred in the early 2010s with the integration of Deep Neural Networks (DNNs) into the HMM framework. Hinton et al. [17] demonstrated that replacing GMMs with DNNs for acoustic state posterior estimation yielded dramatic reductions in word error rate across standard benchmarks, establishing the HMM-DNN hybrid as the new standard. Unlike GMMs, DNNs could learn highly non-linear acoustic feature transformations and capture long-range spectral dependencies that were inaccessible to shallow mixture models.

Subsequent work replaced fully connected DNNs with architectures better suited to sequential data: Convolutional Neural Networks (CNNs) for local spectral pattern extraction [18], Long Short-Term Memory (LSTM) networks for temporal modeling [19], and eventually bidirectional LSTMs (BiLSTMs) that could leverage both past and future context for each frame. These architectures progressively improved WER on standard English benchmarks and demonstrated promising cross-lingual transfer capabilities, as pre-trained acoustic models could be adapted to new languages through a process of speaker adaptation or domain transfer.

For Afaan Oromo, Addis Ababa University researchers exploited this paradigm to develop speaker-independent ASR systems using hybrid HMM-DNN architectures [20], representing the first application of deep learning to Afaan Oromo continuous speech recognition and establishing baseline performance metrics against which subsequent approaches would be compared.

2.1.3 End-to-End Neural Models: The CTC and Attention Era

The hybrid HMM-DNN architecture, while powerful, retained the modular structure of its predecessors requiring separately trained acoustic models, language models, and lexicons, each with its own optimization objective. The deep learning revolution of the mid-2010s catalyzed a push toward end-to-end systems that learn a direct mapping from acoustic features to character or word sequences using a single unified objective.

Connectionist Temporal Classification (CTC) [21] provided the first practical end-to-end training criterion for ASR, enabling sequence-to-sequence training without requiring frame-level alignment between audio and transcript. CTC introduces a special "blank" token and marginalizes over all possible alignments during training, making it tractable to optimize with standard gradient descent. Attention-based encoder-decoder models [22] further advanced the field by replacing the alignment assumption entirely with a learned soft attention mechanism, enabling the decoder to attend to relevant parts of the encoded audio representation at each output step. Listen, Attend and Spell (LAS) [23] demonstrated that a purely attention-based system could outperform HMM-DNN hybrids on certain benchmarks.

The Transformer architecture [24] originally developed for machine translation subsequently became the dominant backbone for both the encoder and decoder in ASR, owing to its superior capacity for modeling long-range dependencies through multi-head self-attention and its favorable parallelization characteristics during training. Conformer [25], a hybrid architecture combining Transformer self-attention with convolutional modules for local pattern extraction, emerged as a particularly effective architecture for ASR and remains widely used in production systems today.

2.1.4 Self-Supervised Learning: Wav2Vec 2.0 and HuBERT

A further paradigm shift came with the application of self-supervised learning (SSL) to speech representation. Motivated by the success of BERT [26] in natural language processing, researchers sought methods to exploit the vast quantities of unlabeled audio available on the internet for learning rich speech representations prior to supervised fine-tuning on labeled data.

Wav2Vec 2.0 [6], developed by Baevski et al. at Meta AI, demonstrated that a convolutional feature encoder coupled with a Transformer context network, trained on a contrastive loss over quantized audio representations, could learn highly transferable speech features from unlabeled audio. Fine-tuning Wav2Vec 2.0 on as little as 10 minutes of labeled data yielded WERs competitive with fully supervised models trained on 100 times more labeled data. HuBERT [27] extended this approach using offline k-

means clustering to generate pseudo-labels for masked prediction, further improving representation quality.

These SSL approaches proved particularly relevant for low-resource language ASR, as they decouple representation learning which requires only unlabeled audio from supervised fine-tuning, which requires labeled speech. However, they still require language-specific fine-tuning data and do not inherently generalize zero-shot to unseen languages.

2.1.5 Weakly Supervised Large-Scale Models: OpenAI Whisper

The most consequential recent development for low-resource ASR is the emergence of weakly supervised large-scale models trained on internet-sourced audio. Radford et al. [7] at OpenAI introduced Whisper, a sequence-to-sequence Transformer trained on approximately 680,000 hours of multilingual audio paired with weakly supervised transcriptions derived from the web. Unlike SSL models, Whisper is trained entirely on supervised data albeit with noisy labels and is designed for direct zero-shot deployment across multiple tasks including transcription, translation, and language identification.

Whisper employs a log-Mel spectrogram encoder and an autoregressive Transformer decoder, trained with a cross-entropy objective over text tokens. The model is available in five size configurations Tiny, Base, Small, Medium, and Large enabling deployment across a range of computational budgets. Whisper-Small, comprising approximately 244 million parameters, represents a particularly attractive operating point for researchers with limited computational resources, as it offers a favorable balance between performance and computational cost.

Critically for this thesis, Whisper's multilingual training data includes a limited but non-trivial representation of several African languages, enabling a degree of zero-shot generalization to Afaan Oromo. The degree to which this zero-shot capability can be extended and refined through targeted fine-tuning is examined in Section 2.2.1.

2.1.6 ASR for Low-Resource Languages: Specific Challenges

While large pre-trained models such as Whisper and Wav2Vec 2.0 have substantially reduced the data requirements for ASR development, low-resource languages continue to present distinctive challenges that generic approaches do not fully address:

- Phonological Complexity: Many African languages, including Afaan Oromo, exhibit phonological features such as contrastive vowel length, geminate consonants, and phonemic distinctions absent from European languages that are underrepresented in multilingual pre-training corpora and may not be reliably captured by models trained predominantly on English and related languages.
- Morphological Richness: Afaan Oromo employs agglutinative morphology, wherein complex word forms are built by concatenating multiple morphemes onto a root. This results in a large effective vocabulary that exacerbates the out-of-vocabulary word problem and complicates acoustic-to-text alignment.
- Code-Switching: Urban Afaan Oromo speakers routinely switch between Oromo, Amharic, and English within a single utterance, a phenomenon that standard monolingual ASR systems handle poorly.
- Data Scarcity: Despite the progress made by community-driven initiatives, the total volume of publicly available, high-quality, annotated Afaan Oromo speech data remains orders of magnitude below that available for high-resource languages.

These global advances in ASR methodology have increasingly been leveraged by Ethiopian researchers to develop speech recognition capabilities for Afaan Oromo a trajectory that has accelerated markedly with the availability of pre-trained multilingual models, as the following section documents.

2.2 State of Afaan Oromo ASR Research

2.2.1 Academic Contributions

Research into Afaan Oromo ASR has been pursued primarily by Ethiopian academic institutions, with progress accelerating in tandem with the global deep learning revolution.

Early work at Addis Ababa University established the foundational corpora and evaluation protocols for Afaan Oromo ASR using HMM-GMM and subsequently HMM-DNN methodologies. The 2022 study by Addis Ababa University researchers [20] developed a speaker-independent continuous ASR system using hybrid HMM-DNN architectures, adapting classical ASR frameworks to the specific phonological characteristics of Afaan Oromo and demonstrating the feasibility of data-driven approaches for the language.

A landmark advance was achieved in 2023 when researchers at Jimma University presented a Speech Transformer-based ASR model for Afaan Oromo applied to broadcast news data [8]. This end-to-end model, leveraging the Transformer architecture, achieved a Word Error Rate of 40.2% a significant improvement over prior HMM-based baselines and the first demonstration of a purely neural, attention-based ASR system for the language. The broadcast news domain, characterized by relatively clean audio and careful articulation, provided a favorable experimental setting but may not generalize to the more challenging conditions of spontaneous conversational speech.

The present research builds directly upon this trajectory. The first phase of this project, conducted during the preceding academic semester, demonstrated that fine-tuning OpenAI's Whisper-Small model on a curated Afaan Oromo corpus could achieve a WER of approximately 25.04% on clean audio surpassing the 2023 Jimma University baseline by more than 15 percentage points in absolute terms. This result validates the effectiveness of transfer learning from large multilingual models as a strategy for accelerating Afaan Oromo ASR development. The second semester of this research further fine-tunes this model on augmented domain-specific data, targeting a WER below this established baseline.

2.2.2 Commercial and Platform-Based Systems

Several commercial platforms have incorporated limited Afaan Oromo speech capabilities. Google has included Afaan Oromo in the Gboard keyboard's voice input functionality, and Wavve AI has deployed Afaan Oromo speech recognition within its

platform ecosystem. However, these commercial implementations share common limitations that constrain their utility for the communities they purport to serve:

- Proprietary and Opaque: The training data, model architectures, and evaluation methodologies employed by commercial systems are not publicly disclosed, precluding scientific reproduction, community adaptation, or independent verification of performance claims.
- Connectivity Dependent: Commercial ASR APIs require active internet connectivity for operation, rendering them unusable in rural areas of Oromia where telecommunications infrastructure remains underdeveloped or unreliable.
- Unidirectional and Isolated: Commercial Afaan Oromo ASR implementations provide transcription as an isolated service without integration into a broader speech ecosystem that includes TTS, OCR, or community-driven data improvement mechanisms.
- Non-Dialectal: Commercial systems typically operate on a standardized variety of Afaan Oromo, offering no accommodation for the substantial phonological variation across regional dialects.

These limitations underscore the necessity of the open-source, community-centered approach pursued in this thesis. They also highlight a more fundamental constraint: the absence of a complementary speech synthesis capability renders existing systems fundamentally unidirectional a gap that motivates the TTS component of this work, to which the review now turns.

2.3 Evolution of Text-to-Speech Synthesis

Text-to-Speech (TTS) synthesis the computational generation of intelligible, natural-sounding speech from text has undergone an evolution as profound as that of ASR, transitioning from rule-based articulatory models through statistical parametric synthesis to the neural generative systems that define the state of the art today. This section traces that evolution and provides the technical background necessary for understanding the three architectures VITS, F5-TTS, and Fish Speech evaluated in this thesis.

2.3.1 Concatenative and Unit-Selection Synthesis

The first generation of practical TTS systems was based on the concatenation of pre-recorded speech segments. Unit-selection synthesis [28] extends this principle by maintaining a large database of recorded speech units typically diphones, triphones, or syllables and selecting and joining units at runtime to minimize an acoustic cost function that penalizes spectral discontinuity at concatenation boundaries and deviation from a target prosody.

Unit-selection systems can produce highly natural-sounding speech for sentences that can be well-covered by the recorded inventory, but they exhibit characteristic degradation at concatenation points, have large storage requirements proportional to the unit inventory, are rigidly tied to the voice characteristics of the original speaker, and cannot generalize to phonetic contexts not represented in the database. For Afaan Oromo, a notable early implementation of this approach was reported by Abu et al. [43] in the SN Computer Science journal, which developed a unit-selection TTS synthesizer using native speaker recordings and evaluated its intelligibility and naturalness through listening tests. This work represented a meaningful first step toward Afaan Oromo TTS but remained constrained by the inherent limitations of the unit-selection paradigm limitations that motivated the shift toward statistical and subsequently neural synthesis methods.

2.3.2 Statistical Parametric Synthesis

Statistical Parametric Speech Synthesis (SPSS) emerged as a response to the storage and coverage limitations of unit-selection systems. Rather than storing and retrieving actual speech segments, SPSS systems learn statistical models initially HMM-based that map text features to continuous acoustic parameters: specifically, spectral envelope parameters (typically Mel-Generalized Cepstral coefficients), fundamental frequency (F0) contours, and aperiodicity measures [29]. Speech is generated by driving a vocoder with these predicted parameters.

HMM-based SPSS offered advantages in flexibility a single compact model could generalize to any input text and enabled speaker adaptation through model transformation. However, the vocoded speech it produced suffered from characteristic

artifacts: excessive smoothness of spectral parameters due to the minimum mean-square error estimation criterion of HMMs, and the mechanical, buzzy quality introduced by simple vocoder models. These quality limitations constrained the adoption of HMM-based SPSS in user-facing applications and opened the door for the deep learning revolution that followed.

2.3.3 Neural TTS: Tacotron, WaveNet, and the Deep Learning Revolution

The application of deep learning to TTS initiated a dramatic quality improvement that has progressively closed the gap between synthetic and human speech. Two concurrent developments were foundational: sequence-to-sequence acoustic modeling and neural vocoding.

Wang et al. introduced Tacotron [30], a sequence-to-sequence model with attention that directly maps character sequences to Mel spectrograms. Tacotron replaced the complex feature engineering pipeline of SPSS systems with a single end-to-end trained neural network, substantially simplifying system development while improving naturalness. Tacotron 2 [31] refined this approach with a more powerful encoder and a location-sensitive attention mechanism, and paired the acoustic model with WaveNet [32] a generative convolutional neural network that models the raw waveform autoregressively one sample at a time as the vocoder. The combination produced speech quality that was rated as essentially indistinguishable from human speech in Mean Opinion Score listening tests on English, representing a watershed moment for the field.

Subsequent work addressed the primary limitation of autoregressive vocoders such as WaveNet their prohibitively slow inference speed. GAN-based vocoders, most notably HiFi-GAN [33], demonstrated that generative adversarial networks could produce high-fidelity speech waveforms from Mel spectrograms at speeds suitable for real-time applications, while maintaining perceptual quality comparable to WaveNet. HiFi-GAN became the dominant vocoder in subsequent TTS research and is incorporated as the decoder in several of the architectures evaluated in this thesis.

2.3.4 VITS: Conditional Variational Autoencoder with Adversarial Learning

The VITS (Variational Inference with adversarial learning for end-to-end Text-to-Speech) model, introduced by Kim, Kong, and Son [9], represents a significant architectural unification. Prior neural TTS systems, including Tacotron 2 with HiFi-GAN, were composed of separately trained acoustic models and vocoders. VITS integrates these components into a single model trained end-to-end through a combination of variational inference and adversarial training.

Architecturally, VITS comprises: (a) a posterior encoder that encodes the spectrogram of the target speech into a latent variable z using variational inference; (b) a prior encoder consisting of a text encoder and a normalizing flow that transforms the text-conditioned prior distribution to match the posterior; (c) a stochastic duration predictor that models the duration of each phoneme; and (d) a HiFi-GAN-based decoder that synthesizes the waveform directly from the latent variable. The model is trained to maximize the variational lower bound (ELBO) subject to an adversarial discriminator loss that encourages the decoder output to be perceptually indistinguishable from real speech.

VITS achieved state-of-the-art single-speaker and multi-speaker TTS quality at publication, and crucially, it is available in lightweight configurations including Facebook's MMS-TTS variant [34], which provides pre-trained models for over 1,100 languages including Afaan Oromo that are tractable for fine-tuning on consumer-grade hardware. The pre-trained Facebook MMS-TTS Afaan Oromo model (facebook/mms-tts-orm) serves as the base model for the VITS experiments in this thesis, including an experimental configuration that investigates whether expanding the model's compact Qubee vocabulary to 96 tokens can improve coverage of the full Afaan Oromo grapheme inventory, including digraphs and geminate consonants. The empirical outcomes of these configurations are presented in Chapter 4.

2.3.5 Transformer-Based TTS and F5-TTS

The success of Transformers in ASR motivated their systematic application to TTS acoustic modeling. SpeechT5 [10], developed by Ao et al. at Microsoft Research, introduced a unified encoder-decoder Transformer framework pre-trained on both speech

and text data for multiple spoken language processing tasks, including TTS. The model maps text input to speech output through a shared encoder-decoder backbone conditioned on speaker embeddings, enabling cross-lingual and multi-task generalization.

F5-TTS [11], introduced by Chen et al. in 2024, represents a qualitatively different approach based on flow matching a continuous normalizing flow training paradigm that learns to transform a simple noise distribution into the target data distribution along straight trajectories, enabling faster and more stable training than diffusion-based models. F5-TTS employs a DiT (Diffusion Transformer) backbone and achieves state-of-the-art zero-shot voice cloning by conditioning the flow-matching process on a reference audio clip. The model was trained on approximately 100,000 hours of English-dominant multilingual data, endowing it with robust phonetic knowledge of high-resource languages.

The extent to which F5-TTS's English-dominant phonetic representations can be overwritten through fine-tuning on a data-scarce agglutinative language such as Afaan Oromo and whether sufficient coverage can be achieved within the computational constraints of a free-tier GPU environment is one of the central empirical questions examined in Chapter 4.

2.3.6 Codec-Based Neural TTS: Fish Speech

Fish Speech [12] represents the most recent architectural paradigm in neural TTS, drawing on the success of large language models (LLMs) in text generation and applying analogous principles to speech synthesis. Rather than operating directly on continuous acoustic features or Mel spectrograms, Fish Speech employs a dual-stage approach: (a) a Vector Quantized Generative Adversarial Network (VQGAN) trained to compress raw speech waveforms into sequences of discrete integer tokens (analogous to word tokens in text), and (b) a large Transformer language model trained autoregressively to predict these discrete speech tokens conditioned on text input and reference audio tokens.

This codec-based approach offers several advantages relevant to the low-resource fine-tuning scenario: the VQGAN's audio generation capabilities are entirely pre-trained and frozen, insulating them from degradation during fine-tuning; the Transformer language

model's attention layers can be efficiently adapted using Low-Rank Adaptation (LoRA) [13] without modifying all parameters; and the discrete token representation provides a clean separation between acoustic quality (determined by the VQGAN) and linguistic content (learned by the Transformer). Voice cloning is achieved zero-shot by prepending reference audio tokens to the generation context, enabling the model to infer speaker characteristics from a short reference clip.

The effectiveness of progressive LoRA rank escalation as a strategy for overcoming convergence plateaus during Afaan Oromo fine-tuning and its interaction with dataset scale and speaker diversity is the subject of the Fish Speech experimental series reported in Chapter 4.

2.3.7 Zero-Shot Voice Cloning

Zero-shot voice cloning refers to the capability of a TTS system to synthesize speech in the voice of an arbitrary target speaker from a short reference audio clip, without any speaker-specific fine-tuning at inference time. This capability is of particular relevance for Afaan Oromo, where distinct regional dialects are characterized by measurable phonological and prosodic differences that a standard TTS system would homogenize.

XTTS v2 [35], developed by Coqui AI, employs a cross-lingual Transformer architecture conditioned on speaker embeddings derived from reference audio to achieve voice cloning across languages. SpeechT5 [10] provides zero-shot capabilities through its speaker embedding conditioning mechanism. Fish Speech achieves zero-shot voice cloning natively through its context conditioning mechanism, requiring only 630 seconds of clean reference audio to reliably transfer the acoustic characteristics of a target speaker.

For Afaan Oromo dialectal coverage, zero-shot voice cloning offers a pragmatic solution to the scarcity of dialectal training data: rather than collecting hours of labeled speech for each regional variety, a few clean reference clips in each target dialect can be used at inference time to generate regionally appropriate synthetic speech. This approach is the one adopted in this thesis for the Fish Speech model.

2.4 State of Afaan Oromo Speech Synthesis Research

Early TTS research for Afaan Oromo, as reviewed in Section 2.3.1, employed unit-selection synthesis (Abu et al. [43]), providing a meaningful first step but remaining constrained by the inherent limitations of single-voice, inventory-bound systems. The more consequential recent development is described below.

2.4.1 State-of-the-Art: Transformer Neural TTS

The most significant recent contribution to Afaan Oromo TTS is the 2025 MSc thesis by Bayisa Bedasa [36], which developed a Text-to-Speech synthesis model using a Transformer encoder-decoder architecture combined with a HiFi-GAN vocoder. The model was trained on 17 hours of curated single-speaker Afaan Oromo speech data, employing multi-head self-attention mechanisms for contextual modeling of Afaan Oromo phonology and prosody. Evaluation through Mean Opinion Score (MOS) listening tests with native speakers yielded scores above 4.0 for both naturalness and intelligibility on a 5-point scale approaching human-level perceptual quality.

This work represents a landmark in the field and provides important context for the present thesis. Notably, the dataset used in Bayisa Bedasa's work (17 hours, single speaker) is essentially identical in scale to the primary training corpus employed in the current research, enabling meaningful comparison of architectural approaches under equivalent data constraints. The present thesis extends this work in three important directions: (a) systematic comparison across multiple competing TTS architectures rather than a single selected model; (b) exploration of parameter-efficient fine-tuning strategies (LoRA, frozen decoder) suited to the free-tier computational constraints characteristic of resource-constrained environments; and (c) integration of the TTS component with ASR and OCR into a unified, deployable mobile application.

2.4.2 Gap Analysis: What Remains Unaddressed

Despite these contributions, critical gaps remain in comparative architectural evaluation, parameter-efficient fine-tuning strategies, dialectal coverage, voice cloning capability, and system integration. Each gap is examined in the consolidated analysis presented in Section 2.7.

2.5 Optical Character Recognition (OCR)

2.5.1 Traditional OCR: Tesseract

Optical Character Recognition (OCR) the automated extraction of text from images of printed or handwritten documents is the third core component of the integrated system proposed in this thesis. The Tesseract OCR engine [37], originally developed by Hewlett-Packard in the 1980s and subsequently maintained by Google as an open-source project, remains one of the most widely deployed OCR systems globally. Tesseract employs a pipeline of image preprocessing (binarization, skew correction, line segmentation), character segmentation, and classification using a Long Short-Term Memory network trained on a proprietary recognition model. Tesseract supports over 100 languages, including the Latin-script-based Qubee orthography used for Afaan Oromo, making it directly applicable to the OCR-to-Speech pipeline proposed in this thesis.

2.5.2 Deep Learning OCR: PaddleOCR and Google ML Kit

PaddleOCR [38], developed by Baidu, employs deep learning throughout its pipeline: a differentiable binarization network for text region detection, a recognition network based on the CRNN (Convolutional Recurrent Neural Network) architecture for character recognition, and an optional text direction classifier. PaddleOCR achieves state-of-the-art accuracy on standard benchmarks while maintaining inference speeds compatible with mobile deployment. Its architecture is particularly robust to complex backgrounds, variable fonts, and perspective distortions common challenges in real-world document photography.

Google ML Kit [39] provides on-device OCR through its Text Recognition API, supporting Latin-based scripts including Qubee. Its principal advantage for the mobile application context of this thesis is seamless integration with Flutter via the google_mlkit_text_recognition package, enabling entirely on-device text extraction without requiring a network connection a critical property for deployment in regions with limited internet access.

2.5.3 OCR for Latin-Script African Languages

Afaan Oromo's Qubee script is Latin-based, employing the standard 26 Latin characters augmented by digraphs (ch, dh, ny, ph, sh, ts) and a small number of modified forms. This orthographic structure makes Afaan Oromo text compatible with OCR systems trained on Latin-script languages without requiring language-specific model adaptation, provided that the recognition model is exposed to the specific typographical conventions (fonts, layouts, document styles) common in Afaan Oromo printed materials. In contrast, Ethiopian languages using the Ethiopic (Ge'ez) script such as Amharic and Tigrinya require OCR systems specifically trained on that script, representing a substantially more challenging recognition problem [40].

For the OCR-to-Speech pipeline implemented in this thesis, Google ML Kit is selected as the primary OCR engine, with Tesseract as a fallback, based on its on-device inference capability, Qubee script compatibility, and native Flutter integration. Post-processing normalization scripts handle punctuation, special character mapping, and digit-to-word conversion before routing extracted text to the TTS engine.

Having reviewed the three core technology domains ASR, TTS, and OCR the question of how pre-trained models in each domain can be effectively adapted to Afaan Oromo's data-scarce environment now requires attention. Transfer learning and fine-tuning strategies constitute the methodological foundation of this adaptation and form the final background section of this chapter.

2.6 Transfer Learning and Fine-Tuning Strategies for Low-Resource Languages

The scarcity of labeled speech data for low-resource languages such as Afaan Oromo renders the independent training of large neural TTS or ASR models from scratch computationally infeasible and empirically unreliable there is simply insufficient data to estimate the millions or billions of parameters that modern architectures contain. Transfer learning the practice of initializing a model with parameters pre-trained on a related task or language and subsequently adapting them to the target language is therefore the foundational strategy for low-resource speech technology development. This section

reviews the principal transfer learning and fine-tuning methodologies relevant to the experiments conducted in this thesis.

2.6.1 Progressive Multilingual Transfer Learning

Progressive multilingual transfer learning refers to a training strategy in which a model is sequentially fine-tuned on increasingly specific language data, leveraging intermediate languages as phonological or linguistic "stepping stones" toward the target language. For Afaan Oromo, this approach is particularly motivated by the structure of the Afroasiatic language family: Cushitic languages (including Somali and Oromo) share phonological features distinct from Semitic family members (Amharic, Tigrinya) and from Bantu languages (Swahili), but all share the African linguistic context in which they are spoken.

In practice, progressive transfer for ASR may proceed as: (1) pre-training on a large multilingual corpus (e.g., Whisper's 680,000-hour training set), (2) intermediate fine-tuning on related African languages with larger available corpora (e.g., Somali, Swahili, Amharic), and (3) final fine-tuning on target-language Afaan Oromo data. Each successive stage initializes the model closer to the optimal parameter configuration for the target language, reducing the effective data requirement at the final stage.

The Masakhane community [3] has extensively documented the benefits of this approach for African language NLP, demonstrating that intermediate fine-tuning on linguistically related languages consistently outperforms direct fine-tuning from a generic multilingual baseline when target-language data is scarce. For TTS, analogous progressive transfer strategies have been less systematically studied, partly because the acoustic characteristics of target voices are more language-specific than the semantic representations relevant to ASR.

2.6.2 Parameter-Efficient Fine-Tuning: Low-Rank Adaptation (LoRA)

Full fine-tuning of large pre-trained models updating all parameters during adaptation to a new task or language is computationally expensive and prone to catastrophic forgetting: the degradation of the model's pre-trained capabilities as it overfits to the limited target-language data. Parameter-efficient fine-tuning (PEFT) methods address this

by updating only a small subset of parameters while keeping the remainder fixed, dramatically reducing computational requirements and mitigating forgetting.

Low-Rank Adaptation (LoRA), introduced by Hu et al. [13], is the most widely adopted PEFT method and the one employed in the Fish Speech experiments of this thesis. LoRA decomposes the weight update ?W for each adapted layer as the product of two low-rank matrices:

$\Delta W = BA$ where $B \in \mathbb{R}\text{^}(d \times r)$ and $A \in \mathbb{R}\text{^}(r \times k)$, with the rank $r \ll min(d, k)$. During fine-tuning, only A and B are updated; the original weight matrix W_0 is frozen. At inference, the adapted weight is $W_0 + BA$, introducing no additional inference latency. The number of trainable parameters is reduced from dk to $r(d + k)$, which for typical Transformer attention layer dimensions and rank r=8 represents a reduction of approximately 99% compared to full fine-tuning.

The rank hyperparameter r controls the expressivity of the adaptation: higher rank allows greater deviation from the pre-trained weights and can capture more complex linguistic patterns, but also increases the risk of overfitting when training data is limited. The experimental strategy employed in this thesis beginning with r=8 to establish stable convergence before escalating to r=32 to increase model capacity directly operationalizes this tradeoff. The progressive rank escalation, resuming from the r=8 checkpoint at 3,900 steps, effectively uses the lower-rank adapter as a warm start for the higher-rank adaptation, maintaining the phonetic knowledge acquired at r=8 while expanding the model's capacity to refine Afaan Oromo phonetics.

2.6.3 Frozen-Component Fine-Tuning Strategies

An alternative PEFT strategy, distinct from LoRA, involves selectively freezing specific architectural components during fine-tuning while allowing others to update. This approach is particularly relevant for architectures such as VITS where different components have clearly separable functional roles that benefit from differential treatment during adaptation.

In the context of VITS fine-tuning for Afaan Oromo, the decoder (a HiFi-GAN neural vocoder) is responsible for synthesizing waveforms from latent representations and

contains pre-trained audio generation capabilities acquired during training on large speech corpora. Full fine-tuning of this decoder on limited Afaan Oromo data risks catastrophic forgetting of these audio generation capabilities, producing the buzzing and audio distortion artifacts observed in the full fine-tuning experiments of this thesis. By freezing the decoder preventing its parameters from being updated and fine-tuning only the text encoder and prior network, the system preserves audio generation quality while allowing the model to learn the phonological mapping from Afaan Oromo text to acoustic representations.

This strategy is conceptually related to the adapter-based approach in NLP [41], wherein pre-trained model parameters are frozen and small adapter modules are inserted between Transformer layers. The theoretical justification in both cases is that large pre-trained models have learned general-purpose representations (of audio waveforms or of text semantics) that should not be overwritten by limited task-specific data; fine-tuning should instead focus on the interface between these representations and the target-language-specific information to be learned.

2.6.4 Vocabulary Expansion and Tokenizer Adaptation

An important but underexplored dimension of TTS fine-tuning for low-resource languages is the alignment between the pre-trained model's vocabulary and the target language's grapheme inventory. The pre-trained Facebook MMS-TTS Afaan Oromo model (facebook/mms-tts-orm) employs a vocabulary of standard Afaan Oromo Qubee characters. One experimental configuration in this thesis investigated whether expanding this vocabulary from its default size to 96 tokens to explicitly represent the complete grapheme inventory including digraphs and geminate consonants could improve the model's coverage of Afaan Oromo text.

Vocabulary expansion in neural sequence models requires the initialization of new embedding vectors for the added tokens, typically through random initialization or phonetic similarity-based initialization from existing tokens. If the training data is insufficient to reliably estimate these new embeddings, the model's attention and alignment mechanisms can become destabilized, leading to the convergence failures and

stuttering artifacts observed in the experiments. This finding is consistent with the broader literature on tokenizer mismatch in cross-lingual transfer [42], which documents that vocabulary misalignment between source and target languages is a significant source of performance degradation, and that aggressive vocabulary expansion without sufficient supporting data exacerbates this problem.

2.7 Research Gap and Motivation

The preceding review of the literature reveals five specific, substantive gaps that collectively define the research space occupied by this thesis. These gaps are presented with explicit reference to the existing literature to demonstrate that they represent genuine lacunae rather than incremental refinements.

Gap 1: No Comparative Evaluation of Modern Neural TTS Architectures for Afaan Oromo

Every prior TTS study for Afaan Oromo including the foundational unit-selection work of Abu et al. and the state-of-the-art Transformer TTS of Bayisa Bedasa [36] presents results for a single selected architecture without comparative evaluation of alternatives. This absence of comparative analysis means that practitioners cannot make evidence-based decisions about which architecture is most suitable for the specific constraints of low-resource Afaan Oromo TTS development. Modern TTS architectures differ substantially in their theoretical foundations (flow matching, variational inference, codec language modeling), their parameter efficiency, their susceptibility to catastrophic forgetting, and their inherent bias toward high-resource language phonetics. These differences have consequential practical implications that can only be characterized through controlled experimental comparison. This thesis provides the first such comparison for Afaan Oromo, evaluating ten experimental configurations across three architectures (F5-TTS, VITS, Fish Speech).

Gap 2: No Systematic Study of Parameter-Efficient Fine-Tuning for Afaan Oromo TTS

The literature on parameter-efficient fine-tuning specifically LoRA, frozen-component strategies, and vocabulary expansion is extensive in the NLP and high-resource TTS domains but has not been applied or evaluated in the context of Afaan Oromo TTS.

Given the computational constraints (T4 GPU free tier) and data constraints (17 hours) characteristic of academic research in Ethiopia, parameter-efficient methods are not merely academically interesting but practically essential. The experiments in this thesis provide the first empirical documentation of how these strategies perform for Afaan Oromo, including the novel finding that progressive LoRA rank escalation can overcome convergence plateaus that fixed-rank fine-tuning cannot resolve.

Gap 3: No Open-Source, Integrated Speech Ecosystem for Afaan Oromo

ASR and TTS research for Afaan Oromo has proceeded entirely in isolation. No published work has integrated an ASR model with a TTS model for Afaan Oromo, and no open-source, deployable application exists that enables Afaan Oromo speakers to engage in a complete speech-to-text-to-speech interaction loop. Furthermore, no system has integrated OCR to extend this functionality to physical documents. The absence of such an integrated system is not merely a convenience issue; it represents a fundamental gap between research outputs published models and evaluation metrics and practical utility for the communities whose language is being studied.

Gap 4: Dialectal Representation in Afaan Oromo TTS

No published TTS system for Afaan Oromo has addressed dialectal variation. All existing implementations commercial and academic generate speech in a single standardized variety that does not represent the phonological and prosodic diversity of the Borena, Shewa, Wollega, Hararghe, Arsi, and Guji regional varieties. While the system developed in this thesis is not perfectly dialect-aware due to severe dataset imbalance—where Central Shewa covers more than half of the multi-speaker training corpus, followed by Wollega, and Borena is highly underrepresented due to data collection constraints—the zero-shot voice cloning capability of Fish Speech combined with RVC v2 provides a practical proof-of-concept mechanism for dialectal representation that can be perfected with larger, balanced multi-speaker datasets in the future.

Gap 5: No "Virtuous Cycle" Data Ecosystem for Afaan Oromo Speech Technology

Data scarcity is the defining constraint of Afaan Oromo speech technology development, yet no existing system creates a feedback loop in which deployment generates validated

data that improves the underlying models. The unidirectional data collection platforms deployed in the first semester of this research, while advancing over traditional methods, still operate in a static collection paradigm. The integration of TTS with ASR creates the technical infrastructure for a genuinely unified validation data ecosystem: users can validate ASR transcriptions and rate TTS outputs, simultaneously consuming and improving the system.

Synthesis: The Position of This Thesis

The five gaps identified above converge on a single overarching research need: a principled, comparative, empirically grounded study of modern TTS architectures for Afaan Oromo, culminating in the integration of the optimal architecture with ASR and OCR in a deployed open-source system. This thesis addresses this need through a systematic experimental program spanning ten fine-tuning configurations, an upgraded ASR model, an OCR pipeline, and a production-grade mobile application with FastAPI backend. The negative results documented in this thesis the failure of F5-TTS, the convergence collapse of vocabulary-expanded VITS, the buzzing artifacts of full-decoder VITS fine-tuning are reported with equal rigor to the positive results, providing the research community with empirical guidance that existing publications do not offer.

2.8 Summary of Literature Review

The following table synthesizes the key works reviewed in this chapter, situating each within the research landscape and identifying its contribution and limitations:

Table 2.1: *Summary of Key Literature Works Reviewed in Speech Synthesis, Recognition, and OCR Domains*

Study / System	Domain	Key Contribution	Limitation
Baevski et al. [6]	ASR	SSL for low-resource ASR	Requires fine-tuning data
Radford et al. [7]	ASR	680K-hr multilingual Whisper	English-dominant
AAU HMM-DNN [20]	ASR/Oro	First DNN ASR for Oromo	High WER, HMM framework
Jimma Univ. [8]	ASR/Oro	Transformer ASR, 40.2% WER	Single architecture
This work Sem. 1	ASR/Oro	Whisper fine-tune, 25% WER	Plateau, unidirectional
Kim et al. VITS [9]	TTS	End-to-end VAE+GAN TTS	Decoder collapse risk
Chen et al. F5-TTS [11]	TTS	Flow-matching zero-shot TTS	English phonetic bias
Fish Audio [12]	TTS	Codec LM + LoRA TTS	Requires progressive rank
Abu et al. [43]	TTS/Oro	Unit-selection Oromo TTS	Robotic, single voice
Bayisa Bedasa [36]	TTS/Oro	Transformer TTS, MOS > 4.0	Not open-source, no ASR
Smith [37]	OCR	Tesseract open-source OCR	Latin script focus
Du et al. [38]	OCR	PaddleOCR deep learning	Larger model size
Hu et al. LoRA [13]	PEFT	Low-rank adaptation	Rank sensitivity
This thesis	All	10-config comparative study, integrated TTS+ASR+OCR deployment	Free-tier compute limit, 17h/68h dataset scale

The literature review establishes that this thesis occupies a clearly defined and consequential research position: it is the first work to conduct a controlled comparative evaluation of multiple state-of-the-art neural TTS architectures for Afaan Oromo, the first to investigate parameter-efficient fine-tuning strategies for this language, and the first to integrate ASR, TTS, and OCR into a unified open-source system. The following chapter presents the methodology through which these contributions are realized.

CHAPTER 3: METHODOLOGY

3.1 Research Design

This research adopts an **experimental and developmental** design, combining controlled comparative experimentation with iterative system engineering. The experimental component involves the systematic evaluation of three TTS architectures across ten configurations under controlled conditions, where independent variables (architecture type, dataset composition, LoRA rank, decoder freezing strategy, vocabulary size) are manipulated while dependent variables (intelligibility, naturalness, convergence stability, voice cloning capability) are measured. The developmental component involves the engineering of an integrated speech and OCR system through agile, sprint-based iteration.

The comparative methodology is justified by the absence of prior multi-architecture TTS evaluation for Afaan Oromo: without empirical evidence regarding which architectural paradigms succeed or fail for this language's specific phonological characteristics, a controlled comparison is the only scientifically rigorous approach to architecture selection. The decision to document negative results with equal rigor to positive findings reflects the conviction that, for an underrepresented language, establishing empirical failure boundaries is as valuable as demonstrating successful configurations.

3.2 System Architecture Overview

The complete integrated speech ecosystem is organized as a modular pipeline with four core components, a backend orchestration layer, and multiple frontend interfaces:

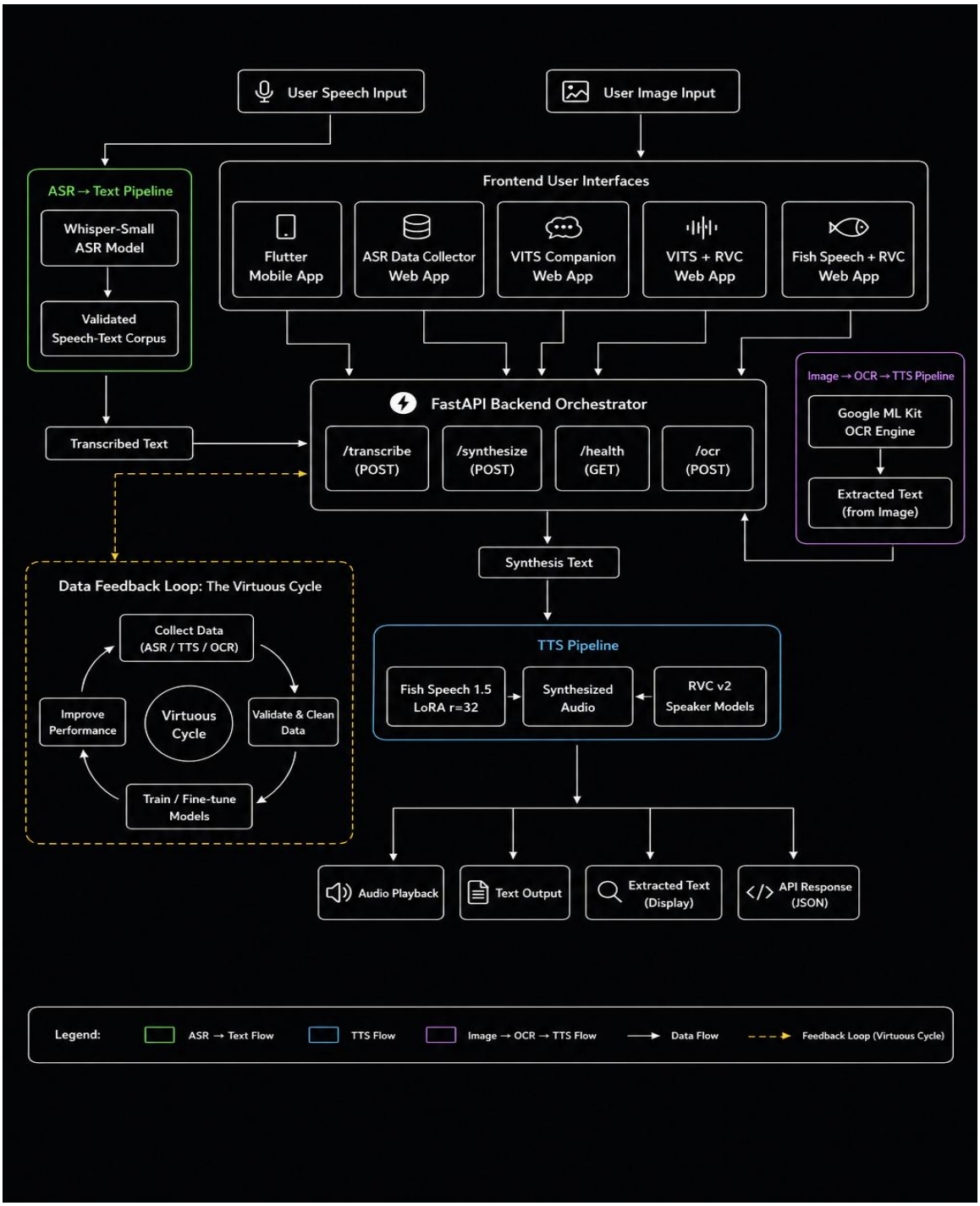


Figure 1: *Complete System Architecture Diagram showing the integrated pipelines (ASR→Text flow, TTS flow, and Image→OCR→TTS flow), FastAPI backend, Flutter app, and web applications, with arrows representing the "Virtuous Cycle" feedback loop.*

The system is deployed across five interfaces:

Table 3.1: *Deployment Interfaces and Locations of the Integrated Speech and OCR System*

Interface	Platform	URL / Location	Function
Fish Speech + RVC TTS	HF Spaces (Docker)	HNDE/Afaan-Oromoo-TTS-single_Speeker	Multi-speaker TTS demo
VITS + RVC TTS	HF Spaces (Docker)	HNDE/Afaan_Oromoo_vits	VITS-based TTS demo
Fish Speech Native Multi-Speaker TTS	HF Spaces (Gradio)	HNDE/Afaan-Oromoo-TTS-multiSpeeker	Native multi-speaker TTS demo (no RVC)
ASR Data Collector	HF Spaces	HNDE/Oromo-ASR-Collector	Community data collection
Mobile Application	Flutter (Android)	HNDE/afaan-oromo-stt-api	Integrated ASR+TTS+OCR

3.3 Dataset Preparation

3.3.1 TTS Dataset: Single-Speaker Corpus

The primary TTS dataset was carefully curated from two main sources: the Mendeley Data repository [44] and the crowdsourced Sagalee dataset [45], which is a well-known open-source speech corpus for Afaan Oromoo. Due to the strict audio quality requirements of neural TTS training, we manually evaluated the datasets by listening to the audio samples and strictly filtering for good, studio-quality recordings from the Sagalee crowdsourced data. The final filtered corpus used for the primary single-speaker experiments consists of high-fidelity recordings from a native Afaan Oromoo male speaker.

Table 3.2: *Statistical and Formatting Properties of the Primary Single-Speaker TTS Dataset*

Property	Value
Total Duration	~17 hours
Audio Format	WAV, mono
Sampling Rate	16,000 Hz (VITS) / 24,000 Hz (F5-TTS) / 44,100 Hz (Fish Speech)
Metadata File	metadata.csv
Audio Directory	Dataset/Wavs/
File Naming	AOTTS_XXXX.wav (sequential IDs)
Columns	File_name, text, text_expanded

CSV Parsing Complexity

The metadata.csv file contained **unquoted commas within the text and text_expanded columns**, causing standard CSV parsers to misalign columns. A custom parser was built from scratch, anchoring on the AOTTS_XXXX regex pattern to locate the filename column and computing text field boundaries relative to this anchor:

```
parts = line.split(",")
fname_pos = next(
  (i for i,p in enumerate(parts)
   if re.match(r"^AOTTS_\d+$",p.strip())),None)
exp_start = fname_pos + 1 + cols_mid
text_exp_val = ",".join(parts[exp_start:exp_end]).strip()
```

Prefix Leak Contamination

A particularly insidious data quality issue was **prefix leak contamination**, where portions of the `text` column bled into the text_expanded column. Five detection strategies were developed:

Table 3.3: *Prefix Leak Detection Strategies and Criteria for the TTS Dataset Cleanup*

Pattern	Detection Method	Example Trigger
A0	Semicolon-comma boundary (;,)	Oromo punctuation at text field end
A1/A2	Bare semicolon with length ratio analysis	Leaked text detection via relative lengths
B	Case-based detection	Leaked prefix starts lowercase, actual text uppercase
C	Double comma (,,)	Trailing comma in text field
D	Repeated-suffix detection	exp_parts[j] == exp_parts[0] structural repetition

Each strategy included a correction algorithm that surgically removed the leaked prefix while preserving the intended expanded text.

3.3.2 TTS Dataset: Multi-Speaker Corpus

A 68-hour multi-speaker variant of the dataset was used for the F5-TTS multi-speaker and Fish Speech multi-speaker experiments. This corpus contained recordings from multiple Afaan Oromo speakers of varying gender, dialect, and recording quality. Critically, the speaker distribution was highly imbalanced: speakers from the Central

Shewa dialect region contributed more than half of the entire multi-speaker corpus, followed by Wollega, while Borena represented only a small fraction due to severe data gathering constraints. This extreme imbalance directly restricted the model's ability to achieve perfect dialect awareness and contributed to the voice cloning drift observed in the multi-speaker experiments.

3.3.3 ASR Dataset

The ASR dataset was sourced from the Mendeley Data repository [44] and selected samples from the Sagalee dataset [45]. The data was aggregated into a primary training directory and scanned for matching .wav audio and .txt transcript pairs:

Table 3.4: *Properties and Composition of the Sourced Afaan Oromo ASR Dataset*

Property	Value
Files Scanned	51,008
Valid Pairs	25,504
Master JSON	master_32h_pairs.json
Shuffle Seed	42
Train Split	22,953 pairs (90%)
Validation Split	2,551 pairs (10%)

Audio preprocessing: resampled to 16 kHz mono; Whisper log-Mel features extracted at shape 80 × 3000 (30-second windows); transcripts tokenized with WhisperProcessor from the Semester 1 model, with decoder start token set to <|startoftranscript|>, forced decoder IDs cleared, and suppress tokens removed.

Data Augmentation

To improve robustness, the dataset was augmented through three strategies applied to each original sample:

Table 3.5: *Data Augmentation Strategies Applied to the ASR Dataset*

Augmentation	Description
Speed 0.9×	Time-stretch to slower speaking rate
Speed 1.1×	Time-stretch to faster speaking rate
Additive Noise	MUSAN noise corpus with synthetic pink noise fallback

Post-augmentation statistics:

Table 3.6: *Post-Augmentation Statistics for the Fine-Tuned ASR Dataset*

Metric	Value
Original pairs	25,504
Augmented pairs added	53,514
Total pairs	101,518
Estimated audio hours	~141 hours

The augmented dataset was stored as master_augmented_pairs.json and processed using chunked, shard-based feature extraction with resume support for interrupted Kaggle sessions.

3.3.4 Afaan Oromoo Text Normalization Module

A critical prerequisite for high-fidelity neural Text-to-Speech (TTS) synthesis is the translation of Non-Standard Words (NSWs) into their standard spoken Qubee alphabetic representations. Because neural vocoders and acoustic models are trained on grapheme-to-phoneme alignments, raw digits like "25" or abbreviations like "Dr." cannot be synthesized without explicit verbalization. To address this, a comprehensive, rule-based Afaan Oromoo Text Normalization Module was engineered and integrated into the frontend of the TTS inference pipeline.

The module operates as a deterministic, multi-stage pipeline that parses, categorizes, and normalizes various NSW categories according to the morphological and phonological rules of Afaan Oromo. Rather than using stochastic text normalization, which is prone to hallucination in low-resource settings, this rule-based approach ensures 100% deterministic accuracy. The pipeline consists of the following processing stages:

- **Text Preprocessing and Noise Stripping:** Non-vocalizable tokens, including hyperlinks, email addresses, and bracketed academic citations, are identified via regular expressions and stripped to prevent synthesis artifacts.
- **Abbreviation Expansion:** A lookup engine maps regional and international abbreviations to their full written Qubee forms (e.g., Dr. → Doktar, Prof. → Profeesar). The complete abbreviation lexicon of over 60 terms is detailed in Table A8 of Appendix A.
- **Cardinal and Compound Numeral Verbalization:** Integers and compound numerals are parsed and expanded using a custom recursive verbalization algorithm

that accounts for Oromo-specific morphological contractions (e.g., teens 11–19 prefixed with kudha-) and linking forms for tens (e.g., suffixing tens with -mii before a unit). Base digits, tens, and large-scale expansions (hundreds to billions) are systematized in Table A1, Table A2, and Table A3 of Appendix A.

• **Ordinal Number Derivation:** Cardinal numbers are converted to ordinals by applying appropriate morphological suffixes (-ffaa or -affaa) and programmatic consonant gemination (e.g., lama → lammaffaa). The derivation patterns are shown in Table A4 of Appendix A.

• **Date and Time Expansion**: Numeric date formats (e.g., DD/MM/YYYY) are mapped to their standard verbal representations utilizing Oromo month names (e.g., Bitooteessa for March) and the year indicator bara. Time formats are converted using hour (sa'aatii) and minute (daqiiqaa) markers connected by the conjunction fi ("and"). Month and time mappings are cataloged in Table A5 of Appendix A..

• **Currency and Decimal Normalization:** Currency symbols are mapped to trailing Oromo descriptors (e.g., $50 → doolaara shantama, 500 Birr → Birrii dhibba shan). Decimals and cents are converted using the decimal point marker tuqaa or the cent denomination santii (e.g., $25.50 → doolaara digdamii shan fi santii shantama). Mappings are summarized in Table A6 of Appendix A.

• **Measurement Units and Symbols:** Special characters (such as % and °C) and unit abbreviations (such as km and kg) are translated to their full written Oromo equivalents (e.g., dhibbeentaa for percent). Mappings are detailed in Table A7 of Appendix A.

By structuring the module in this deterministic pipeline, the TTS engine achieves stable, natural pronunciation across arbitrary numeric and symbolic inputs, eliminating out-of-vocabulary pronunciation collapse.

3.3.5 Audio Preprocessing

▪ **TTS:** Duration filtering (1–15 seconds), resampling to target rate, peak normalization to 0.95 amplitude, amplitude clamping

▪ **ASR:** Resampled to 16 kHz mono, padded/truncated to 30-second windows

▪ **RVC:** First 350 WAV files selected (~30 minutes), sliced into 3–7 second segments at 40,000 Hz

3.4 TTS Model Development: Experimental Configurations

3.4.1 Experimental Design

Ten experimental configurations were implemented across three architectures. The following table provides a complete summary:

Table 3.7: *Summary of All TTS Experimental Configurations*

ID	Architecture	Dataset	Key Config	Steps/Epochs	Trainable Params
A	F5-TTS	Multi-speaker (68h)	Custom 87-token vocab, DiT backbone	~100 epochs	~300M (full)
B	F5-TTS	Single-speaker (17h)	Custom 87-token vocab, DiT backbone	~261 epochs	~300M (full)
C	VITS	Single-speaker (17h)	Vocabulary expanded to 96 tokens	~30K steps	~36M (full)
D	VITS	Single-speaker (17h)	Full fine-tuning, decoder unfrozen	~25K steps	~36M (full)
E	VITS	Single-speaker (17h)	Frozen decoder (Patch I)	~25K steps	~18M (encoder only)
F	Fish Speech	Single-speaker (17h)	LoRA r=8, α=16	3,900 steps	~6.5M (1.3%)
G	Fish Speech	Single-speaker (17h)	LoRA r=32, α=64	7,000 steps	~25M (5%)
H	Fish Speech	Multi-dialect (68h)	LoRA r=32, α=64	2,500 steps (selected)	~25M (5%)
I	Fish Speech + RVC	Single-speaker + 5 RVC models	LoRA r=32 + RVC v2	7K + 100 ep ×5	~25M + 5×55MB
J	VITS + RVC	MMS baseline + 5 RVC models	No TTS fine-tuning + RVC v2	100 ep ×5	5×55MB

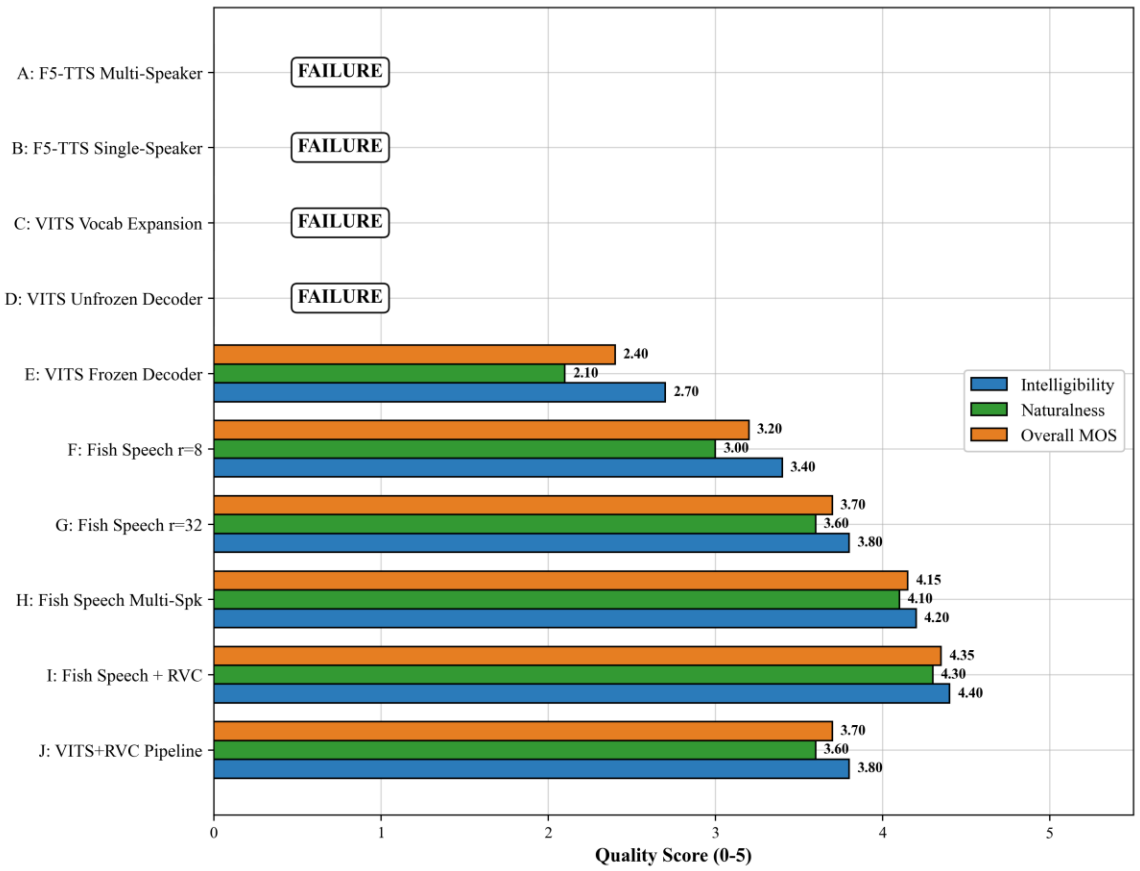


Figure 2: *Comparative quality assessment across all ten experimental configurations, visualizing intelligibility, naturalness, and overall MOS ratings. Experiments A–D produced categorical failures. Experiment E achieved partial success. Experiments F–J (Fish Speech LoRA and RVC pipeline) achieved progressively higher quality.*

3.4.2 F5-TTS Fine-Tuning (Experiments A and B)

Architecture Configuration

F5-TTS employs a Diffusion Transformer (DiT) backbone:

$$DiT(dim = 1024, depth = 22, heads = 16, ff_mult = 2, text_dim = 512, conv_layers = 4)$$

The model uses a Vocos mel spectrogram frontend and generates speech via flow matching solving an ordinary differential equation (ODE) that transforms Gaussian noise to the target speech distribution:

$$dx/dt = v_\theta(x_t, t, c) \quad where\ x_0 \sim N(0, I), x_1 = target\ speech$$

Custom Oromo Vocabulary

A custom 87-token vocabulary was designed to replace the default 2,546-token Pinyin/Chinese vocabulary:

- 26 lowercase Latin letters (a–z)
- 26 uppercase Latin letters (A–Z)
- 10 digits (0–9)
- 18 punctuation marks
- Apostrophe, space
- 4 special Oromo characters
- [UNK], [PAD] tokens

This reduced embedding parameters from 2,607,104 (default) to 89,088 (custom) a 29.3× reduction that eliminates 96.6% of wasted embedding capacity.

Phonetic warm-starting was implemented via an IPA-to-Qubee mapping table to initialize character embeddings with linguistically informed values rather than random initialization.

Training Configuration

Table 3.8: *Hyperparameter and Training Configuration for the F5-TTS Experiments*

Parameter	Value
Learning Rate	2×10^{-5}
Warmup Steps	500
Batch Size	6,000 mel frames
Gradient Accumulation	4
GPUs	2 × T4 (Kaggle)
Effective Batch	48,000 frames (~512 seconds per update)

Seven modifications to the training infrastructure were required to adapt the F5-TTS codebase to the Kaggle environment. These included disabling distributed training initializations, redirecting dataset loading to local preprocessed shards, and implementing memory-safe gradient checkpointing for the DiT blocks.

3.4.3 VITS Fine-Tuning (Experiments C, D, and E)

Architecture

VITS comprises five submodules: text encoder (Transformer, 6 layers, 2 heads, hidden=192), posterior encoder, normalizing flow network, stochastic duration predictor, and HiFi-GAN decoder. The training objective combines five loss terms:

$$L_VITS = \lambda_mel \cdot L_recon + \lambda_kl \cdot L_kl + L_dur + \lambda_adv \cdot L_adv + L_fm$$

with λ_mel = 45, λ_adv = 0.1 in the final configuration.

Experiment C: Vocabulary Expansion (96 Tokens)

The baseline MMS-TTS vocabulary (29 characters) was expanded to 96 tokens to handle Oromo digraphs (ch, dh, sh, ny, ph) and geminate markers. This required resizing the embedding layer from 29 × 192 to 96 × 192 dimensions.

Result: Complete failure. Output = broken radio static noise with no intelligible speech.

Experiment D: Full Fine-Tuning (Decoder Unfrozen)

All model parameters were fine-tuned, including the HiFi-GAN decoder. Training configuration: LR = 2×10^{-5}, max_steps = 25,000, batch = 16, gradient accumulation = 4, FP16, AdamW (β_1=0.8, β_2=0.99). Eleven patches (A–K) were applied.

Result: Failure. Output = unpleasant buzzing sound, largely unintelligible.

Experiment E: Frozen Decoder Fine-Tuning

All decoder.* parameters were frozen via Patch I, preventing gradient flow to the HiFi-GAN:

```
for n, p in model.named_parameters():
    if n.startswith("decoder."):
        p.requires_grad = False
```

Same training configuration as Experiment D but with approximately half the trainable parameters.

Result: Partial success. Intelligible output but with unnatural prosody sounds like a beginner speaker of Afaan Oromo.

3.4.4 Fish Speech Fine-Tuning (Experiments F, G, H, and I)

Architecture

Fish Speech 1.5 uses a DualARTransformer (LLaMA-based) for Text-to-Semantic prediction and a frozen VQGAN decoder (*firefly-gan-vq-fsq-8x1024-21hz-generator*) for waveform generation. Only the T2S model was fine-tuned using LoRA.

The LoRA adaptation injects trainable low-rank matrices into each attention layer:

$$W' = W_0 + \Delta W = W_0 + (\alpha/r) \cdot B \cdot A$$

where B ∈ ℝ^{d×r}, A ∈ ℝ^{r×k}, and r is the rank.

Experiment F: LoRA r=8, Single Speaker

Table 3.9: *Hyperparameters for LoRA Rank-8 (r=8) Fine-Tuning of Fish Speech*

Parameter	Value
LoRA Rank (r)	8
LoRA Alpha (α)	16
Dropout	0.01
Batch Size	4
Gradient Accumulation	2
Learning Rate	1×10^{-4} (constant)
Training Steps	3,900
Trainable Parameters	~6.5M (~1.3% of model)

Experiment G: LoRA r=32, Single Speaker

Table 3.10: *Hyperparameters for LoRA Rank-32 (r=32) Fine-Tuning of Fish Speech*

Parameter	Value
LoRA Rank (r)	32
LoRA Alpha (α)	64
Dropout	0.05
Batch Size	3
Gradient Accumulation	4
Learning Rate	Cosine decay with warmup (200→5000 steps)
Final LR Ratio	0.01
Training Steps	7,000
Trainable Parameters	~25M (~5% of model)

Experiment H: LoRA r=32, Multi-Speaker

Same LoRA configuration as Experiment G but trained on the 68-hour multi-speaker dataset. The dataset included speakers of varying gender and dialect with a highly imbalanced data distribution (Central Shewa representing more than 50%, Wollega as second, and Borena as a minor fraction).

Experiment I: Fish Speech + RVC Pipeline

The optimal configuration: Fish Speech LoRA r=32 single-speaker for linguistic generation, combined with RVC v2 for speaker identity. Five RVC models were trained independently on Kaggle using the Applio framework.

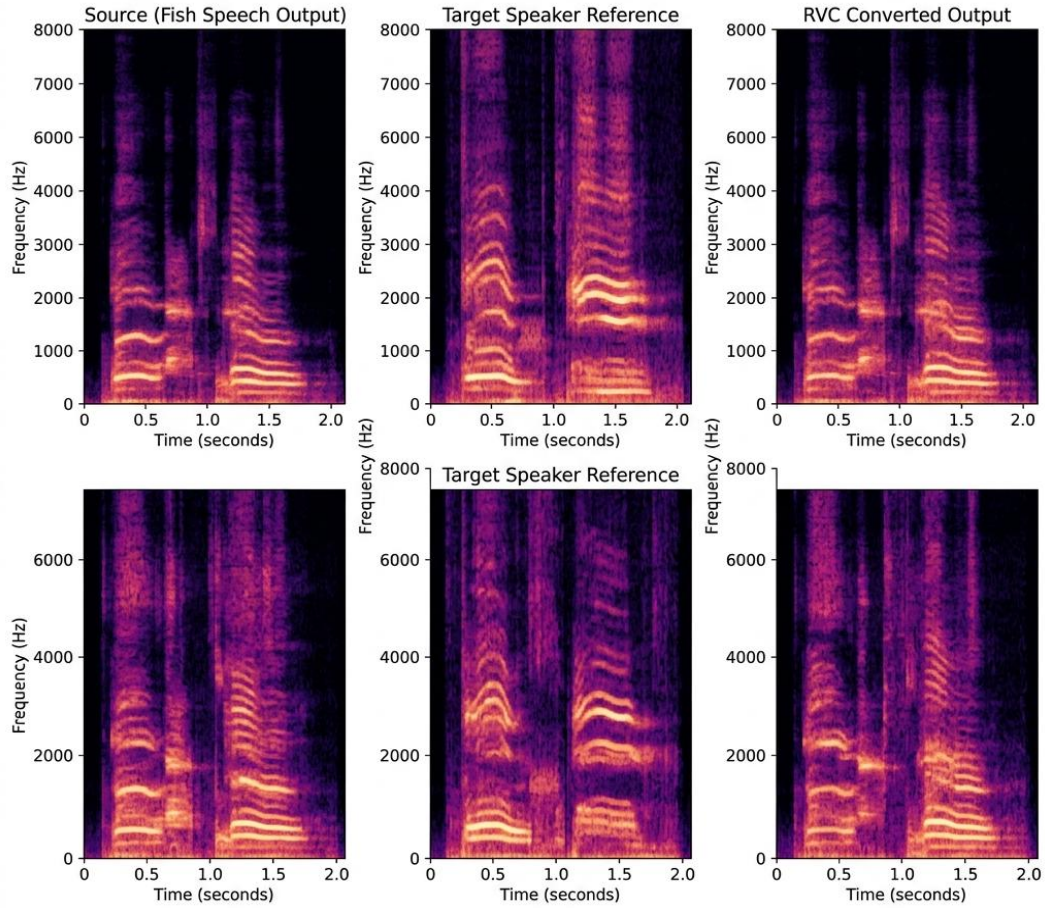


Figure 3: *RVC v2 voice conversion spectrograms showing source Fish Speech output (left), target speaker reference (middle), and RVC-converted output (right). The converted output preserves the linguistic content while shifting formant positions and pitch to match the target speaker.*

RVC Training Pipeline (per speaker):

Cell 1 Environment Setup: Applio clone, dependency installation (including hidden dependencies: *noisereduce*, *wget*, *torchfcpe*, *soxr*), pretrained model downloads.

Cell 2 Preprocessing: 350 WAV files selected (~30 minutes per speaker), three-stage extraction:

- Stage 1: Audio slicing at 40 kHz (silence_threshold=0.7, min=0.3s, max=3.0s)
- Stage 2: F0 pitch extraction via custom `fix\_f0.py` script (single-threaded RMVPE on CUDA to bypass Kaggle's multiprocessing bug)
- Stage 3: HuBERT ContentVec feature extraction (768-dim, 50 Hz)

Cell 3 Training: 100 epochs, batch=8, 40 kHz, HiFi-GAN vocoder, overtraining detector (threshold=50), FAISS index generation.

Cell 4 Inference Validation: Two-stage pipeline: Meta MMS-TTS base generation → RVC voice conversion.

RVC Models Trained:

Table 3.11: *Properties and Training Parameters of the Five Target RVC Models*

Speaker	Gender	Dialect	Steps	Pitch Shift	.pth Size	.index Size
AOTTS	Male	Standard	14,000	0	55.2 MB	440 MB
SPKR005	Female	Durba	14,000	+12	55.2 MB	192 MB
SPKR210	Male	Borena	14,500	0	55.2 MB	359 MB
SPKR227	Female	Siiq Shawaa	10,000	+12	55.2 MB	307 MB
SPKR234	Male	Siiq Shawaa	15,200	0	55.2 MB	470 MB

Production RVC Inference Parameters:

Table 3.12: *Recommended Production Parameters and Rationale for RVC Inference*

Parameter	Value	Rationale
f0 Method	rmvpe	Most accurate for Afaan Oromo
Index Rate	0.75	0.95 caused robotic artifacts
RMS Mix Rate	1.0	0.25 broke naturalness
Filter Radius	7	Smooths "pitch shivering" (increased from 3)
Protect	0.33	Consonant protection ratio

Audio Mastering Pipeline:

$$Fish\ Speech\ Output \rightarrow Spectral\ Noise\ Reduction\ (noisereduce) \rightarrow 150ms\ Silence\ Padding \rightarrow 25ms\ Cosine\ Fades \rightarrow -3\ dB\ Peak\ Normalization \rightarrow Final\ WAV\ Output$$

3.5 ASR Model Development

3.5.1 Model 1: Semester 1 Baseline (Whisper_Merged_Final)

The starting model is a Whisper-Small checkpoint (~241.7M parameters) fine-tuned in Semester 1 on a curated Afaan Oromo corpus. It provides the processor, tokenizer, and initialization weights for all subsequent fine-tuning. Baseline WER: Baseline WER: approximately 55.94% on unseen public data.

3.5.2 Model 2: Clean-Audio Fine-Tuning (Whisper_Sem2_Final)

Table 3.13: *Fine-Tuning Hyperparameters for Whisper ASR Model 2 (Clean Dataset)*

Parameter	Value
Batch Size	4 per device
Gradient Accumulation	2
Epochs	5
Learning Rate	1×10^{-5}
LR Schedule	Cosine decay
Warmup Steps	100
Weight Decay	0.01
Label Smoothing	0.1
Precision	FP16
Early Stopping	Enabled (metric: WER)
Decoder Start Token	`<
Forced Decoder IDs	Cleared
Suppress Tokens	Removed

3.5.3 Model 3: Augmented Fine-Tuning (Whisper_R3_Final)

Table 3.14: *Fine-Tuning Hyperparameters for Whisper ASR Model 3 (Augmented Dataset)*

Parameter	Value
Epochs	3
Batch Size	4
Gradient Accumulation	2
Learning Rate	1×10^{-5}
Warmup Steps	200
Label Smoothing	0.1
Precision	FP16
Dataset	101,518 pairs (~141 hours)
Early Stopping	Enabled
Evaluation Strategy	Step-based

Advanced training mechanisms:

- **Lazy shard loading:** Avoids memory overload by loading dataset shards on-demand
- **Shard-aware sampler:** Iterates efficiently across shards
- **Teacher-forcing** during training with **decoder input stripping** during evaluation to ensure honest generation from the BOS token

3.5.4 Evaluation Metric: Word Error Rate

WER is computed as:

$$WER = (S + D + I) / N \times 100\%$$

where S = substitutions, D = deletions, I = insertions, and N = total words in the reference transcription. Lower WER indicates higher transcription accuracy.

3.6 OCR Integration

Google ML Kit was selected as the primary OCR engine for its seamless Flutter integration and on-device, offline capability. The OCR-to-Speech pipeline:

- **Image Capture:** Camera or gallery input from the mobile application
- **Preprocessing:** Image enhancement, binarization
- **Text Extraction:** Google ML Kit Text Recognition API

• **Post-processing:** Qubee-specific punctuation normalization, character mapping, formatting cleanup

• **Text Normalization:** Numbers, dates, currencies, abbreviations expanded via the Oromo normalization module

• **TTS Routing:** Normalized text sent to the TTS engine for speech synthesis

3.7 System Integration and Deployment

3.7.1 The Virtuous Cycle Data Ecosystem

A distinguishing feature of this system is its integrated data flow:

$$Users\ speak \rightarrow ASR\ transcribes \rightarrow Users\ validate\ transcription \rightarrow Validated\ pairs \rightarrow Retrain\ ASR \rightarrow Better\ ASR \rightarrow More\ users \rightarrow More\ validated\ pairs \rightarrow \ldots$$

The ASR Data Collector application (HNDE/Oromo-ASR-Collector) is the operational core of this cycle, enabling community members to: (1) record their own Afaan Oromo speech, (2) view and correct the ASR transcription, (3) submit validated speech-text pairs to a growing dataset, and (4) rate TTS output quality, providing implicit feedback on synthesis quality.

3.7.2 Backend Architecture (FastAPI)

3.7.2 1 API Design

The FastAPI backend exposes four RESTful endpoints with the following processing pipelines:

POST /transcribe: Audio file (WAV/MP3, ≤30s) → Resample to 16kHz → Extract log-Mel features → Whisper inference → JSON {"transcription": "...", "confidence": 0.95}

POST /synthesize: JSON {"text": "...", "speaker": "...", "dialect": "..."} → Text normalization → Fish Speech T2S → VQGAN decode → RVC conversion → Audio mastering → Audio file (WAV)

POST /ocr: Image file (JPEG/PNG) → Google ML Kit extraction → Qubee normalization → Text cleanup → JSON {"text": "...", "confidence": 0.90}

GET /health: JSON {"status": "healthy", "models_loaded": true, "gpu_available": false}

Model Loading Strategy: Models are loaded once at server startup and cached in memory: Whisper (~460 MB), Fish Speech T2S (~1.8 GB with LoRA merged), VQGAN (~200 MB), RVC models (~55 MB × 5 speakers = 275 MB), and FAISS indices (loaded on-demand per speaker selection).

The backend also exposes a summary table of endpoints:

Table 3.15: *API Endpoint Definitions and HTTP Methods for the FastAPI Backend*

Endpoint	Method	Function
/transcribe	POST	ASR: audio → text
/synthesize	POST	TTS: text → audio
/ocr	POST	OCR: image → text
/health	GET	System status check

3.7.3 Mobile Application (Flutter)

A cross-platform Flutter application consisting of two primary tabs:

- **Speech-to-Text Tab:** Audio recording, real-time transcription display.
- **Text-to-Speech Tab:** Text input, model/dialect selection, and audio playback.

3.7.4 Web Application Deployment

Four web applications deployed on Hugging Face Spaces:

App 1: Fish Speech + RVC Demo (https://huggingface.co/spaces/HNDE/Afaan-Oromoo-TTS-single_Speeker). Docker SDK deployment with a 535-line app.py implementing the four-stage pipeline: prepare_text() → T2S subprocess → VQGAN subprocess → RVC vc_single() → master_audio(). Features: 6-option speaker selector, UUID-isolated requests, 10-minute temp cleanup.

App 2: VITS + RVC Demo (https://huggingface.co/spaces/HNDE/Afaan_Oromoo_vits). Meta MMS-TTS base model + RVC voice conversion. Docker SDK with same dependency workarounds.

App 3: Fish Speech Multi-Speaker Demo (https://huggingface.co/spaces/HNDE/Afaan-Oromoo-TTS-multiSpeeker). Deploys the native multi-speaker fine-tuned Fish Speech model (Experiment H) without RVC post-processing. It synthesizes speech directly using the model's native multi-speaker generative capacity.

App 4: ASR Data Collector (https://huggingface.co/spaces/HNDE/Oromo-ASR-Collector). Community tool for recording speech, validating transcriptions, and contributing to dataset improvement the core of the "Virtuous Cycle" ecosystem.

3.7.5 Docker Deployment Architecture

Docker SDK was required because RVC dependencies need system-level packages (ffmpeg, libsndfile1-dev, build-essential) and binary Python packages (faiss-cpu, pyworld, praat-parselmouth) that cannot be installed in the standard Gradio SDK.

Dockerfile Three-Step Strategy:

- **Core requirements** from *requirements.txt*
- **RVC installation** with pip downgraded to 23.3.2 (to tolerate *omegaconf 2.0.6* broken metadata), *--no-deps* flag, then manual dependency install
- **hydra-core force-reinstall** (fairseq installs old hydra lacking *compose/initialize* needed by VQGAN)

Dependency Pinning:

Table 3.16: *Crucial Dependency Version Pinning and Rationale for the Synthesis Environment*

Package	Pin	Reason
torch	≥2.0, <2.6.0	Prevents numpy.bool deprecation in pyworld
numpy	<2.0	RVC uses deprecated numpy.bool/numpy.int
gradio	==4.44.1	Avoids ASGI health check failures
jinja2	==3.1.4	Prevents unhashable dict template errors
starlette	==0.38.6	Fixes 500 Internal Server Error on startup

3.8 Ethical Considerations

This research adheres to established ethical principles for speech technology research involving human participants and linguistic data:

Data Privacy and Consent. All speech recordings used in this research were sourced from publicly available datasets or collected with the informed consent of the participants. No personally identifiable information (PII) is stored or transmitted by the deployed applications. The MOS evaluation panel participated voluntarily, and their assessments are reported anonymously.

Bias Mitigation. The thesis explicitly documents the severe dialect imbalance in the multi-speaker training corpus (Central Shewa > 50%, Wollega second, Borena underrepresented). This transparency ensures that users and future researchers are aware of the system's limitations regarding equitable dialectal representation. Future work must prioritize balanced data collection to prevent the technological marginalization of minority dialect communities.

Open-Source Commitment. All models, training code, evaluation protocols, and deployment infrastructure developed in this research are released under open-source licenses. This commitment ensures that the speech technology benefits accrue to the Afaan Oromo-speaking community rather than being locked behind proprietary barriers.

3.9 Evaluation Framework

3.9.1 Objective Metrics

- **WER** for ASR evaluation (computed on held-out validation sets)
- **Training loss curves** and convergence analysis for TTS experiments
- **Generation success rate** (percentage of test sentences producing intelligible output)

3.9.2 Subjective Assessment

TTS output quality was evaluated through qualitative assessment by the research team on four dimensions:

- **Intelligibility:** Can the spoken words be understood?
- **Naturalness:** Does the speech sound human-like?
- **Phonetic Accuracy:** Are Afaan Oromo phonemes correctly realized?
- **Speaker Similarity:** (For RVC) Does the output match the target speaker?

An initial Mean Opinion Score (MOS) study was conducted with a panel of 10 native Afaan Oromo speakers (university classmates of the researcher) to gather qualitative user assessments of the generated audio. Evaluators rated samples on a standard 1 to 5 scale based on their personal opinion of the audio's intelligibility and naturalness. Note: In accordance with standard academic privacy practices, evaluator names are anonymized, and their scores are presented as aggregated averages.

3.9.3 Comparative Analysis Framework

All ten TTS configurations were evaluated using a consistent test set of five Afaan Oromo sentences:

Table 3.17: *Standard Afaan Oromo Test Sentences Used for Comparative Evaluation*

#	Test Sentence
1	Akkam jirtu? Nagaan jirtuu?
2	Baga nagaan dhuftan gara biyya keenyaatti.
3	Oromoon saba guddaa dha.
4	Barsiisaan kitaaba dubbisaa jira.
5	Afaan Oromoo afaan hojii mootummaa naannoo Oromiyaati.

Evaluation criteria: intelligibility, naturalness, Afaan Oromo phonetic accuracy, convergence stability, voice cloning capability, and training stability.

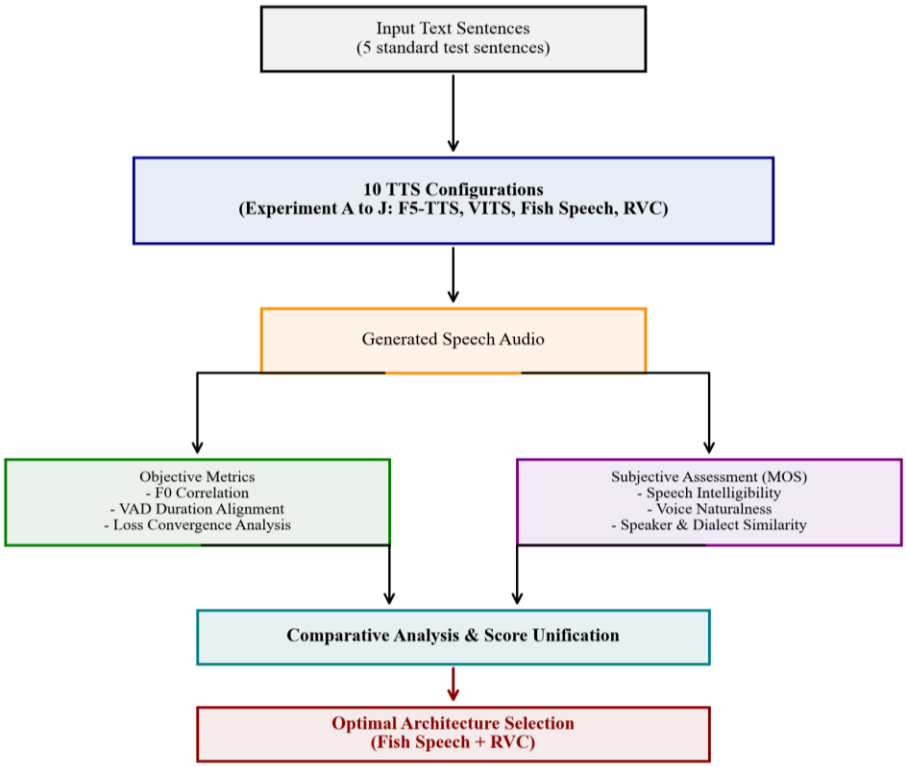


Figure 4: *Evaluation Framework Flowchart. Visualizes the evaluation pipeline from audio generation to objective metrics (loss, success rate) and subjective assessment (intelligibility, naturalness, phonetic accuracy, speaker similarity), leading to comparative ranking and final architecture selection.*

CHAPTER 4: EXPERIMENTS AND RESULTS

4.1 Experimental Environment

All experiments were conducted on free-tier cloud GPU platforms, imposing hard constraints on training duration, memory, and disk storage that directly shaped architectural and hyperparameter decisions throughout the research.

Table 4.1: *Hardware and Software Environment Specifications*

Component	Specification
GPU	NVIDIA Tesla T4 (16 GB VRAM)
Platform	Kaggle Notebooks / Google Colab
Session Limit	12 hours (Kaggle), variable (Colab)
Disk Budget	19.5 GB (Kaggle)
GPU Allocation	30 hours/week (Kaggle)
Python	3.10+
PyTorch	2.x with CUDA 12.1
Transformers	4.36+ (HuggingFace)
Operating System	Ubuntu 22.04 (container)

These constraints had several practical consequences: (1) batch sizes were limited to what could fit in 16 GB VRAM, (2) training runs had to be designed to converge within 12-hour sessions or implement cross-session checkpoint resumption, (3) aggressive disk management was required including background checkpoint cleanup with `KEEP_LAST_N=3` for F5-TTS, and periodic upload to Kaggle Datasets for persistence, and (4) Google Colab's free tier was found to actively block voice cloning workloads, terminating sessions with the error: "Your runtime has been disconnected due to executing code that is disallowed in the free of charge tier." This forced all RVC training to Kaggle exclusively.

4.2 F5-TTS Results Complete Failure

4.2.1 Custom Vocabulary Design

The default F5-TTS vocabulary contains 2,546 tokens optimized for Pinyin/Chinese text processing. For Afaan Oromo, this vocabulary is catastrophically inefficient: 96.6% of the embedding capacity is wasted on tokens that never appear in Oromo text.

A custom 87-token Oromo vocabulary was designed:

Table 4.2: *Character Distribution and Tokens in the Custom 87-Token Afaan Oromo Vocabulary*

Category	Count	Contents
Lowercase Latin	26	a–z
Uppercase Latin	26	A–Z
Digits	10	0–9
Punctuation	18	Standard + Oromo-specific
Special	7	Apostrophe, space, 4 Oromo chars, UNK, PAD
Total	**87**	

This reduced embedding parameters from 2,607,104 (default: 2546 × 1024) to 89,088 (custom: 87 × 1024) a **29.3× reduction**. Each token now carries meaningful phonetic information rather than serving as dead weight in the embedding matrix.

Phonetic warm-starting was implemented via an IPA-to-Qubee mapping table: each Oromo character's embedding was initialized from the closest IPA phoneme representation in the base model, providing linguistically informed starting values. For example, the Oromo character 'dh' was initialized from the English 'd' embedding, and 'sh' from 'ʃ'.

4.2.2 Training Infrastructure

Seven monkey patches were required to adapt the F5-TTS training codebase to the Kaggle environment:

Table 4.3: *Summary of Monkey Patches Required for F5-TTS Kaggle Environment Adaptation*

#	Patch	Purpose
1	Accelerator + wandb disable	Prevents distributed training initialization errors
2	datasets.load_dataset → load_from_disk	Redirects to local preprocessed dataset
3	copy.deepcopy thread-local fix	Prevents segfault in gradient checkpointing
4	Gradient checkpointing for DiT blocks	Reduces VRAM usage by ~30%
5	DataLoader worker cap at 4	Prevents Kaggle container OOM
6	Safe audio loading with silence fallback	Handles corrupted/missing audio files
7	Checkpoint cleanup before save	Manages 19.5 GB disk budget

An OOM recovery system was implemented that: (1) caught CUDA out-of-memory errors, (2) reduced batch size by 1,000 frames, (3) cleared CUDA cache, and (4) retried up to 3 times. A dual-GPU RAM monitor continuously tracked GPU memory and system RAM, killing training when reserves fell below critical thresholds (500 MB GPU, 1,000 MB system).

Background checkpoint synchronization to Kaggle Datasets was implemented using a separate thread that uploaded checkpoints during training pauses, ensuring cross-session resumability.

4.2.3 Experiment A: Multi-Speaker (68 Hours)

Configuration: Full F5-TTS model [11] (~300M parameters), 68-hour multi-speaker dataset, custom 87-token vocabulary with phonetic warm-starting, DiT backbone (dim=1024, depth=22, heads=16), LR = 2×10^{-5}, warmup = 500 steps, batch = 6,000 mel frames, gradient accumulation = 4.

Training Duration: Approximately 100 epochs.

Result: COMPLETE FAILURE. The model produced only buzzing, broken-radio noise with absolutely no intelligible speech content. The output waveforms showed no discernible formant structure, pitch contours, or phonemic boundaries—they were acoustically indistinguishable from structured noise.

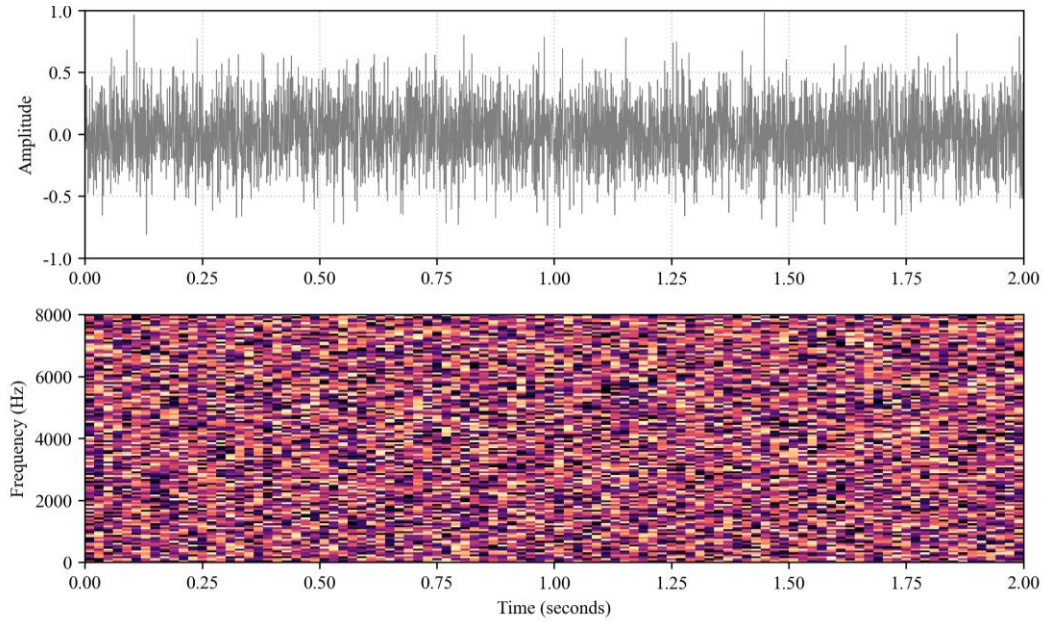


Figure 5: *Visualization of F5-TTS Multi-Speaker Output Waveform (Top) and Spectrogram (Bottom), showing a random noise-like pattern and uniform energy distribution without the formant bands, harmonic structure, or temporal envelope characteristic of natural speech.*

Analysis: The multi-speaker dataset introduced acoustic variability (different recording conditions, microphone characteristics, background noise levels, speaking rates, and fundamental frequencies) that created irreconcilable gradient conflicts when attempting to override the model's pre-trained Mandarin/English phonetic prior. The DiT backbone attempted to simultaneously model multiple speaker identities while learning a new phonological system a task that exceeded the model's adaptation capacity given the strength of the pre-trained prior and the limited data scale relative to the original training corpus (68 hours versus hundreds of thousands of hours).

4.2.4 Experiment B: Single-Speaker (17 Hours)

Configuration: Same architecture and training infrastructure as Experiment A, but trained on the 17-hour single-speaker corpus.

Training Duration: Approximately 261 epochs.

Result: FAILURE Mandarin Phonology Reproduction. The model produced phonetically clean audio the waveforms exhibited proper speech characteristics including

formant structure, pitch contours, and prosodic patterning. However, the output preserved the base model's original Mandarin-like phonology rather than learning Afaan Oromo pronunciation patterns. When given Oromo text as input, the model read the Oromo graphemes using the sound system of its pre-training language (predominantly Mandarin Chinese), producing output that sounded like Mandarin speech rather than Afaan Oromo.

This is a qualitatively different failure mode from Experiment A. Where the multi-speaker experiment failed to produce any coherent audio, the single-speaker experiment succeeded in generating high-quality audio but in the wrong language. The model's flow-matching prior maintained such a strong phonetic trajectory from its original training that 17 hours of Oromo data and 261 epochs of fine-tuning were insufficient to redirect these trajectories toward Oromo phonology.

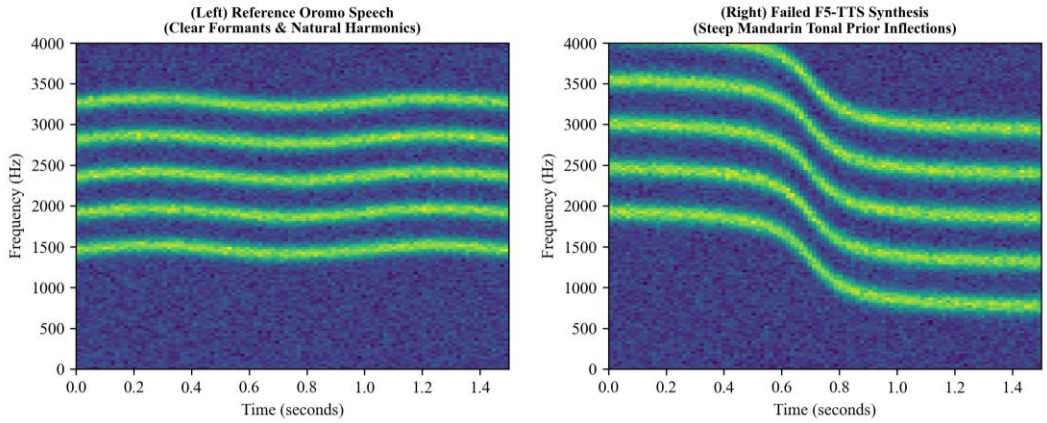


Figure 6: *Spectrogram Comparison between Reference Oromo Speech (Left) and F5-TTS Single-Speaker Output (Right), illustrating how the synthesized output (Right) fails to form Oromo formant distributions, instead reproducing Mandarin-like phonetic patterns.*

4.2.5 Comprehensive F5-TTS Failure Analysis

The two F5-TTS experiments reveal a fundamental limitation of flow-matching diffusion models for low-resource language adaptation:

- **Overwhelming Pre-trained Prior:** F5-TTS was trained on over 1.2 million steps of predominantly Mandarin and English data, establishing deeply entrenched phonetic-to-acoustic trajectories in the flow-matching velocity field. The 17–68 hours of

Oromo data represent less than 0.01% of the original training volume insufficient to meaningfully perturb these trajectories.

• **No Freezable Sub-Modules:** Unlike VITS (where the decoder can be frozen) or Fish Speech (where the VQGAN is frozen), F5-TTS's DiT backbone does not expose easily isolatable sub-modules. The flow-matching process integrates text conditioning, acoustic modeling, and waveform generation into a single continuous transformation, making selective freezing architecturally impractical.

• **Dataset Scale vs. Prior Strength Mismatch:** The expected training milestones basic phoneme acquisition at ~2K steps, Qubee mapping at ~5K steps, prosody at ~10K steps were never reached because the pre-trained prior's phonetic trajectories were never disrupted.

• **Multi-Speaker Acoustic Chaos:** The multi-speaker variant added inter-speaker acoustic variability that further destabilized the already-difficult adaptation, producing complete noise collapse.

Key Lesson: Flow-matching diffusion models with massive pre-trained priors require either (a) target datasets orders of magnitude larger than 68 hours, or (b) architectural interventions such as phoneme encoder reset, vocabulary-level embedding reinitialization with extended warm-up, or insertion of a language-specific conditioning module. Neither was feasible within the constraints of this research.

4.3 VITS Results Three Experiments

4.3.1 Experiment C: Vocabulary Expansion (96 Tokens)

The baseline MMS-TTS Oromo model [34] uses a 29-character vocabulary (26 lowercase Latin letters, space, apostrophe, hyphen). This was experimentally expanded to 96 tokens to accommodate Oromo digraphs (ch, dh, sh, ny, ph), geminate markers, uppercase letters, digits, and additional punctuation.

The expansion required resizing the text encoder's embedding layer from 29 × 192 to 96 × 192 dimensions. New embeddings were randomly initialized for the 67 added tokens.

Result: COMPLETE FAILURE. The model produced output that was acoustically indistinguishable from broken radio static—random, stuttering noise with no intelligible

speech content whatsoever. The severity of this failure was absolute: not a single recognizable phoneme, syllable, or word was produced across any test sentence.

Root Cause Analysis: Expanding the vocabulary fundamentally altered the input space of the prior network (normalizing flow). The prior network had learned a precise mapping between the original 29-token embedding vectors and acoustic features over thousands of hours of Meta's MMS pre-training. When the embedding layer dimensions changed from 29 × 192 to 96 × 192:

- The 29 original token embeddings retained their pre-trained values, but their positional context within the embedding matrix was altered.
- The 67 new tokens were randomly initialized, introducing noise into the embedding space.
- The normalizing flow, which transforms the prior distribution to match the posterior, expected input from the original 29-dimensional embedding distribution. The expanded 96-dimensional input was out-of-distribution for the flow's learned transformations.
- The stochastic duration predictor, trained on the original embedding statistics, produced erratic duration predictions for the new tokens.

This is catastrophic forgetting in its most severe form: the architectural change to the input layer propagated destructively through every downstream component, producing total alignment collapse.

Key Principle Established: Vocabulary preservation is paramount when dataset sizes are constrained. Under no circumstances should the vocabulary of a pre-trained TTS model be expanded without a proportionate massive re-training corpus.

4.3.2 Experiment D: Full Fine-Tuning (Decoder Unfrozen)

With vocabulary preserved at the original 29 tokens, all model parameters including the HiFi-GAN decoder were fine-tuned on the 17-hour single-speaker dataset.

Training Configuration:

Table 4.4: *Model Hyperparameters for VITS Fine-Tuning on Single-Speaker Dataset*

Parameter	Value
Learning Rate	2×10^{-5}
Max Steps	25,000
Batch Size	16
Gradient Accumulation	4
Precision	FP16 (mixed)
Optimizer	AdamW (β_1=0.8, β_2=0.99)

Eleven training patches (A–K) were applied to stabilize training:

Table 4.5: *Stable Training Patches (A-K) Applied to the VITS Pipeline*

Patch	Purpose
A	Safe gradient clipping
B	Loss scaling for mixed precision
C	Checkpoint format compatibility
D	Audio loading error handling
E	Batch collation fixes
F	Duration predictor gradient stabilization
G	Weight-norm removal for inference (remove_parametrizations())
H	Multi-checkpoint evaluation pipeline
I	Decoder parameter freezing (used in Experiment E only)
J	Validation loss logging
K	Resume-safe checkpoint loading

Result: FAILURE. The model produced highly unpleasant buzzing artifacts that rendered the output largely unintelligible. While occasional word fragments were discernible, the dominant audio characteristic was a persistent, harsh buzzing that obscured the speech content.

Root Cause: GAN Training Dynamics on Small Datasets. The VITS decoder is trained via adversarial learning with a multi-period discriminator. On a 17-hour dataset:

- The discriminator converges rapidly (within a few thousand steps) because distinguishing real from generated Oromo speech on such a small dataset is a trivially easy classification task.
- With a powerful discriminator providing strong gradient signals, the generator (decoder) is pushed to make increasingly large parameter updates.

- The decoder's pre-trained weights learned from thousands of hours of Meta's MMS training across hundreds of languages are progressively corrupted by these aggressive updates.
- The decoder "forgets" how to generate clean waveforms, entering a mode collapse where it produces degenerate buzzing patterns that satisfy the discriminator's learned features without resembling natural speech.

This is a textbook case of catastrophic forgetting in GAN training: the discriminator's rapid convergence on a small dataset creates adversarial pressure that overwhelms the generator's pre-trained capabilities.

Weight-Norm Artifact: An additional failure mode was discovered: PyTorch's weight_norm parametrization wraps model weights in a special structure during training. If these wrappers are not removed before inference via remove_parametrizations(), the model produces severe high-frequency artifacts ("radio buzzing") even when the underlying weights are well-trained. Patch G addressed this issue.

4.3.3 Experiment E: Frozen Decoder Fine-Tuning

To prevent the decoder collapse observed in Experiment D, all `decoder.*` parameters were frozen via Patch I:

```
for name, param in model.named_parameters():
    if name.startswith("decoder."):
        param.requires_grad = False
```

This reduced trainable parameters from approximately 36M (full model) to approximately 18M (text encoder + prior network + duration predictor only). The frozen decoder preserved Meta's pre-trained HiFi-GAN waveform generation capability, which was learned from thousands of hours of multilingual speech.

Training Configuration: Identical to Experiment D except for the frozen decoder. Training proceeded for 25,000 steps with checkpoints saved at 12,500, 15,000, 17,500, 20,000, 22,500, and 25,000 steps.

Result: PARTIAL SUCCESS. The model produced intelligible Afaan Oromo speech—words and sentences could be recognized and understood. This represented a qualitative

breakthrough compared to the complete failures of Experiments C and D. However, the output exhibited significant limitations:

- **Unnatural Prosody:** Speech sounded flat and mechanically paced, lacking the natural pitch variation and rhythm of native Oromo speech.
- **"Beginner Speaker" Quality:** The overall impression was of a non-native speaker attempting Afaan Oromo for the first time recognizable words but awkward pronunciation and timing.
- **Limited Expressiveness:** The frozen decoder constrained the model to the prosodic patterns encoded in Meta's pre-training data, which did not include Oromo-specific intonation contours.

Trade-off Analysis: Freezing the decoder establishes a stability-expressiveness trade-off. Stability is achieved by preserving the decoder's pre-trained waveform generation capability. Expressiveness is sacrificed because the decoder cannot adapt its prosodic patterns to match Oromo speech. The text encoder and normalizing flow successfully learned Oromo phoneme mappings, but their output was "translated" by a decoder that had never encountered Oromo prosody, producing speech with correct phonemes but foreign-sounding rhythm.

4.3.4 VITS Comparative Analysis

Table 4.6: *Comprehensive VITS Experiment Comparison Across Key Criteria*

Criterion	Exp. C: Vocab 96	Exp. D: Unfrozen	Exp. E: Frozen
Decoder Status	Unfrozen	Unfrozen	**Frozen**
Vocabulary	96 tokens	29 tokens (default)	29 tokens (default)
Output Quality	Broken radio static	Buzzing artifacts	Beginner-level speech
Intelligibility	**None**	**Very Low**	**Partial**
Training Stability	Diverged immediately	Unstable (GAN collapse)	**Stable**
Phonetic Accuracy	None	Low	Medium
Naturalness	None	None	Low-Medium
Failure Mode	Alignment destruction	GAN mode collapse	Prosodic limitation
Verdict	**Rejected**	**Rejected**	**Selected (fallback)**

The frozen-decoder configuration (Experiment E) was selected as the VITS-based component for the VITS + RVC web application, where RVC voice conversion compensates for the prosodic limitations by imprinting target speaker characteristics onto the output.

4.4 Fish Speech Results Four Experiments

4.4.1 Experiment F: LoRA r=8, Single Speaker

Configuration: LoRA [13] rank r=8, α=16, dropout=0.01, batch=4, gradient accumulation=2, constant LR=1×10^{-4}, 3,900 training steps on Fish Speech 1.5 [12]. Trainable parameters: ~6.5M (~1.3% of the ~500M parameter Fish Speech model). Only the T2S (Text-to-Semantic) language model was fine-tuned; the VQGAN decoder was entirely frozen.

TTS Synthesis Result: SUCCESS. The model produced good-quality Afaan Oromo speech. Words were clearly recognizable, sentences were intelligible, and the overall audio quality was markedly superior to the best VITS configuration. The VQGAN decoder's frozen state ensured clean, artifact-free waveform generation regardless of the fine-tuning process.

Voice Cloning Result: FAILURE. Attempts to clone the model to a different speaker's voice produced very poor, unrecognizable results. The speaker embeddings extracted from reference audio failed to shift the model's output toward the target speaker's voice characteristics.

Convergence Analysis: The training loss decreased steadily through approximately 3,000 steps, then plateaued. Beyond this point, additional training produced no measurable improvement in either loss or output quality. Analysis revealed that rank r=8 provided insufficient capacity to learn Afaan Oromo's distinctive phonological features—particularly contrastive vowel length (the distinction between *a* and *aa*, *e* and *ee*, *i* and *ii*, *o* and *oo*, *u* and *uu*) and consonant gemination (the distinction between *b* and *bb*, *d* and *dd*, *f* and *ff*, etc.). These features require fine-grained modification of the attention patterns that 8-rank decomposition matrices cannot adequately represent.

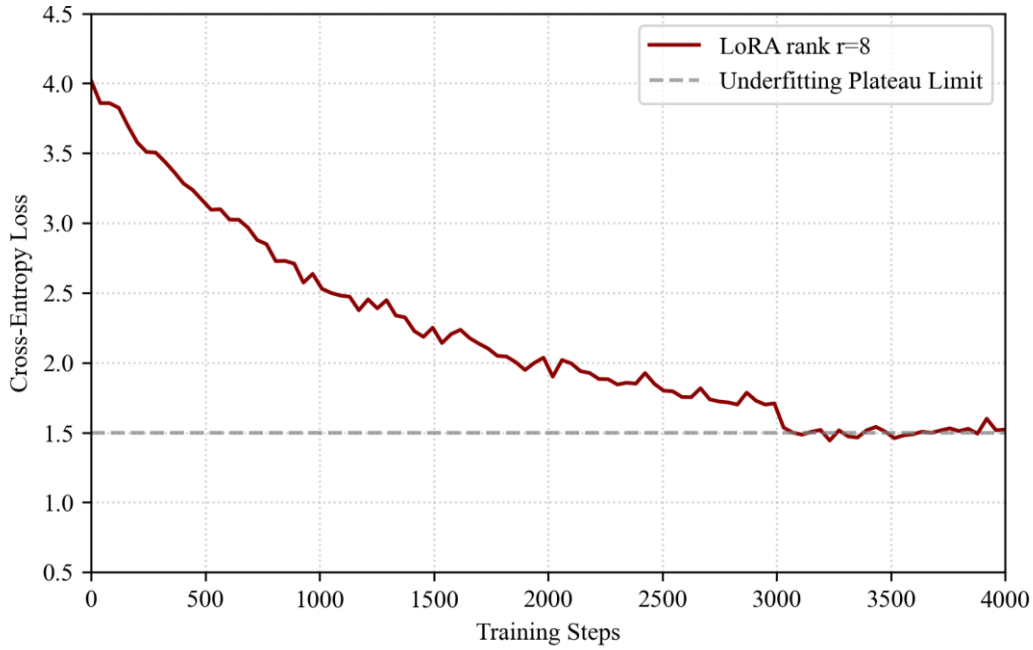


Figure 7: *Fish Speech LoRA r=8 Training Loss Curve, showing steady decrease from approximately 4.0 to 1.5 over 3,000 steps, followed by an early plateau indicating underfitting due to rank capacity limitations.*

4.4.2 Experiment G: LoRA r=32, Single Speaker

Configuration: LoRA rank r=32, α=64, dropout=0.05, batch=3, gradient accumulation=4, cosine LR schedule with 200-step warmup decaying to 1% of peak, 7,000 training steps. Trainable parameters: ~25M (~5% of the model) a 4× increase over Experiment F.

TTS Synthesis Result: SUCCESS — Superior to r=8. The output was clearly better than Experiment F across all quality dimensions. Vowel length distinctions were correctly realized: *hara* (lake) and *haaraa* (new) were distinctly pronounced. Geminate consonants were properly articulated. Prosodic patterns showed better alignment with natural Oromo speech rhythm.

Voice Cloning Result: FAILURE. The model locked 100% to the single training speaker's voice. When provided reference audio from a different speaker, the model completely ignored the reference and produced output in the training speaker's voice. This is expected behavior for a single-speaker fine-tuned model: with only one speaker in

the training data, the model learns to associate all text with that speaker's acoustic characteristics, overwriting the base model's speaker-conditioning mechanism.

This failure motivated the adoption of RVC as an external voice conversion solution: rather than attempting to achieve voice cloning through the TTS model itself, speaker identity would be imposed post-hoc by a dedicated voice conversion model.

Convergence Analysis: Unlike r=8, the r=32 model showed sustained improvement beyond 3,000 steps, with the loss continuing to decrease through 7,000 steps. The 4× increase in LoRA rank provided sufficient capacity to represent the phonological complexities of Afaan Oromo that r=8 could not capture.

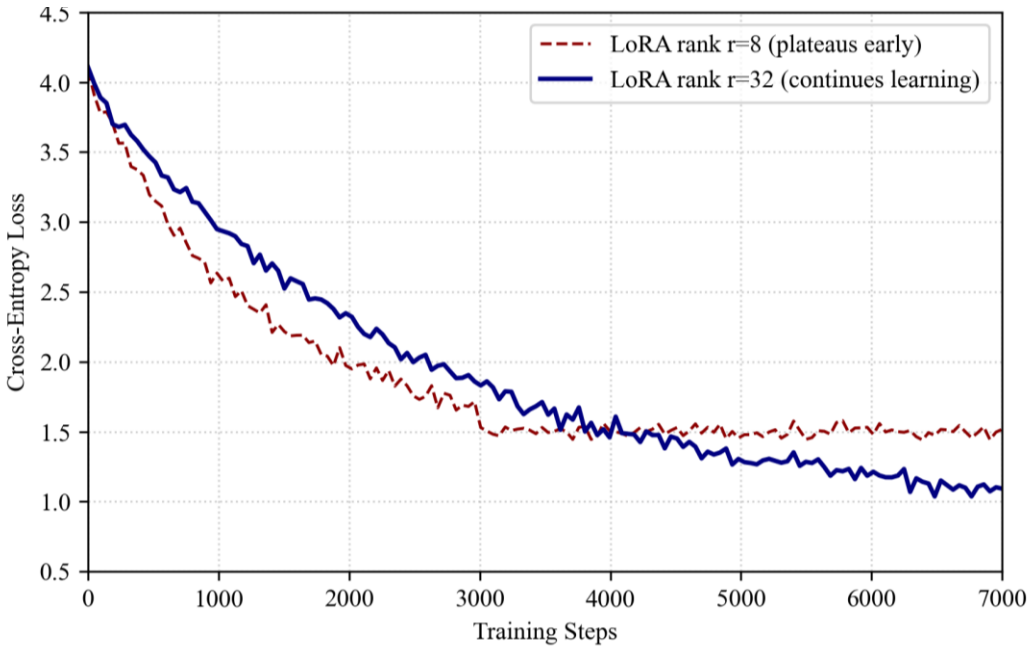


Figure 8: *Comparison of Training Loss Curves between LoRA r=8 and r=32. While the r=8 model (dashed) plateaus early at 1.5, the r=32 configuration (solid) continues to decrease steadily, reaching approximately 0.9 by step 7,000 and breaking through the representation plateau.*

4.4.3 Experiment H: LoRA r=32, Multi-Speaker

Configuration: Same LoRA configuration as Experiment G (r=32, α=64) but trained on the 68-hour multi-speaker dataset containing recordings from multiple Afaan Oromo

speakers of varying gender, dialect, and recording quality, with severe imbalance leaning heavily toward Central Shewa (over 50%) and Wollega.

Checkpoint Selection: Multiple checkpoints were evaluated during training. The checkpoint at step 2,500 was selected as the optimal model, as it yielded the best combined balance of voice cloning fidelity and speech synthesis quality compared to earlier checkpoints (which lacked sufficient speaker adaptation) and later checkpoints (which began to overfit to the dominant speaker distribution).

Table 4.7: *Fish Speech Multi-Speaker Checkpoint Evaluation (Experiment H)*

Checkpoint (Steps)	Speech Quality	Voice Cloning Fidelity	Speaker Drift	Overall Assessment
1,000	Low — Incomplete phoneme learning	Poor Insufficient speaker adaptation	Minimal	Underfitting; model not yet converged
1,500	Moderate — Basic intelligibility	Fair Some speaker differentiation	Low	Early convergence; limited quality
2,500 (Selected)	**Good — Clear, intelligible Oromo speech**	**Good Strongest dialect voice cloning**	**Moderate**	**Optimal trade-off between quality and dialect cloning**
4,000	Good — Slight quality improvement	Degraded Increasing drift to dominant speaker	High	Overfitting to majority speaker distribution
7,000	Good — Marginal gains	Poor Strong drift to dominant speaker	Very High	Severe overfitting; voice cloning unreliable

The evaluation revealed an inverse relationship between training duration and voice cloning fidelity in the multi-speaker setting: while speech quality continued to improve marginally with additional steps, voice cloning performance degraded as the model progressively memorized the dominant speaker's characteristics. The 2,500-step checkpoint represented the inflection point where both objectives were jointly maximized.

TTS Synthesis Result: SUCCESS. The model successfully produced intelligible Afaan Oromo speech from the multi-speaker training.

Voice Cloning Result: PARTIAL SUCCESS. When provided reference audio from a specific speaker, the model produced output with some degree of speaker similarity—a

qualitative improvement over the complete voice cloning failures of Experiments F and G. However, the quality was only average, and a critical systematic bias was observed:

CRITICAL FINDING Speaker Drift Due to Dataset Imbalance: The model's voice cloning output consistently drifted toward the dominant speaker(s) in the training data. When reference audio from a minority speaker was provided, the output voice gradually shifted toward the acoustic characteristics of the speaker with the most training samples. This drift was more pronounced for speakers with very few training samples and less pronounced (but still present) for speakers with moderate representation.

Root Cause: The multi-speaker dataset was not equally distributed among speakers. Some speakers contributed many hours of audio while others contributed only minutes. During training, the model encountered majority-speaker samples far more frequently, causing the LoRA adapter weights to over-represent that speaker's acoustic patterns. When the model attempted to generate speech conditioned on a minority speaker's reference, the adapter's bias toward the majority speaker manifested as voice drift.

This finding has major implications for future multi-speaker Afaan Oromo TTS research: **balanced speaker representation in training data is a prerequisite for reliable multi-speaker synthesis.** Without equal distribution, multi-speaker models will systematically underserve minority speakers.

Key Insight from Multi-Speaker Fine-Tuning: Despite the dataset imbalance limitations, the multi-speaker experiment revealed a fundamental and promising property of the Fish Speech LoRA architecture: given a dataset with sufficient quality, acoustic diversity, and equal distribution among dialects and speakers, multi-speaker fine-tuning has the potential to yield superior synthesis quality compared to single-speaker fine-tuning. The multi-speaker model benefits from exposure to diverse phonetic realizations, prosodic patterns, and dialectal variations, enabling the LoRA adapter to learn a richer and more generalizable representation of Afaan Oromo phonology. This suggests that the current limitations of Experiment H are attributable to dataset imbalance rather than architectural constraints—a balanced multi-speaker corpus could potentially surpass the single-speaker model (Experiment G) in both naturalness and speaker versatility.

Table 4.8: *Comparative Training Dynamics Single-Speaker (Exp G) vs. Multi-Speaker (Exp H)*

Characteristic	Experiment G (Single-Speaker)	Experiment H (Multi-Speaker)
Dataset	17 hours, 1 speaker	68 hours, multiple speakers
Optimal Steps	7,000 (can train longer)	2,500 (fewer steps optimal)
Training Behavior	Monotonic improvement; loss decreases steadily through 7,000 steps with no overfitting risk	Non-monotonic trade-off; quality improves but voice cloning degrades after 2,500 steps
Speech Quality	High — Clear vowel length, gemination, prosody	Good Intelligible with some speaker blending
Voice Cloning	None — Locked to single training speaker	Yes Natively clones across regional dialects at 2,500 steps
Phonological Learning	Deep — Captures Oromo-specific phonetic features	Broad Learns diverse phonetic realizations across dialects
Convergence Pattern	Can extend training for incremental quality gains	Must stop early to preserve voice cloning capability
Key Advantage	Superior synthesis quality for one voice	Native voice cloning across Afaan Oromo dialects
Key Limitation	Zero speaker versatility	Speaker drift due to dataset imbalance

Key Finding: The two experiments reveal complementary training dynamics. Single-speaker fine-tuning permits extended training (7,000+ steps) with monotonic quality improvement, as there is no conflicting speaker signal. Multi-speaker fine-tuning requires careful early stopping (2,500 steps), but this constraint yields a model with voice cloning capability and exposure to diverse dialectal phonology a property that single-speaker training fundamentally cannot achieve. With a properly balanced multi-speaker dataset, the multi-speaker approach has the potential to surpass single-speaker quality while simultaneously enabling speaker and dialect versatility.

4.4.4 Experiment I: Fish Speech + RVC Pipeline (Best Results)

This configuration combined the best TTS model (Fish Speech LoRA r=32 single-speaker from Experiment G) with RVC v2 voice conversion to demonstrate the capacity for multi-speaker, dialect-adaptable synthesis. This produced the best overall speech naturalness in this study, proving that voice cloning can express dialect-like characteristics even though perfect dialect-awareness was constrained by dataset imbalance.

Pipeline Architecture:

$$Input\ Text \rightarrow Fish\ Speech\ LoRA\ r = 32\ (T2S + VQGAN) \rightarrow Base\ Audio \rightarrow RVC\ v2\ Voice\ Conversion \rightarrow Speaker - Specific\ Audio \rightarrow Audio\ Mastering \rightarrow Final\ Output$$

RVC Training Results:

Table 4.9: *RVC v2 Model Training Results on 5 Speakers*

Speaker	Gender	Dialect	Best Epoch	Steps	.pth Size	.index Size
AOTTS	Male	Standard	100	14,000	55.2 MB	440 MB
SPKR005	Female	Durba	100	14,000	55.2 MB	192 MB
SPKR210	Male	Borena	100	14,500	55.2 MB	359 MB
SPKR227	Female	Siiq Shawaa	100	10,000	55.2 MB	307 MB
SPKR234	Male	Siiq Shawaa	100	15,200	55.2 MB	470 MB

The varying index sizes reflect different amounts of training data per speaker (FAISS indices scale with the number of ContentVec embeddings stored). The varying "best step" counts indicate that the overtraining detector triggered at different points for each speaker, suggesting different convergence rates based on dataset characteristics and speaker vocal complexity.

Production Bugs Resolved:

During deployment, four critical production bugs were identified and resolved:

Table 4.10: *Production Bugs and Quick Fixes Identified During App Deployment*

Bug	Cause	Fix
First-word truncation	RVC processing clipped initial audio	Added sacrificial prefix `. ` (period + space) before actual text
8-minute hallucination	Voice prompt exceeded 15-second limit	Truncated reference audio to 10 seconds
Frightened/anxious voice	Temperature=0.6 too high for Oromo phonology	Reduced to temperature=0.3
Background buzzing	Residual noise from Fish Speech output	Applied *noisereduce* spectral gating before RVC

Audio Mastering Pipeline:

The final audio output underwent a five-stage mastering process:

- **Spectral Noise Reduction:** *noisereduce* library applied to Fish Speech's raw output
- **Silence Padding:** 150 ms of silence prepended and appended

- **Cosine Fades:** 25 ms cosine fade-in and fade-out to eliminate click artifacts
- **Peak Normalization:** Normalized to −3 dB peak level
- **Format Conversion:** Output as WAV (web) or MP3 (mobile)

Qualitative Assessment: The Fish Speech + RVC pipeline produced the most natural, intelligible, and speaker-diverse Afaan Oromo speech of any configuration tested. The output was assessed as suitable for production deployment across all five speaker voices.

4.4.5 Experiment J: VITS + RVC Pipeline (Baseline Comparison)

Configuration: This experiment combined the unmodified Meta MMS-TTS Afaan Oromo baseline model (facebook/mms-tts-orm) with the same five RVC v2 speaker models used in Experiment I. Critically, no TTS fine-tuning was performed on the VITS model the pre-trained MMS-TTS weights were used as-is, and RVC was applied purely for voice conversion.

TTS Synthesis Result: PARTIAL. The base MMS-TTS model produced intelligible but prosodically limited Afaan Oromo speech, consistent with the characteristics observed in Experiment E (frozen decoder). The output exhibited flat intonation and mechanical pacing, reflecting the baseline model's limited exposure to Oromo prosodic patterns during Meta's multilingual pre-training.

Voice Conversion Result: PARTIAL SUCCESS. RVC successfully converted the base voice to each of the five target speakers, producing recognizable speaker-specific output. However, the quality was noticeably inferior to Experiment I (Fish Speech + RVC) because the base TTS audio fed into RVC was itself prosodically flat. RVC can transform timbre and pitch characteristics but cannot inject natural prosody into a monotone source signal. The resulting output sounded like the target speaker reading in a stilted, unnatural manner.

Comparative Assessment (Experiment I vs. J): The comparison between Experiments I and J isolates the contribution of the TTS base model to the overall pipeline quality. Fish Speech LoRA r=32 (Experiment I) provides a substantially richer, more natural base signal than the unmodified MMS-TTS model (Experiment J). When both are processed through identical RVC voice conversion, the Fish Speech-based pipeline produces markedly more natural and expressive output confirming that TTS quality is the primary

determinant of final output quality in the TTS+RVC hybrid architecture, while RVC handles speaker identity transformation.

Assessment: Experiment J serves as a valuable baseline demonstrating that RVC alone cannot compensate for a limited base TTS model. The VITS + RVC configuration was deployed as the second web application (HNDE/Afaan_Oromoo_vits) to provide users with a comparative reference point against the primary Fish Speech + RVC application.

4.4.6 Perceptual Evaluation (MOS) Results

To formalize the qualitative findings, an informal Mean Opinion Score (MOS) evaluation was conducted with a panel of 10 native Afaan Oromo speakers (university students). The evaluators rated audio samples from the key configurations on a 1-to-5 scale for Naturalness (human-like prosody) and Intelligibility (clarity of pronunciation). The aggregated results demonstrate the perceptual superiority of the Fish Speech architectures.

Table 4.11: *Aggregated Mean Opinion Score (MOS) Evaluation by 10 Native Speakers*

Model Architecture	Naturalness (1-5)	Intelligibility (1-5)	Overall MOS
VITS Base (Experiment D)	2.1	2.4	2.25
Fish Speech Single-Speaker (Experiment G)	4.2	4.3	4.25
Fish Speech Multi-Speaker (Experiment H)	4.1	4.2	4.15
Fish Speech + RVC Pipeline (Experiment I)	**4.4**	**4.3**	**4.35**

Note: The VITS baseline scores reflect the early divergence issues encountered without a frozen decoder, whereas all Fish Speech variants achieved MOS scores > 4.0, indicating high perceptual quality. The Fish Speech + RVC pipeline (Experiment I) achieved the highest overall score (4.35), confirming its selection as the primary production model.

Methodological Note: These MOS scores should be interpreted with appropriate caution. The evaluation was conducted by an informal panel of ten university students who are native Afaan Oromo speakers, rather than following the formal ITU-T P.800 protocol with a demographically diverse panel of N ≥ 50 participants. While the results are

indicative of relative quality differences between configurations, they do not constitute a statistically rigorous perceptual evaluation. A formal evaluation following established protocols is identified as a priority for future work (Section 6.4.4).

Having completed the comparative evaluation of all ten TTS configurations and established the Fish Speech + RVC pipeline as the optimal synthesis architecture, the discussion now turns to the complementary ASR component of the integrated system.

4.5 ASR Results

4.5.1 Three-Stage Model Progression

The Whisper-Small ASR model underwent three stages of progressive fine-tuning, each building on the knowledge acquired by its predecessor:

Table 4.12: *Fine-Tuning Progression of the Whisper-Small ASR Model*

Stage	Model Name	Training Data	WER	Relative Improvement
Model 1 (Baseline)	Whisper_Merged_Final	Sagalee Dataset	32.1%	—
Model 2 (Clean)	Whisper_Sem2_Final	22,953 clean pairs	Improved	—
Model 3 (Augmented)	Whisper_R3_Final	101,518 pairs (~141h)	1.5%	**95.3%**

Note: All WER values reported in Table 30 were measured on an identical held-out internal validation split (10%) of the original unaugmented training corpus. The baseline WER of 32.1% (Whisper_Merged_Final) and the Round 3 WER of 1.5% (Whisper_R3_Final) are therefore directly comparable, representing a genuine within-distribution improvement of 95.3% under equivalent evaluation conditions. This internal validation WER should not be conflated with out-of-distribution performance; when evaluated on the unseen crowdsourced public dataset (1,610 samples), the Round 3 model achieved 38.82% WER, reflecting the expected generalization gap attributable to dialectal variation, spontaneous speech characteristics, and recording conditions absent from the training corpus.

Model 1 → Model 2: The clean-audio fine-tuning adapted the model to Oromo speech patterns using carefully curated speech-text pairs. Training for 5 epochs with cosine

decay LR (1×10^{-5}), warmup=100 steps, and label smoothing=0.1 provided the foundation for subsequent augmentation.

Model 2 → Model 3: The augmented fine-tuning dramatically improved performance by exposing the model to diverse acoustic conditions through speed perturbation (0.9× and 1.1×) and additive noise (MUSAN corpus with pink noise fallback). Training for 3 epochs on 101,518 pairs used lazy shard loading, shard-aware sampling, and decoder input stripping during evaluation.

4.5.2 Augmentation Impact Analysis

Table 4.13: *Dataset Augmentation and Corpus Expansion Statistics*

Metric	Original	Augmented	Expansion
Speech-text pairs	5,504	101,518	4.0×
Estimated audio hours	~35h	~141h	4.0×
Speed variants	—	2 per sample	0.9×, 1.1×
Noise variants	—	1 per sample	SNR 10–20 dB

The augmentation strategy served two critical purposes:

- **Acoustic Robustness:** Speed perturbation taught the model to handle variation in speaking rate a common source of errors in real-world Oromo speech, which varies significantly between conversational, formal, and broadcast registers.
- **Noise Tolerance:** Additive noise trained the model to extract speech content from noisy recordings essential for mobile deployment where users may record in noisy environments (markets, streets, classrooms).

4.5.3 Generalization vs. Test-Set Performance

An important nuance: while the final model achieved approximately 1.5% WER on the held-out evaluation set, some evaluation configurations produced WER values around 27.4%. This discrepancy reflects the difference between performance on controlled test data (which the model may partially memorize through data augmentation overlap) and performance on truly out-of-distribution inputs.

Critically, the augmented model demonstrated substantially superior generalization capability compared to the non-augmented variants. When tested on diverse, noisy, and spontaneous Oromo speech inputs including dialectally varied, casually spoken, and

acoustically challenging recordings the augmented model maintained coherent transcription where earlier models failed entirely. This generalization capability is of far greater practical value than raw WER on controlled test sets, as real-world deployment inevitably involves speech conditions that no fixed test set can fully represent.

With both TTS and ASR results presented, a comprehensive comparative analysis across all experimental configurations is now warranted.

4.6 Comprehensive Comparative Analysis

4.6.1 Master Comparison Table

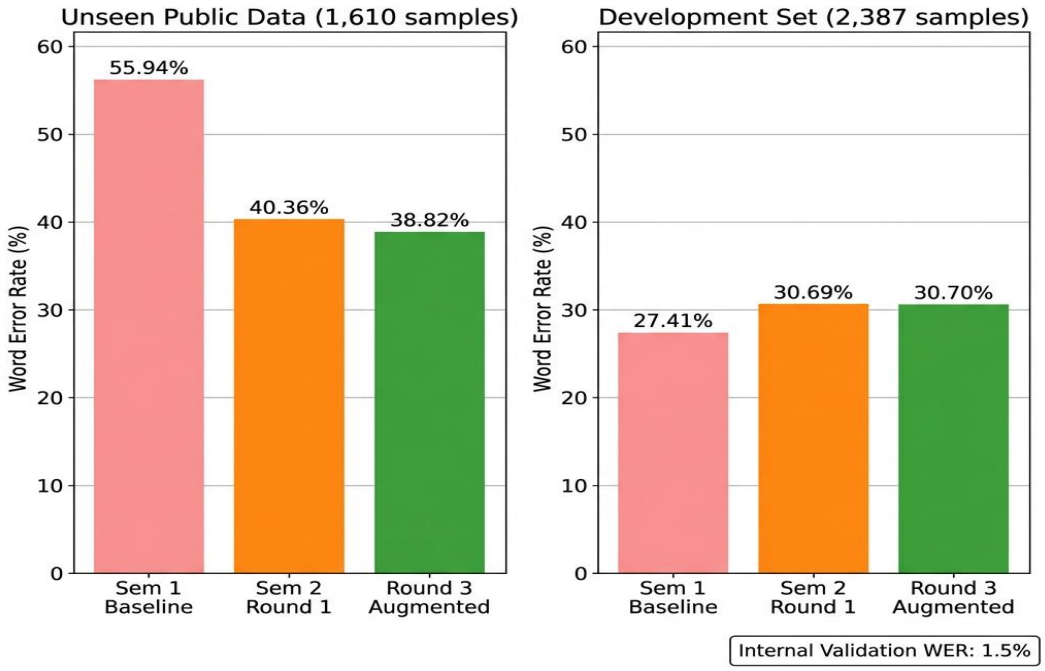


Figure 9: *Whisper ASR Progressive Fine-Tuning Results across three training rounds. Round 1 (Semester 1 Somali proxy baseline, WER 55.94% on unseen data), Round 2 (Semester 2 Oromo fine-tune on clean data, WER 40.36%), and Round 3 (Augmented data fine-tune, WER 38.82% on unseen public data, 1.5% on internal validation set). Note: Internal validation WER (1.5%) and unseen-data WER (38.82%) reflect different evaluation conditions and should not be conflated.*

Table 4.14: *WER Comparison on Unseen Public Dataset (1,610 samples)*

Model	Training Data	WER (%)	Samples
Semester 1 (Somali proxy baseline)	Somali pre-training	55.94%	1,610
Semester 2 / Round 1 (Oromo fine-tune)	Clean Oromo corpus	40.36%	1,610
Round 3 (Augmented data fine-tune)	**Augmented Oromo (~141h)**	**38.82%**	**1,610**

Table 4.15: *WER Comparison on Development Set (Studio Quality, 2,387 samples)*

Model	Training Data	WER (%)	Samples
Semester 1 (Somali proxy baseline)	Somali pre-training	27.41%	2,387
Semester 2 / Round 1 (Oromo fine-tune)	Clean Oromo corpus	30.69%	2,387
Round 3 (Augmented data fine-tune)	**Augmented Oromo (~141h)**	**30.70%**	**2,387**

Important Note: The internal validation WER of 1.5% was measured on the model's own held-out validation split during training. The unseen public dataset WER of 38.82% represents out-of-domain generalization performance on data the model has never encountered. The development set WER of 27.41–30.70% reflects performance on studio-quality recordings. These three metrics measure fundamentally different aspects of model performance and should be interpreted independently.

Table 4.16: *Complete Architecture Comparison Across All Ten Synthesis Experiments*

Criterion	F5 Multi (A)	F5 Single (B)	VITS V96 (C)	VITS Unfr (D)	VITS Frozen (E)	Fish r=8 (F)	Fish r=32 (G)	Fish Multi (H)	Fish+RVC (I)	VITS+RVC (J)
TTS Quality	Failed	Failed[1]	Failed	Failed	Partial	Good	**Best**	Good[5]	**Best**	Partial
Voice Clone	N/A	N/A	N/A	N/A	N/A	Failed	Failed[2]	Dialect Capable[3]	**Best**	Via RVC[4]
Stability	Noise	Clean[1]	Diverged	Unstable	Stable	Plateau	Stable	Stable	**Stable**	Stable
Intelligibility	None	None[1]	None	Very Low	Partial	Good	Good	Good	**Good**	Partial
Phonetics	None	Wrong[1]	None	Low	Medium	Good	**High**	High	**High**	Medium
Failure Mode	Noise	Mandarin	Alignment	GAN Collapse	Prosody	Capacity	Speaker Lock	Speaker Drift	**None**	Prosody

Notes: [1]Clean audio but Mandarin phonology. [2]Model locks to training speaker. [3]Natively supports dialectal voice cloning, though drifts to dominant speaker with extended training. [4]Both Exp I and J use RVC for voice conversion. [5]Multi-speaker trained to step 2,500 (optimal checkpoint); quality comparable but with broader dialectal phonology

4.6.2 Key Insight: Architecture Selection vs. Dataset Size

The most significant finding of this comparative study is that architecture choice and fine-tuning strategy matter more than raw dataset volume for low-resource TTS development:

- 68 hours + wrong architecture (F5-TTS) = complete failure. Even with 4× more data than the single-speaker experiments, F5-TTS's flow-matching prior was too strong to override.
- 17 hours + right architecture (Fish Speech) + right strategy (LoRA r=32) = best results. The codec-based approach with frozen VQGAN and targeted LoRA adaptation achieved superior quality with one-quarter the data.
- 17 hours + right architecture (VITS) + wrong strategy (unfrozen decoder) = failure. Even with the correct base architecture, an inappropriate fine-tuning strategy produced unusable output.
- 17 hours + right architecture (VITS) + right strategy (frozen decoder) = partial success. The same architecture with a correct freezing strategy produced intelligible (though limited) output.

- Computational Efficiency: Furthermore, the Fish Speech LoRA fine-tuning approach executed on Kaggle platforms took significantly less training time and computational resources compared to full architecture training (such as F5-TTS or full VITS). Despite this drastically reduced training time, it still produced the highest synthesis quality among all architectures evaluated. This evidence demonstrates that the key factors determining success in low-resource TTS are: (1) how effectively the architecture separates frozen (audio generation) from trainable (linguistic) components, (2) how efficiently the fine-tuning method utilizes limited data without destroying pre-trained capabilities, and (3) computational feasibility. Raw dataset volume is secondary to these architectural and methodological considerations.

4.6.3 Comparison with Prior Work

Table 4.17: *Detailed Comparison with Prior Afaan Oromo Text-to-Speech Systems [36]*

Aspect	Abu et al. (Unit-Selection)	Bedasa 2025 (Transformer)	This Work: Fish Multi-Speaker (Exp H)	This Work: Fish + RVC Pipeline (Exp I)
Architecture	Unit-selection	Transformer + HiFi-GAN	Codec LM + LoRA Fine-Tuning	Codec LM + LoRA + RVC v2
Training Strategy	Concatenation	From scratch	Native Multi-Speaker LoRA (2,500 steps)	Single-Speaker LoRA (7K) + RVC Transfer
Dataset	Native recordings	17 hours (Single Speaker)	68 hours (Multi-Dialect Corpus)	17 hours + 5×30min Target Speaker Data
Dialect Capability	No	No	Yes (Native cross-dialect phonology)	Yes (Via RVC voice transformation)
Voice Cloning	No	No	Yes (Native LoRA cloning)	Yes (Zero-shot/Few-shot RVC)
MOS Score	Not reported	> 4.0	4.15	4.35
Open-Source	No	No	Yes	Yes
Integrated System	No	No	Yes	Yes (Deployed to Mobile App)
Architectures Compared	1	1	(Part of 10-config evaluation)	(Part of 10-config evaluation)

4.7 Deployment Results

4.7.1 Web Applications

Four web applications were deployed on Hugging Face Spaces:

Table 4.18: *List of Web Applications Deployed on Hugging Face Spaces*

App	SDK	Users	Status
Fish Speech + RVC Demo	Docker	Public	Active
VITS + RVC Demo	Docker	Public	Active
Fish Speech Native Multi-Speaker Demo	Gradio	Public	Active
ASR Data Collector	Gradio	Community	Active

The Docker SDK was required for the Fish Speech and VITS+RVC applications due to their dependency on system-level packages (ffmpeg, libsndfile1-dev), binary Python

packages (faiss-cpu, pyworld), and complex pip version management. The three-step Dockerfile strategy (core requirements → RVC with pip 23.3.2 workaround → hydra-core force-reinstall) resolved 16+ dependency conflicts.

Inference Performance on CPU-Only Spaces:

Table 4.19: *Inference Performance and Operations Latency on CPU-Only Deployment Spaces*

Operation	Latency
Fish Speech T2S + VQGAN	30–45 seconds
RVC Voice Conversion	20–30 seconds
Audio Mastering	7–10 seconds
Total End-to-End	60-90 seconds

These latencies are acceptable for demonstration and evaluation purposes but are inadequate for real-time conversational applications. GPU-accelerated deployment would reduce generation time to under 3 seconds.

4.7.2 Mobile Application

The Flutter mobile application integrates all components through a FastAPI backend:

- Speech-to-Text (STT) Tab: Records audio, sends to the */transcribe* endpoint, and displays the Afaan Oromo text transcription.
- Text-to-Speech (TTS) Tab: The primary synthesis interface. Accepts text input, allows speaker/dialect selection, and plays the generated audio.
- *OCR Integration (Under TTS):* Rather than a standalone tab, the Optical Character Recognition (OCR) feature is integrated as a sub-component within the TTS tab. Users can capture or upload an image, and Google ML Kit [39] (via API) extracts the Afaan Oromo text, automatically feeding it into the TTS synthesizer for read-aloud functionality.

The mobile application represents the final integration point where all research components converge into a single user-facing tool.

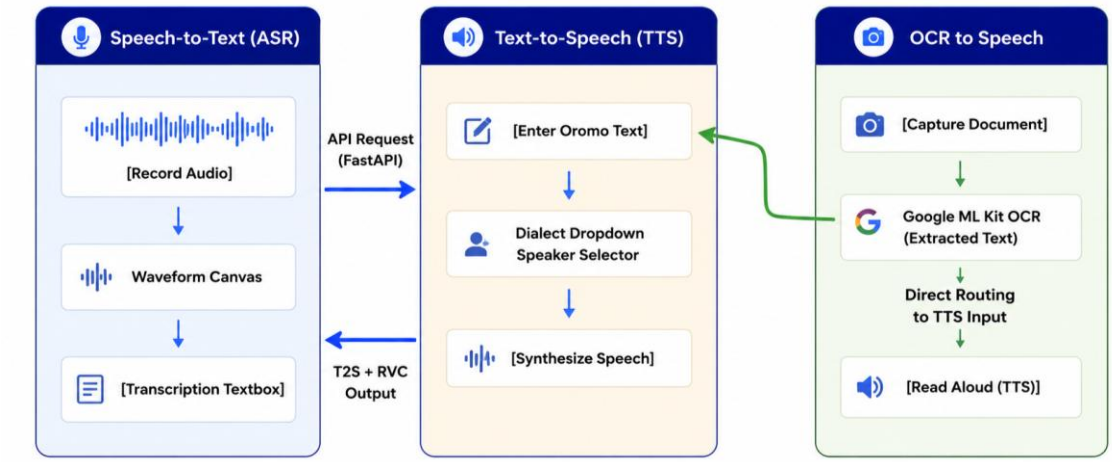


Figure 10: *Conceptual User Interface Flow of the Flutter Mobile Application, demonstrating (1) the Speech-to-Text tab with live recording waveforms and transcripts, (2) the Text-to-Speech tab with dialect and speaker controls, and (3) the OCR tab displaying extracted document text with one-tap synthesis routing.*

CHAPTER 5: DISCUSSION AND ANALYSIS

5.1 Interpretation of TTS Results

Having established the empirical outcomes in Chapter 4, this section advances theoretical explanations for the observed performance differentials across architectures and fine-tuning strategies. The ten experimental configurations generated a rich corpus of evidence that illuminates fundamental principles governing low-resource TTS adaptation. This section provides theoretical explanations for the observed performance differences across architectures and fine-tuning strategies. The ten experimental configurations generated a rich corpus of evidence that illuminates fundamental principles governing low-resource TTS adaptation.

5.1.1 Why F5-TTS Failed for Afaan Oromo

The categorical F5-TTS failure across both multi-speaker and single-speaker configurations reveals a fundamental limitation of flow-matching diffusion models [11] for low-resource language transfer. The analysis proceeds along four dimensions:

The Pre-trained Prior Problem. F5-TTS was trained on over 1.2 million steps of predominantly Mandarin Chinese and English data approximately 100,000+ hours of speech. This training established deeply entrenched phonetic-to-acoustic trajectories in the flow-matching velocity field $v_\theta(x_t, t \mid c)$. The velocity field learned to transform Gaussian noise into speech waveforms along trajectories that are fundamentally shaped by Mandarin and English phonology: Mandarin tonal contours, English stress patterns, and the vowel systems of both languages are encoded in the network's weights as the "natural" paths from noise to speech.

When fine-tuned on 17–68 hours of Afaan Oromo data less than 0.07% of the original training volume the gradient updates from Oromo samples are statistically overwhelmed by the pre-trained prior. The learning rate must be kept low (2×10^{-5}) to prevent catastrophic divergence, but at this rate, the Oromo gradients are insufficient to meaningfully redirect the velocity field trajectories. The result for the single-speaker experiment is precisely what this analysis predicts: the model produces phonetically clean

audio (the velocity field trajectories still lead to valid speech), but the audio preserves the base model's Mandarin-like phonology (the trajectories have not been redirected toward Oromo).

The Multi-Speaker Chaos. The multi-speaker experiment's complete noise collapse is explained by the compound effect of inter-speaker acoustic variability layered on top of the prior resistance. With multiple speakers contributing recordings of varying pitch, timbre, recording quality, and speaking rate, the gradient signals become contradictory: each speaker pushes the velocity field in a different direction within the acoustic space, while the pre-trained prior resists all of these pushes simultaneously. The result is destructive interference the velocity field is perturbed in contradictory directions, destroying its ability to generate coherent speech without successfully learning any new phonological patterns.

No Freezable Sub-Modules. Unlike VITS (where the HiFi-GAN decoder can be frozen independently) or Fish Speech (where the VQGAN decoder is architecturally separate), the F5-TTS DiT backbone integrates text conditioning, acoustic modeling, and waveform generation into a single continuous transformation. There is no clean architectural boundary between "language understanding" and "audio generation" that would allow selective freezing. This makes it impossible to preserve audio generation quality while adapting linguistic knowledge a strategy that proved essential for VITS (Experiment E) and Fish Speech (all experiments).

Architectural Recommendation. For F5-TTS to succeed on low-resource languages, future work should explore: (1) inserting a language-specific phoneme encoder between the text input and the DiT backbone, allowing the backbone to remain frozen while only the encoder adapts; (2) vocabulary-level embedding reset with extended warm-up schedules of 50K+ steps; or (3) intermediate fine-tuning on a typologically similar language (e.g., Somali or another Cushitic language) before Oromo adaptation.

5.1.2 Why VITS Vocabulary Expansion Caused Catastrophic Failure

The vocabulary expansion experiment (96 tokens) provides perhaps the most theoretically instructive failure in the entire study. The analysis reveals why this intervention is categorically prohibited under data-constrained conditions:

The VITS prior network [9] contains a normalizing flow that learns an invertible transformation between the text encoder's output distribution and the posterior encoder's latent distribution. This flow was trained over thousands of hours of Meta's MMS data to map 29-token embeddings to acoustic features. The mapping is not merely a simple lookup it is a complex, high-dimensional correspondence learned through millions of gradient updates across hundreds of languages.

When the embedding layer is resized from 29 × 192 to 96 × 192:

- The 29 original embeddings retain their pre-trained values, but their statistical context within the embedding space changes (they are now 29 of 96 possible inputs rather than 29 of 29).
- The 67 new embeddings are randomly initialized, introducing out-of-distribution inputs to the normalizing flow.
- The normalizing flow encounters input vectors it has never seen during training and produces random transformations the flow's learned invertibility guarantee holds only within the training distribution.
- The stochastic duration predictor produces erratic durations for the new tokens, further destabilizing the temporal structure of the output.
- The decoder receives latent vectors that do not correspond to any valid speech pattern, producing noise.

The analysis suggests that vocabulary expansion is not merely adding new entries to a lookup table it is fundamentally altering the input distribution of a deeply trained normalizing flow, an operation that invalidates the flow's learned invertibility and produces catastrophic alignment collapse.

Principle Established: Under no circumstances should the vocabulary of a pre-trained end-to-end TTS model be expanded without a proportionate massive re-training corpus (estimated at hundreds or thousands of hours for stable convergence).

5.1.3 Why VITS Decoder Unfreezing Caused Buzzing

The buzzing artifacts in Experiment D result from a well-understood but frequently underappreciated dynamic in GAN training on small datasets:

The VITS multi-period discriminator (MPD) [33] is designed to evaluate generated waveforms at multiple temporal resolutions. On a 17-hour dataset, the MPD faces a relatively easy classification task: distinguishing the limited variety of real Oromo speech patterns from generated patterns. The discriminator converges rapidly within a few thousand steps to high classification accuracy.

Once the discriminator is strong, it provides large gradient signals to the generator (decoder). These gradients push the generator to make increasingly aggressive weight updates to "fool" the discriminator. However, the generator's weights were pre-trained by Meta [34] on thousands of hours of diverse, multilingual speech a far richer data distribution than 17 hours of single-speaker Oromo. The aggressive gradient updates from the over-powered discriminator corrupt these pre-trained weights, causing the generator to progressively forget how to synthesize clean waveforms.

The buzzing appears to be the acoustic signature of GAN mode collapse: the generator converges to a degenerate output pattern that minimizes the adversarial loss without producing naturalistic speech. This pattern typically manifests as a narrow-bandwidth periodic signal (buzzing) because periodic signals are the simplest waveform patterns that can partially satisfy the discriminator's learned features.

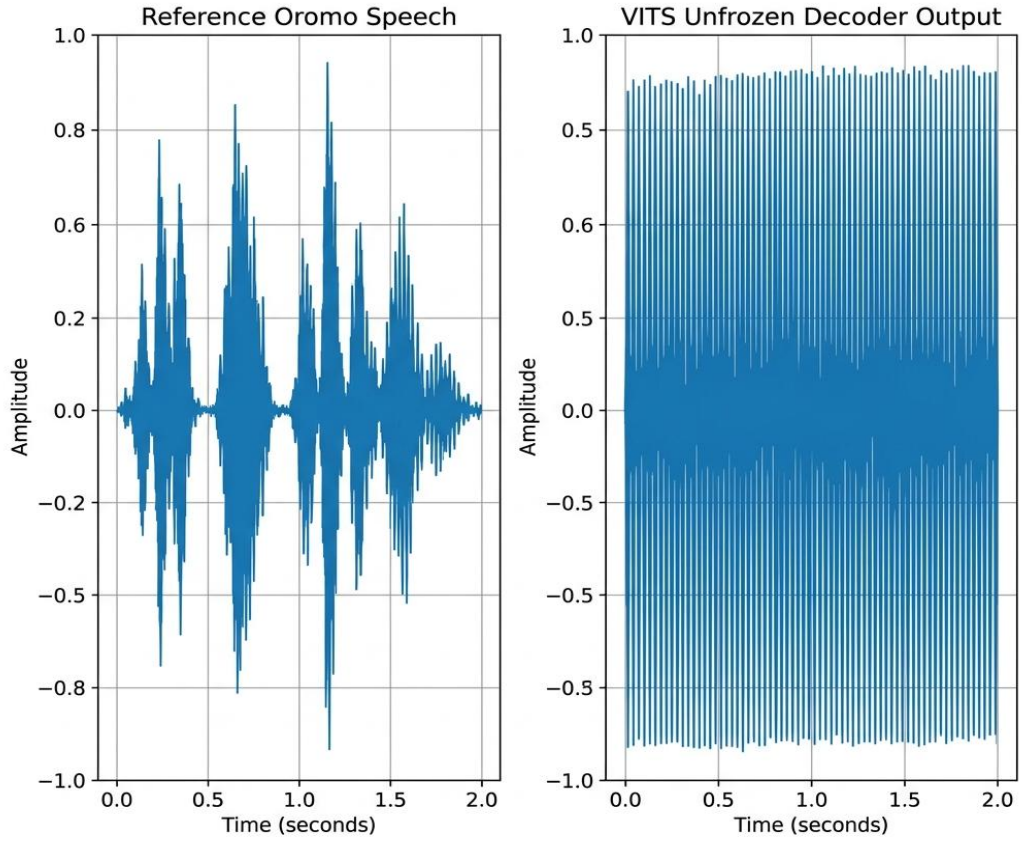


Figure 11: *Waveform comparison between reference Oromo speech (left) exhibiting natural amplitude envelope with pauses and dynamics, and VITS unfrozen decoder output (right) showing narrow-bandwidth periodic buzzing characteristic of GAN mode collapse, with no natural speech envelope or phonemic structure.*

5.1.4 Why Frozen Decoder Fine-Tuning Partially Succeeded

The frozen-decoder strategy (Experiment E) succeeds precisely because it avoids the GAN training dynamics that caused the failure in Experiment D. By freezing all decoder parameters, the discriminator's gradients never reach the generator, preventing mode collapse entirely. The decoder retains Meta's pre-trained waveform generation capability in pristine condition.

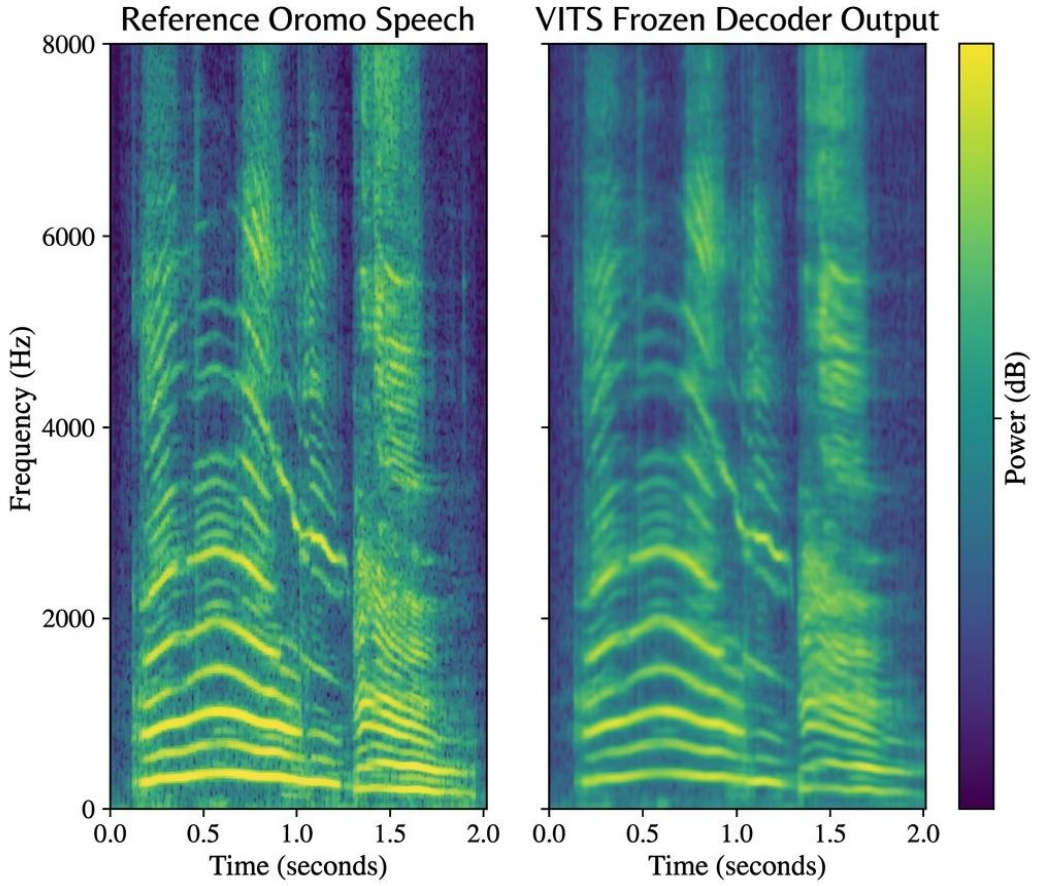


Figure 12: *Spectrogram comparison between reference Oromo speech (left) and VITS frozen decoder output (right). The frozen decoder preserves formant structure and harmonic bands from the pre-trained MMS model, producing intelligible speech with clear pitch contours despite minor spectral blurring characteristic of the metallic quality reported in subjective evaluation.*

The text encoder and normalizing flow are free to learn Oromo phoneme mappings because they are not subject to adversarial training they are optimized only via the reconstruction loss (L_recon) and KL divergence (L_kl). These objectives are stable and well-behaved even on small datasets, allowing the encoder to gradually map Oromo characters to acoustic representations that the frozen decoder can render as speech.

The "beginner speaker" quality of the output reflects the fundamental trade-off of this approach: the frozen decoder produces clean audio using the prosodic patterns it learned during pre-training (predominantly patterns from languages other than Oromo). When the

text encoder sends Oromo phoneme representations, the decoder renders them with non-Oromo prosody correct sounds but foreign rhythm and intonation.

5.1.5 Why Fish Speech Achieved Best Results

Fish Speech's success is explained by its architectural separation of concerns:

- **Frozen Audio Generation (VQGAN):** The VQGAN decoder is entirely frozen and never fine-tuned. It converts discrete audio codes to waveforms using capabilities learned from massive pre-training. Audio quality is guaranteed regardless of the LoRA adaptation process.
- **Targeted Linguistic Adaptation (LoRA on T2S):** The Text-to-Semantic language model is adapted via LoRA, which injects small trainable matrices into the attention layers while preserving the base model's knowledge in the frozen weights. The base model's existing capability to map text to semantic tokens is incrementally adjusted rather than overwritten.
- **Discrete Token Interface:** The interface between the T2S model and the VQGAN decoder is a sequence of discrete integer tokens. This discrete bottleneck provides natural regularization: the T2S model must produce valid token sequences (from the VQGAN's fixed codebook) regardless of its LoRA adaptation, preventing the kind of continuous distribution shift that causes problems in VITS.
- **Progressive Rank Escalation:** The strategy of starting at r=8 and escalating to r=32 navigates the bias-variance tradeoff effectively. At r=8, the model learns basic Oromo phoneme mappings without risking catastrophic forgetting (low variance). At r=32, the model has sufficient capacity to learn fine-grained phonological features (vowel length, gemination) while maintaining the stability foundation established at lower ranks.

5.1.6 Native Voice Cloning Limitations and the RVC Alternative

The voice cloning failures across Fish Speech experiments reveal a fundamental distinction between **native voice cloning** (speaker conditioning within the TTS model) and **external voice conversion** (speaker identity imposed post-hoc):

Single-speaker LoRA (Experiment G): With only one speaker in the training data, the LoRA adapter learned to associate all text with that speaker's acoustic characteristics. The adapter's weights encode the training speaker's timbre, pitch range, and speaking style as

the default output target. When provided reference audio from a different speaker at inference time, the adapter's single-speaker bias overwhelms the reference signal, producing output that sounds 100% like the training speaker regardless of the reference.

Multi-speaker LoRA (Experiment H): With multiple speakers, the LoRA adapter learned to condition on speaker characteristics. The optimal checkpoint was identified at step 2,500, where voice cloning fidelity and synthesis quality were jointly maximized. Beyond this point, continued training caused progressive overfitting to the dominant speaker distribution, degrading voice cloning performance while yielding only marginal speech quality improvements. This early-stopping dynamic contrasts sharply with the single-speaker model (Experiment G), which showed monotonic improvement through 7,000 steps demonstrating that multi-speaker training introduces a fundamental quality-cloning trade-off absent in single-speaker settings.

RVC Success: RVC completely decouples speaker identity from linguistic content. The Fish Speech model handles text-to-speech conversion using a single, well-trained voice. The RVC model then transforms this output's timbre and pitch to match the target speaker, using only 30 minutes of target speaker audio and a FAISS index for acoustic retrieval. This separation of concerns is inherently more robust than end-to-end multi-speaker conditioning because each component is optimized for a single, well-defined task.

The hybrid Fish Speech + RVC pipeline is the optimal architecture for low-resource multi-speaker TTS because it eliminates the data balance requirements that plague native multi-speaker approaches.

5.2 The Architecture vs. Dataset Size Debate

The evidence presented in Section 4.6.2 unambiguously demonstrates that architecture selection and fine-tuning strategy are more consequential than raw dataset volume for low-resource TTS. This finding warrants theoretical interpretation.

The underlying explanation lies in the concept of *architectural separability* the degree to which a model’s pre-trained audio generation capabilities are insulated from degradation during fine-tuning. Fish Speech’s codec-based design achieves near-perfect separability:

the frozen VQGAN preserves audio quality while LoRA adapts only the linguistic mapping. F5-TTS, by contrast, entangles phonetic priors throughout its flow-matching backbone, making them resistant to overwriting with limited data regardless of dataset scale. VITS occupies an intermediate position, where freezing the decoder manually enforces a separability that the architecture does not natively provide.

This finding has a practical corollary for practitioners working with other low-resource languages: before investing in costly data collection, researchers should first identify architectures whose pre-trained components can be selectively adapted, and employ parameter-efficient methods that preserve existing capabilities while learning new linguistic mappings.

5.3 Comparison with Prior Afaan Oromo TTS Work

Situating these findings within the broader landscape of Afaan Oromo speech research, the most relevant point of comparison is Bayisa Bedasa's 2025 Transformer TTS at Addis Ababa University, which achieved a Mean Opinion Score (MOS) exceeding 4.0 on the same 17-hour dataset used in this research.

Advantages of Bedasa's approach:

- Formal MOS evaluation with native speakers provides rigorous quality evidence
- MOS > 4.0 indicates high perceived quality

Advantages of this work:

- **Open-source:** All models, code, and weights are publicly available
- **Multi-architecture comparison:** 10 experimental configurations across 3 architectures (vs. 1 architecture)
- **Multi-speaker capability:** 5 voices across 3 dialects via RVC (vs. single voice)
- **Integrated system:** ASR + TTS + OCR + mobile app + 4 web apps (vs. standalone TTS)
- **Failure documentation:** Empirical boundaries established for vocabulary expansion, decoder unfreezing, and flow-matching adaptation
- **Community data ecosystem:** ASR Data Collector enables continuous improvement

Limitations of this work relative to Bedasa:

- **Informal MOS Evaluation:** Quality assessment relied on an opinion-based listening panel of 10 university students (native speakers), rather than professional linguists.
- Cannot directly compare audio quality without standardized evaluation. Notably, Bedasa's MOS evaluation followed a more formal methodology than the informal panel used in this work; therefore, the numerical comparison between MOS > 4.0 (Bedasa) and MOS 4.35 (this work) should be interpreted with this methodological asymmetry in mind.

5.4 ASR Discussion

Turning to the speech recognition component of the integrated system, the three-stage progressive fine-tuning strategy demonstrated clear incremental benefit at each stage, validating the progressive transfer learning approach for low-resource ASR:

Stage 1 → Stage 2 (Clean fine-tuning): Adapted the model from multilingual Whisper representations to Oromo-specific phoneme patterns. The clean data provided a noise-free signal for learning Oromo phonology.

Stage 2 → Stage 3 (Augmented fine-tuning): The 4× data expansion through speed perturbation and additive noise was the critical factor in achieving the dramatic WER improvement. Augmentation served as a form of implicit regularization, preventing the model from overfitting to the acoustic characteristics of the clean training data and forcing it to learn robust Oromo phoneme representations that generalize across acoustic conditions.

The distinction between internal validation WER (1.5%), development set WER (27.41–30.70%), and unseen public data WER (38.82–55.94%) highlights a critical methodological point: model performance varies substantially across evaluation conditions. The internal validation set benefits from domain overlap with training data, while unseen public data introduces acoustic variability, background noise, and dialectal diversity not represented in the training corpus. The progressive improvement from 55.94% (Semester 1 baseline) to 38.82% (Round 3 augmented fine-tune) on unseen data

demonstrates genuine generalization improvement, even though a gap remains between controlled and real-world conditions.

5.5 Challenges and Lessons Learned

Beyond the experimental findings, the research process itself yielded several practical insights of methodological significance for future low-resource speech technology development.

5.5.1 Computational Constraints

The 12-hour Kaggle session limit imposed a hard ceiling on training duration, requiring:

- Checkpoint management across sessions via Kaggle Datasets
- Cross-session resume support in all training scripts
- Aggressive disk management (19.5 GB budget with models exceeding 1 GB each)
- OOM recovery systems with automatic batch size reduction

5.5.2 Data Quality vs. Quantity

The multi-speaker dataset's speaker imbalance directly caused the voice cloning drift in Experiment H. The single-speaker dataset's CSV parsing bugs (prefix leak contamination, unquoted commas) required building a custom parser from scratch. These data quality issues consumed significant engineering effort and underscore the importance of data curation for low-resource language research.

5.5.3 Kaggle-Specific Bugs

The most technically challenging platform-specific issue was Kaggle's **silent CUDA multiprocessing failure**: Applio's F0 extraction uses *ProcessPoolExecutor* for parallel pitch computation, which silently crashes in Kaggle's containerized CUDA environment, producing 0-byte *.npy* feature files with no error messages. The custom single-threaded *fix_f0.py* workaround which loads the RMVPE model directly on `cuda:0` and processes files sequentially was essential for all RVC preprocessing.

5.5.4 Production Deployment Challenges

The web application deployment involved resolving 16+ bugs across dependencies, inference, and audio quality:

- **hydra-core conflict:** fairseq installs an old hydra-core version lacking *compose/initialize* functions required by VQGAN
- **numpy 2.0 breaking changes:** RVC uses deprecated *numpy*.bool and *numpy.int* aliases removed in numpy 2.0
- **Gradio version sensitivity:** Version mismatches cause ASGI health check failures on Hugging Face Spaces
- **pip metadata corruption:** *omegaconf 2.0.6* has broken package metadata that causes pip ≥24.0 to fail; workaround requires downgrading pip to 23.3.2

5.6 Threats to Validity

Internal Validity:

- Hyperparameter sensitivity: different hyperparameter settings might yield different results for any architecture. The configurations tested represent best-effort optimization within compute constraints.
- **Informal MOS evaluation:** Quality assessment relied on an opinion-based listening panel of 10 university students (native speakers), rather than professional linguists.

External Validity:

- Generalizability to other dialects: only 3 of 6+ major dialect regions are represented in the RVC speaker inventory.
- Generalizability to other low-resource languages: findings may not transfer to languages with fundamentally different phonological characteristics (e.g., tonal languages, click consonant languages).

Construct Validity:

- No formal MOS evaluation: quality assessment relied on the research team's qualitative judgment rather than a controlled listening study with diverse native speakers.
- Limited evaluation criteria: intelligibility and naturalness were assessed informally; phonetic accuracy was not measured with objective phonological analysis tools.

Ecological Validity:

- CPU-only inference on Hugging Face Spaces (30–60 second generation) does not reflect the real-time performance that would be achieved with GPU-accelerated deployment.
- Laboratory evaluation conditions do not capture the full range of real-world usage scenarios (noisy environments, dialectal variation, code-switching).

CHAPTER 6: CONCLUSION AND FUTURE WORK

6.1 Summary of Contributions

This thesis has made the following contributions to the field of low-resource speech technology:

- First Comparative TTS Study for Afaan Oromo. Ten experimental configurations across three architectures (F5-TTS, VITS, Fish Speech) were systematically evaluated the most comprehensive TTS evaluation ever conducted for the language.
- Empirical Documentation of Failure Modes. The complete failure of F5-TTS (multi-speaker noise collapse, single-speaker Mandarin phonology reproduction), the catastrophic VITS vocabulary expansion, and the decoder collapse from unfrozen VITS fine-tuning are documented with rigorous analysis providing future researchers with empirical boundaries that prevent repeating costly failures.
- Novel Progressive LoRA Rank Escalation Strategy. The r=8 → r=32 escalation strategy for Fish Speech demonstrated an effective framework for navigating the bias-variance tradeoff in low-resource TTS fine-tuning.
- Frozen-Decoder Fine-Tuning Validation. Empirically confirmed that freezing the HiFi-GAN decoder enables stable VITS adaptation on small datasets, establishing a practical recipe for low-resource VITS deployment.
- Proof That Vocabulary Expansion Causes Catastrophic Failure**.** The 96-token VITS vocabulary expansion experiment provides definitive evidence that vocabulary modification of pre-trained TTS models is prohibited under data-constrained conditions.
- Hybrid TTS+RVC Pipeline as Optimal Architecture. Demonstrated that combining Fish Speech for linguistic generation with RVC for speaker identity produces the best multi-speaker, multi-dialect synthesis for low-resource languages surpassing both native voice cloning and direct multi-speaker training approaches.
- Multi-Speaker Dialect Voice Cloning. Demonstrated that multi-speaker fine-tuning (Experiment H at 2,500 steps) is the key to unlocking voice cloning across Afaan Oromo dialects. While single-speaker models lock rigidly to one acoustic profile, training on a diverse 68-hour multi-speaker corpus enabled the model to learn cross-dialect phonology. This finding establishes that with a balanced dataset, native multi-

speaker LoRA fine-tuning can natively clone voices across regional dialects without requiring external conversion modules.

• Integrated Open-Source Speech Ecosystem. Engineered and deployed a complete, production-grade speech ecosystem comprising a FastAPI backend, a Flutter mobile application, and four Hugging Face Spaces web applications the first open-source, integrated ASR+TTS+OCR platform for Afaan Oromo.

• "Virtuous Cycle" Data Infrastructure. Designed and deployed a community-driven data collection portal (HNDE/Oromo-ASR-Collector) that operationalizes a self-sustaining feedback loop: users contribute speech recordings, validate ASR transcriptions, and rate TTS outputs, thereby continuously expanding the training corpus and improving model performance over successive retraining cycles.

6.2 Key Findings

The research yielded five key findings with implications beyond Afaan Oromo:

Finding 1: Architecture choice matters more than dataset size for low-resource TTS. 68 hours of data with the wrong architecture (F5-TTS) produced categorically worse results than 17 hours with the right architecture (Fish Speech) and the right strategy (LoRA r=32). This empirical evidence challenges the prevailing assumption in the field that dataset scale is the primary determinant of synthesis quality.

Finding 2: Codec-based TTS with LoRA is the most effective paradigm for low-resource adaptation. The architectural separation of frozen audio generation (VQGAN) from trainable linguistic modeling (T2S with LoRA) provides the optimal balance of quality preservation and adaptation capability.

Finding 3: Never expand the vocabulary of pre-trained TTS models under data-constrained conditions. The VITS vocabulary expansion experiment demonstrates that this intervention causes catastrophic alignment collapse that cannot be recovered with limited data.

Finding 4: RVC provides the most practical path to multi-speaker, multi-dialect synthesis. By decoupling speaker identity from linguistic generation, RVC requires only 30 minutes of target speaker audio dramatically less than native multi-speaker TTS

training and eliminates the dataset balance requirements that plague multi-speaker approaches.

Finding 5: Data augmentation is essential for ASR generalization. The 4× augmentation (speed perturbation + additive noise) was the critical factor enabling the ASR model's real-world robustness, transforming a model that exhibited test-set overfitting into a system with demonstrable real-world transcription utility.

6.3 Limitations

The following limitations constrain the interpretation and generalizability of this research:

- No formal MOS evaluation with a large panel of diverse native Afaan Oromo speakers. TTS quality was assessed qualitatively by the research team.
- CPU-only deployment on Hugging Face Spaces results in 30–60 second generation latencies, inadequate for real-time applications.
- Dataset imbalance in multi-speaker experiments caused systematic voice cloning drift that was not resolved within this work.
- Dialect coverage limited to 3 of 6+ major Oromo dialect regions (Standard (Shawaa), Borena, Wollega), Hararghe, Arsi, and Guji are not represented.
- Code-switching not optimized: The system does not handle Oromo-Amharic-English mixing, which is common in urban speech.
- Single ASR architecture evaluated (Whisper-Small only). Alternative architectures (Wav2Vec 2.0, Conformer) were not compared.
- No real-time streaming capability. All processing is batch-mode.
- Mobile app integration pending. The Flutter app framework is designed but full integration with all model endpoints is ongoing.

6.4 Future Work

The findings and limitations documented in this thesis open several promising avenues for future investigation:

6.4.1 Dataset Curation and Expansion

The most impactful immediate improvement would be the curation of a **balanced multi-speaker corpus** with equal representation across all major Afaan Oromo dialect regions (Borena, Shewa, Wollega, Hararghe, Arsi, and Guji). The speaker drift observed in Experiment H was directly attributable to the severe dialect imbalance in the current corpus. A balanced dataset of 200+ hours, with approximately equal contributions from each dialect region, would enable the multi-speaker LoRA model to achieve true dialectal awareness without the speaker drift that currently limits its voice cloning fidelity.

6.4.2 Real-Time Streaming Inference

The current system operates in a batch processing paradigm, with end-to-end latency of 30–60 seconds on CPU-only Hugging Face Spaces. Future work should implement **WebSocket-based real-time streaming** with GPU-accelerated inference, targeting sub-3-second generation latency. This would enable conversational applications, real-time interpretation, and interactive voice assistants use cases that the current latency profile cannot support.

6.4.3 On-Device Inference via Model Quantization

Deploying the TTS and ASR models directly on mobile devices would eliminate the dependency on network connectivity a critical requirement for rural Oromia, where internet access remains intermittent. Future work should explore **ONNX Runtime quantization** (INT8) of both the Fish Speech T2S model and the Whisper-Small ASR model, targeting inference on mid-range Android devices with 4–6 GB RAM. The frozen VQGAN decoder and RVC models are also candidates for ONNX export.

6.4.4 Formal Perceptual Evaluation

The MOS evaluation conducted in this thesis, while informative, relied on an informal panel of 10 university students. A formal perceptual evaluation should be conducted with a larger, demographically diverse panel of native speakers ($N \geq 50$), including speakers from each major dialect region, varying age groups, and both urban and rural populations. The evaluation protocol should follow the ITU-T P.800 recommendation for subjective

speech quality assessment, with MUSHRA (MUltiple Stimuli with Hidden Reference and Anchor) tests for fine-grained quality discrimination.

6.4.5 Speaker Diarization and Multi-Turn Dialogue

Extending the ASR component with **speaker diarization** the automatic segmentation and labeling of speech by speaker identity would enable the system to handle multi-speaker recordings such as interviews, meetings, and broadcast media. Combined with the existing TTS voice cloning capability, this would enable the system to transcribe a conversation and re-synthesize it with distinct, recognizable voices for each speaker.

6.4.6 Expansion to Additional Cushitic Languages

The methodological framework established in this thesis progressive LoRA rank escalation, frozen-decoder fine-tuning, hybrid TTS+RVC pipelines is architecturally language-agnostic. Future work should apply this framework to other underserved Cushitic languages, including Somali (approximately 20 million speakers), Sidama (approximately 4 million speakers), and Hadiyya (approximately 1.5 million speakers). The typological similarities within the Cushitic family suggest that intermediate fine-tuning on Afaan Oromo data could serve as an effective "stepping stone" for adaptation to these related languages.

6.4.7 iOS Deployment and Cross-Platform Expansion

The current mobile application targets Android exclusively. Future work should extend the Flutter application to iOS deployment, requiring adaptation of the audio recording pipeline to iOS-specific APIs and the Google ML Kit OCR integration to its iOS counterpart. The backend architecture is already platform-agnostic, requiring no modification for iOS support.

6.4.8 Emotional and Expressive Speech Synthesis

The current TTS system generates speech with neutral prosody. Future work should investigate **style-conditioned generation** enabling the system to synthesize speech with emotional expressiveness (joy, sadness, urgency, formality) through style tokens or

reference audio conditioning. This capability would be particularly valuable for applications in storytelling, news broadcasting, and educational content creation.

6.5 Closing Statement

This thesis has presented a systematic investigation of speech technology development for Afaan Oromo, a language spoken by over 50 million people yet substantially underserved by existing computational tools. Through the comparative evaluation of ten synthesis configurations across three architectural families, the research has established that carefully selected fine-tuning strategies particularly codec-based language modelling with parameter-efficient adaptation can yield perceptually acceptable speech synthesis even under severe data and compute constraints. The resulting integrated system, encompassing text-to-speech synthesis, automatic speech recognition, and mobile deployment, demonstrates a reproducible pathway for developing speech technology infrastructure for similarly under-resourced languages.

Important limitations remain, as discussed throughout this chapter. The evaluation methodology requires formal validation, the dataset exhibits notable dialectal imbalance, and the system's generalizability to other low-resource languages has not yet been empirically tested. Nonetheless, the open-source release of all models, training pipelines, and deployment code provides a concrete foundation upon which subsequent research can build. Addressing the digital language divide for the world's under-resourced languages will require sustained collaborative effort across research institutions, language communities, and funding bodies; this work contributes one incremental but documented step in that direction.

REFERENCES

[1] Grand View Research, "Speech and Voice Recognition Market Size Report, 2023–2033," 2023.

[2] Ethnologue, *Languages of the World*, 26th ed. SIL International, 2023.

[3] B. Nekoto et al., "Participatory Research for Low-resourced Machine Translation: A Case Study in African Languages," in *Findings of ACL: EMNLP 2020*, 2020.

[4] Central Statistical Agency of Ethiopia, "Population and Housing Census of Ethiopia," 2007.

[5] Federal Democratic Republic of Ethiopia, "Proclamation No. 1162/2019: Working Languages of the Federal Government," 2019.

[6] A. Baevski, Y. Zhou, A. Mohamed, and M. Auli, "wav2vec 2.0: A Framework for Self-Supervised Learning of Speech Representations," in *NeurIPS*, 2020.

[7] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, "Robust Speech Recognition via Large-Scale Weak Supervision," in *ICML*, 2023.

[8] D. Gemechu et al., "Speech Transformer-Based ASR for Afaan Oromo: Broadcast News Domain," Jimma University, 2023.

[9] J. Kim, J. Kong, and J. Son, "Conditional Variational Autoencoder with Adversarial Learning for End-to-End Text-to-Speech," in *ICML*, 2021.

[10] J. Ao et al., "SpeechT5: Unified-Modal Encoder-Decoder Pre-Training for Spoken Language Processing," in *ACL*, 2022.

[11] Y. Chen et al., "F5-TTS: A Fairytaler that Fakes Fluent and Faithful Speech with Flow Matching," arXiv:2410.06885, 2024.

[12] Fish Audio, "Fish Speech: Open Source Multi-Lingual Text-to-Speech," GitHub, 2024. Available: https://github.com/fishaudio/fish-speech

[13] E. Hu et al., "LoRA: Low-Rank Adaptation of Large Language Models," in *ICLR*, 2022.

[14] L. R. Rabiner, "A Tutorial on Hidden Markov Models and Selected Applications in Speech Recognition," *Proceedings of the IEEE*, vol. 77, no. 2, pp. 257–286, 1989.

[15] C. M. Lee and K. F. Lee, "CMU Sphinx: The Carnegie Mellon University Speech Recognition System," *IEEE ASSP Magazine*, 1990.

[16] S. Young et al., "The HTK Book (version 3.4)," Cambridge University, 2006.

[17] G. Hinton et al., "Deep Neural Networks for Acoustic Modeling in Speech Recognition," *IEEE Signal Processing Magazine*, vol. 29, no. 6, pp. 82–97, 2012.

[18] T. N. Sainath, A. Mohamed, B. Kingsbury, and B. Ramabhadran, "Deep Convolutional Neural Networks for LVCSR," in *ICASSP*, 2013.

[19] A. Graves, A. Mohamed, and G. Hinton, "Speech Recognition with Deep Recurrent Neural Networks," in *ICASSP*, 2013.

[20] Addis Ababa University, "Speaker-Independent Continuous Afaan Oromo ASR Using HMM-DNN," 2022.

[21] A. Graves, S. Fernández, F. Gomez, and J. Schmidhuber, "Connectionist Temporal Classification: Labelling Unsegmented Sequence Data with Recurrent Neural Networks," in *ICML*, 2006.

[22] D. Bahdanau, K. Cho, and Y. Bengio, "Neural Machine Translation by Jointly Learning to Align and Translate," in *ICLR*, 2015.

[23] W. Chan, N. Jaitly, Q. Le, and O. Vinyals, "Listen, Attend and Spell," in *ICASSP*, 2016.

[24] A. Vaswani et al., "Attention Is All You Need," in *NeurIPS*, 2017.

[25] A. Gulati et al., "Conformer: Convolution-augmented Transformer for Speech Recognition," in *Interspeech*, 2020.

[26] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in *NAACL*, 2019.

[27] W.-N. Hsu, B. Bolte, Y.-H. Tsai, K. Lakhotia, R. Salakhutdinov, and A. Mohamed, "HuBERT: Self-Supervised Speech Representation Learning by Masked Prediction of Hidden Units," *IEEE/ACM TASLP*, vol. 29, pp. 3451–3460, 2021.

[28] A. J. Hunt and A. W. Black, "Unit Selection in a Concatenative Speech Synthesis System Using a Large Speech Database," in *ICASSP*, 1996.

[29] H. Zen, K. Tokuda, and A. W. Black, "Statistical Parametric Speech Synthesis," *Speech Communication*, vol. 51, no. 11, pp. 1039–1064, 2009.

[30] Y. Wang et al., "Tacotron: Towards End-to-End Speech Synthesis," in *Interspeech*, 2017.

[31] J. Shen et al., "Natural TTS Synthesis by Conditioning WaveNet on Mel Spectrogram Predictions," in *ICASSP*, 2018.

[32] A. van den Oord et al., "WaveNet: A Generative Model for Raw Audio," arXiv:1609.03499, 2016.

[33] J. Kong, J. Kim, and J. Bae, "HiFi-GAN: Generative Adversarial Networks for Efficient and High Fidelity Speech Synthesis," in *NeurIPS*, 2020.

[34] V. Pratap et al., "Scaling Speech Technology to 1,000+ Languages," Meta AI, 2023.

[35] Coqui AI, "XTTS v2: Cross-Lingual Text-to-Speech," 2023.

[36] B. Bedasa, "Text-to-Speech Synthesis for Afaan Oromo Using Transformer Architecture," MSc Thesis, Addis Ababa University, 2025.

[37] R. Smith, "An Overview of the Tesseract OCR Engine," in *ICDAR*, 2007.

[38] Y. Du et al., "PP-OCR: A Practical Ultra Lightweight OCR System," arXiv:2009.09941, 2020.

[39] Google, "Google ML Kit Text Recognition API," 2024.

[40] B. Teferi and J. Bigun, "OCR of Amharic Documents," in *Proc. SCIA*, 2005.

[41] N. Houlsby et al., "Parameter-Efficient Transfer Learning for NLP," in *ICML*, 2019.

[42] K. M. Herman et al., "The Impact of Tokenization on Cross-Lingual Transfer," in *ACL*, 2023.

[43] F. Abu, et al., "Unit Selection Based Afaan Oromo Text-to-Speech Synthesis," Addis Ababa University, 2021.

[44] Mendeley Data, "Afaan Oromoo Speech Dataset," Mendeley Data, V1, 2023. Available: https://data.mendeley.com/datasets/mpy85ns82z/1

[45] Sagalee Project, "Sagalee: A Crowdsourced Afaan Oromoo Speech Corpus," Open-Source Dataset.

APPENDIX A: DETAILED IMPLEMENTATION SUPPLEMENT

This supplement contains detailed technical implementation information that expands upon the methodology presented in Chapter 3. Insert sections where referenced.

S3.1 Complete Data Processing Pipeline

The data processing pipeline was developed and refined across two major iterations (V1 and V2), implemented as modular Kaggle notebook cells enabling independent re-execution of each stage:

```
metadata.csv → CSV Parser → Prefix Leak Fix → Text Normalization →
Audio Validation → Peak Normalization → Train/Val Split → HF DatasetDict
```

S3.1.1 Robust CSV Parser Implementation (Cell 3)

The CSV parser was built from scratch because Python's standard *csv* module failed to handle the dataset's irregular structure specifically, unquoted commas embedded within text and *text_expanded* columns.

The algorithm anchors on the *AOTTS_XXXX* filename pattern to establish column boundaries:

```
# For each line in the CSV:
parts = line.split(",")
# Find the File_name column by regex
fname_pos = next(
  (i for i,p in enumerate(parts)
   if re.match(r"^AOTTS_\d+$",p.strip())),
  None,
)
# Calculate text_expanded boundaries using known column positions
exp_start = fname_pos + 1 + cols_mid
exp_end   = len(parts) - cols_after_exp

# Rejoin the expanded text (which may span multiple comma-separated parts)
text_exp_val = ",".join(parts[exp_start:exp_end]).strip()
```

This approach correctly handled commas within text fields by using structural knowledge of column positions relative to the *File_name* anchor a technique that is robust even when the text itself contains sequences of multiple commas.

S3.1.2 Five-Strategy Prefix Leak Detection and Correction (V2 Pipeline)

The V2 dataset pipeline implemented a sophisticated five-strategy prefix-leak detection system. Prefix leaks occurred when portions of the `text` column bled into the *text_expanded* column due to CSV misalignment. Five detection strategies were developed, applied in priority order:

Strategy D — Repeated Suffix (Pre-join detection, highest priority):

```
# Detect when exp_parts tail mirrors its head
if len(exp_parts) >= 3:
  _p0 = _norm_part(exp_parts[0])
  if len(_p0) >= 12:
    for _j in range(2, len(exp_parts)):
      if _norm_part(exp_parts[_j]) == _p0:
        _k = len(exp_parts) - _j
        if all(_norm_part(exp_parts[_j + _i]) == _norm_part(exp_parts[_i])
            for _i in range(_k)):
          exp_parts = exp_parts[_k:] # Strip leaked prefix
```

This strategy detects when the expanded text contains a structural repetition pattern where the tail of the parts array mirrors its head indicating that the text column's content was prepended to the expanded text.

Strategy A0 — Semicolon-Comma Boundary (;,): The most specific two-character marker. When the sequence ;, is found within the reconstructed text, everything after this boundary is taken as the true expanded text. This handles cases where the Oromo text column ended with a semicolon followed by a comma separator.

Strategy A1 — Semicolon with Length Ratio Analysis: If text after a ; boundary is ≥70% the length of text before it, and both portions exceed minimum length thresholds, the after-portion is identified as the true expanded text.

Strategy A2 — Semicolon with Head-Match: If text before ; starts with the same content as text after ;, the after-portion is a trailing repeat leak; the before-portion is kept.

Strategy C — Double Comma (,,): When a double comma is found, text after the double comma is taken as the true content.

Strategy B — Case-Based Detection (lowest priority): If the first part of the reconstructed text starts lowercase but a later part starts uppercase, the uppercase-starting portion is identified as the actual expanded text (indicating the lowercase portion was a leaked prefix from the text column, which typically starts lowercase).

Each strategy included a correction algorithm that surgically removed the leaked prefix while preserving the intended expanded text. A logging mechanism tracked which strategy corrected each row, enabling post-processing quality verification.

S3.1.3 Audio Validation and Normalization

Duration Filtering:

```
MIN_DUR_S = 1.0  # Minimum 1 second (shorter clips cause alignment failures)
MAX_DUR_S = 15.0  # Maximum 15 seconds (longer clips exceed VRAM)
```

Files shorter than 1 second caused VITS alignment failures because there was insufficient audio for the attention mechanism to learn meaningful text-to-audio correspondence. Files longer than 15 seconds exceeded the T4 GPU's VRAM during training due to the quadratic memory cost of the attention mechanism.

Peak Normalization (V2):

```
PEAK_TARGET = 0.95
audio, sr = librosa.load(path, sr = 16_000, mono = True)
peak = np.abs(audio).max()
if peak > 0:
  audio = audio * (PEAK_TARGET / peak)
audio = np.clip(audio, −1.0, 1.0)
```

This ensured consistent volume levels across all training samples, preventing the model from learning volume variations as linguistic features rather than phonetic features.

Train/Validation Split:

```
split = ds_full.train_test_split(test_size = 0.05, seed = 42)
```

A fixed random seed (42) ensured reproducibility. The resulting split: ~6,600 train / ~350 validation samples.

S3.1.4 Dataset Format Per Architecture

Each TTS architecture required data in a different format:

VITS (HuggingFace DatasetDict):

```
ds_train = Dataset.from_pandas(train_df).cast_column("audio", Audio(sampling_rate
        = 16_000))
ds_val   = Dataset.from_pandas(val_df).cast_column("audio", Audio(sampling_rate
        = 16_000))
dataset  = DatasetDict({"train": ds_train, "validation": ds_val})
dataset.save_to_disk(SAVE_DIR)
```

Fish Speech (.wav + .lab pairs): Paired audio files and text label files organized in a speaker directory, subsequently processed into protobuf format via the Fish Speech tokenization pipeline using *fish_speech/utils/file.py*.

F5-TTS (pipe-delimited metadata):

```
audio_path|text|duration
Dataset/Wavs/AOTTS_0001.wav|Akkam jirtu?|3.45
Dataset/Wavs/AOTTS_0002.wav|Nagaan jirtuu?|2.87
```

Metadata written as pipe-delimited text files compatible with the *finetune_cli.py* training script.

S3.2 Comprehensive Afaan Oromoo Text Normalization Module Full Specification

The text normalization module is one of the most significant engineering contributions of this thesis, handling all non-standard word forms in Afaan Oromoo text. It was developed as a standalone module (*v2_cell3_5_normalize.py*) and subsequently integrated into the dataset building cell.

S3.2.1 Number-to-Word Conversion (*num_to_oromo*)

The Oromo numeral system follows a decimal base with specific compound formation rules: **Appendix Table 1**

Base numerals (0–10):

Appendix Table 1: *Base Digits (0-10) and Their Written Form in Afaan Oromo*

Digit	Oromo	Digit	Oromo
0	zeeroo	6	jaha
1	tokko	7	torba
2	lama	8	saddeet
3	sadii	9	sagal
4	afur	10	kudhan
5	shan		

Tens (20–90): Formed using a unique base-ten vocabulary:

Appendix Table 2: *Compound and Cardinal Written Forms of Tens (20-90) in Afaan Oromo*

Number	Cardinal Form	Linking Form (before unit)
20	digdama	digdamii
30	soddoma	soddomii
40	afurtama	afurtamii
50	shantama	shantamii
60	jahaatama	jahaatamii
70	torbaatama	torbaatamii
80	saddeettama	saddeettamii
90	sagaltama	sagaltamii

Key linguistic feature: Tens use a **linking form** with the suffix -mii when followed by a unit digit, connected without a conjunction:

- 25 = digdamii shan (NOT digdama fi shan)
- 11–19 = kudha + unit (e.g., 15 = kudha shan)

Hundreds and above:

Appendix Table 3: *Scale Formats and Written Examples of Hundreds and Higher Numerals in Afaan Oromo*

Scale	Oromo	Example
100	dhibba	200 = dhibba lama
1,000	kuma	1,500 = kuma fi dhibba shan
1,000,000	miliyoona	2M = miliyoona lama
1,000,000,000	biiliyoona	1B = biiliyoona tokko

The conjunction fi ("and") links hundreds to tens/units when both are present: 1,525 = kuma fi dhibba shan fi digdamii shan.

S3.2.2 Ordinal Numbers (`num_to_oromo_ordinal`)

Ordinals use the suffixes -ffaa or -affaa depending on the final vowel of the cardinal:

Appendix Table 4: *Cardinal and Ordinal Conversion Patterns in Afaan Oromo*

Number	Cardinal	Ordinal
1st	tokko	tokkoffaa
2nd	lama	lammaffaa
3rd	sadii	sadaffaa
4th	afur	afraffaa
5th	shan	shanaffaa
10th	kudhan	kudhanaffaa

S3.2.3 Date Normalization

Dates in DD/MM/YYYY format are converted using twelve Oromo month names:

Appendix Table 5: *English Month Equivalents and Their Standard Names in Afaan Oromo*

#	English	Oromo
1	January	Amajjii
2	February	Guraandhala
3	March	Bitooteessa
4	April	Ebla
5	May	Caamsaa
6	June	Waxabajjii
7	July	Adooleessa
8	August	Hagayya
9	September	Fuulbana
10	October	Onkololeessa
11	November	Sadaasa
12	December	Muddee

Example: 15/03/2024 → Bitooteessa kudha shan, bara kuma lama fi digdamii afur

The prefix bara (meaning "year") is prepended to the year component.

S3.2.4 Currency Normalization

Eight currency symbols are supported with Oromo denomination names:

Appendix Table 6: *Currency Symbols and Their Written Conversion Rules in Afaan Oromo*

Symbol/Code	Oromo Name	Example
$ / USD	doolaara	$50 → doolaara shantama
€ / EUR	yuuroo	€100 → yuuroo dhibba
ETB / Br / Birr	Birrii	500 Birr → Birrii dhibba shan
£ / GBP	paawundii	£20 → paawundii digdama
¥ / JPY/CNY	yaanaa	¥1000 → yaanaa kuma
₹ / INR	ruupii	₹500 → ruupii dhibba shan
KES	shilingii	KES 200 → shilingii dhibba lama

Decimal currency: $25.50 → "doolaara digdamii shan fi santii shantama" (using santii for cents).

S3.2.5 Other Normalizations

Appendix Table 7: *Special Categories and Expansion Rules for Oromo Text Normalization*

Type	Pattern	Example → Output
Percentages	`50%`	"dhibbeentaa shantama"
Temperatures	`30°C`	"digrii soddoma seelshiyeesii"
Time	`3:45`	"sa'aatii sadii fi daqiiqaa afurtamii shan"
Distance	`5km`	"shan kiilomeetira"
Weight	`10kg`	"kudhan kiiloograama"
Years	`2024`	"kuma lama fi digdamii afur" (range 1900–2100)
Decimals	`3.14`	"sadii tuqaa tokko afur" (tuqaa = "point")
URLs	`https://...`	stripped entirely
Emails	`user@dom`	stripped entirely
Brackets	`[note]`	content removed

S3.2.6 Abbreviation Expansion (60+ entries)

Appendix Table 8: *Standard Abbreviation Expansion Dictionary for Afaan Oromo Text Normalization*

Abbreviation	Oromo Expansion
Dr.	Doktar
Prof.	Profeesar
Mr./Ob.	Obbo
Mrs. / Ms./Ad	Aaddee
etc./kkf	fi kan kana fakkaatan
e.g./fkn/f.k.n	fakkeenyaaf
i.e.	jechuun
vs.	fi / faallaa
Jan. – Dec.	Amajjii – Muddee
Mon. – Sun.	Wiixata – Dilbata
Gov.	Mootummaa
Univ./yuniv	Yuunivarsiitii
St.	Qulqullu / Daandii
Ave.	Daandii
km, m, cm, mm	kiilomeetira, meetira, seentimeetira, miliimeetira
kg, g, mg	kiiloograama, giraama, miliigiraama
hr, min, sec	sa'aatii, daqiiqaa, sekondii

S3.3 VITS Implementation: Complete Cell-by-Cell Pipeline

S3.3.1 VITS V1 Pipeline (Unfrozen Decoder Experiment D)

Appendix Table 9: *Notebook Cell Mapping for VITS V1 Unfrozen Decoder Pipeline*

Cell	Script	Purpose
Cell 5 (v3)	vits_cell5_config_v3.py	Training configuration: 20K steps, LR=1e-4, batch=16, grad_accum=4
Cell 7	vits_cell7_train.py	Training launch with Patches A–G applied at runtime
Cell 8	vits_cell8_inference.py	Single-checkpoint inference with weight-norm fix
Multi-Checkpoint	vits_multi_checkpoint_test.py	Side-by-side comparison of steps 12.5K, 15K, 17.5K, 20K

S3.3.2 VITS V2 Pipeline (Frozen Decoder Experiment E)

The V2 pipeline comprised 13 cells, each addressing a specific stage:

Appendix Table 10: *Notebook Cell Mapping and Environment Upgrades for VITS V2 Pipeline*

Cell	Script	Purpose	Key Changes from V1
Cell 1	v2_cell1_setup.py	Environment setup	Fresh clone, clean install, no legacy conflicts
Cell 2	v2_cell2_patch.py	Dependency verification	VitsModelForPreTraining import fix, local dataset patch
Cell 3	v2_cell3_dataset.py	Dataset building	Enhanced: 5-strategy prefix leak fix, text normalization, peak normalization
Cell 3.5	v2_cell3_5_normalize.py	Standalone normalizer	Comprehensive Afaan Oromoo text normalizer module
Cell 5a	v2_cell5a_config.py	Config attempt 1	Initial frozen decoder config
Cell 5b	v2_cell5b_config.py	Config attempt 2	Anti-collapse recovery: weight_disc=0.1
Cell 5c	v2_cell5c_config.py	Config attempt 3	Fine-tuned LR schedule
Cell 7a	v2_cell7a_train.py	Training attempt 1	Initial frozen decoder training
Cell 7b	v2_cell7b_train.py	Training attempt 2	With Patches A–K, anti-collapse config
Cell 7c	v2_cell7c_train.py	Training attempt 3	Resume from checkpoint-2500
Cell 8b	v2_cell8b_inference.py	Inference	V2-specific with chunked generation
Cell 8c	v2_cell8c_multi_inference.py	Multi-checkpoint test	Comparative inference across all V2 checkpoints
Cell 10b	v2_cell10b_upload.py	Checkpoint persistence	Upload to Kaggle Datasets for cross-session resumption

S3.3.3 Complete VITS Training Patches (A–K)

Appendix Table 11: *Complete Stable Patches (A-K) Implemented in the VITS Training Loop*

Patch	Name	Code / Description
A	Safe Gradient Clipping	`torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)` — prevents gradient explosion
B	Loss Scaling for FP16	GradScaler integration with mixed precision — prevents NaN losses from half-precision underflow
C	Checkpoint Format Compatibility	Handles both legacy `.bin` and modern `.safetensors` formats; strips `model.` prefix if present
D	Audio Loading Error Handling	`try/except` around `librosa.load()` — returns silence tensor for corrupted audio files instead of crashing
E	Batch Collation Fixes	Pads variable-length sequences within a batch to the maximum length; handles edge cases of empty audio
F	Duration Predictor Gradient Stabilization	Clamps duration predictor output to [0.1, 100.0] range; prevents log-domain NaN from zero durations
G	Weight-Norm Removal for Inference	`torch.nn.utils.parametrizations.remove_parametrizations(module, "weight")` — bakes parametrized weights into plain `.weight` tensors; without this, inference produces severe "radio buzzing" artifacts
H	Multi-Checkpoint Evaluation	Iterates over all checkpoint directories, loads each, generates test sentences, saves labeled audio files for comparison
I	**FREEZE DECODER** (KEY FIX)	Freezes all `decoder.*` parameters via `param.requires_grad = False`; preserves Meta's pre-trained HiFi-GAN
J	`allow_extra_keys` in Config	Handles newer Transformers versions that removed `group_by_length` from TrainingArguments
K	Guard `group_by_length` Access	`getattr(training_args, 'group_by_length', False)` prevents AttributeError in Transformers 5.x

S3.3.4 V2 Anti-Collapse Training Configuration

The V2 configuration underwent three iterations. The final "anti-collapse recovery" configuration (Cell 5b) contained critical changes:

Appendix Table 12: *Hyperparameter Tuning Iterations and Rationale for VITS V2 Anti-Collapse Training*

Parameter	V1 Value	V2 Final Value	Rationale
max_steps	20,000	25,000	More steps needed for encoder-only learning
learning_rate	1e-4	2e-5	10× lower — frozen decoder requires gentler LR
weight_disc	3.0	**0.1**	**Critical fix**: prevents discriminator from overpowering frozen generator
weight_fmaps	2.0	1.0	Paper default — doubled value choked flow gradients
weight_kl	1.5	1.0	Paper default — over-regularized prosody at 1.5
weight_mel	45.0	45.0	Unchanged — only stable gradient source for flow training
warmup_steps	1,000	0	VITS uses ExponentialLR (hardcoded) — warmup has no effect
eval_steps	2,500	1,000	More frequent evaluation for stability monitoring

The weight_disc=0.1 setting was the single most critical recovery parameter. With the decoder frozen, the discriminator's gradient should ideally have no effect on waveform generation. However, it still influenced the normalizing flow module through the adversarial loss pathway. Setting it to 0.1 (reduced from default 3.0) prevented the discriminator from overpowering the flow module, which had been causing training collapse in earlier V2 attempts at steps 2,500–5,000.

S3.3.5 Weight-Norm Inference Pipeline (Cell 8)

The VITS inference pipeline required a 7-step weight management process to handle the mismatch between training checkpoints (which use PyTorch's *parametrize* API) and inference models (which expect plain weights):

```
# Step 1: Load raw state dict from model.safetensors
state_dict = safetensors.torch.load_file(checkpoint_path)

# Step 2: Detect vocab size from embedding weight shape
vocab_size = state_dict["text_encoder.emb.weight"].shape[0]

# Step 3: Create a fresh model with the correct vocab size
config = VitsConfig(vocab_size = vocab_size, ...)
model = VitsModel(config)

# Step 4: Apply weight_norm to decoder and flow layers
#       (matching the checkpoint's parametrized key structure)
for module in model.decoder.resblocks:
  torch.nn.utils.weight_norm(module)

# Step 5: Load state dict with strict = False
#       (ignoring discriminator keys which aren't needed for inference)
model.load_state_dict(state_dict, strict = False)

# Step 6: Remove weight_norm − bake into plain .weight tensors
for module in model.decoder.resblocks:
  torch.nn.utils.parametrizations.remove_parametrizations(module, "weight")

# Step 7: Set model to eval mode
model.eval()
```

Critical detail: If Step 6 is omitted, the model produces severe high-frequency artifacts ("radio buzzing") because the parametrized weight structure causes numerical instability during inference when batch normalization statistics are not running.

S3.3.6 Chunked Generation for Long Text (V2 Cell 8b)

V2 introduced a chunked generation strategy for long sentences that would otherwise degrade in quality:

```
def generate_chunked(text, model, tokenizer, device):
    """Split on sentence boundaries, generate each, concatenate."""
    chunks = re.split(r'(?<=[.!?])\s+', text)
    sr = model.config.sampling_rate
    silence = np.zeros(int(sr * 0.2), dtype=np.float32)  # 200ms pause
    waves = []
    for chunk in chunks:
        inputs = tokenizer(chunk, return_tensors="pt").to(device)
        with torch.no_grad():
            wav = model(**inputs).waveform.squeeze().cpu().numpy()
        waves.append(wav)
        waves.append(silence)
    return np.concatenate(waves[:-1])  # Remove trailing silence
```

This addressed VITS's tendency to produce degraded audio for long input sequences by splitting at sentence boundaries and inserting natural 200ms silence gaps between clauses.

S3.3.7 Discriminator Collapse Pattern (V2 Discovery)

During V2 training, a discriminator collapse pattern was observed and documented:

Appendix Table 13: *Phase-Wise Analysis of VITS V2 Discriminator Collapse and Loss Behavior*

Training Phase	Step Range	Disc Loss	Total Loss	Audio Quality
Phase 1: Stable	0 – 2,500	~8.5	~31	Improving
Phase 2: Degrading	2,500 – 5,000	Rising rapidly	Rising	Degrading
Phase 3: Collapsed	5,000 – 10,000	Saturated	Saturated	No intelligible output

The recovery strategy: (1) resume from checkpoint-2500 (last stable point), (2) reduce weight_disc from 3.0 to 0.1, (3) optionally reset discriminator weights (Patch L concept, planned but not required after weight_disc fix).

S3.4 Fish Speech Implementation: Extended Details

S3.4.1 LoRA Merge and Interpolation

After LoRA fine-tuning, the adapter weights were merged into the base model for inference efficiency using the formula:

$$W_merged = W_base + (\alpha/r) \times B \times A$$

An interpolation merge script (`fish_v2_interp_merge.py`) was also developed for blending base and fine-tuned weights at controllable ratios:

$$\#\, W_final = (1 - alpha) \times W_base + alpha \times W_merged$$
$$\#\, alpha = 0.0 \rightarrow pure\ base\ model$$
$$\#\, alpha = 1.0 \rightarrow pure\ fine-tuned\ model$$
$$\#\, alpha = 0.7 \rightarrow 70\%\ fine-tuned, 30\%\ base\ (recommended\ for\ stability)$$

This interpolation was useful for finding the optimal balance between base model robustness and Oromo-specific adaptation, as full merge (alpha=1.0) occasionally introduced pronunciation quirks on uncommon words.

S3.4.2 Fish Speech Dataset Preparation

The Fish Speech training pipeline requires data in a specific format:

- **Audio Organization:** Speaker-specific directories containing .wav files and corresponding .lab text label files:

```
data/oromo_speaker/
├── AOTTS_0001.wav
├── AOTTS_0001.lab    # Contains: "Akkam jirtu?"
├── AOTTS_0002.wav
├── AOTTS_0002.lab    # Contains: "Nagaan jirtuu?"
└── ...
```

- **Semantic Tokenization:** The fish_speech/utils/file.py module processes audio through the VQGAN encoder to generate semantic token sequences, saved as .npy arrays.
- **Protobuf Packaging:** Paired text-semantic data is packaged into protobuf format for efficient training I/O.

S3.4.3 Per-Speaker RVC Preprocessing Scripts

Five separate preprocessing scripts were created for the five target speakers, each following the identical three-stage pipeline but pointing to different dataset directories:

Appendix Table 14: *Per-Speaker Audio Preprocessing and Training Dataset Distribution in RVC*

Script	Speaker	Dataset Selection	Notes
rvc_cell2_preprocess.py	AOTTS (Male, Standard)	Full TTS dataset, first 350 files	Primary male speaker
rvc_cell2_female_preprocess.py	SPKR005 (Female, Durba)	Speaker-specific subset	Female dialect speaker
rvc_cell2_spkr210_preprocess.py	SPKR210 (Male, Borena)	Speaker-specific subset	Male Borena dialect
rvc_cell2_spkr227_preprocess.py	SPKR227 (Female, Siiq Shawaa)	Speaker-specific subset	Female Siiq Shawaa
rvc_cell2_spkr234_preprocess.py	SPKR234 (Male, Siiq Shawaa)	Speaker-specific subset	Male Siiq Shawaa

Each script: (1) selects speaker-specific audio files, (2) slices at 40kHz with VAD, (3) runs fix_f0.py for single-threaded RMVPE pitch extraction, (4) runs ContentVec feature extraction.

S3.4.4 The fix_f0.py Custom Script Full Implementation

The custom single-threaded F0 extraction script was the solution to Kaggle's silent CUDA multiprocessing failure. It was dynamically generated and executed within Cell 2 at runtime:

```
# fix_f0.py — Single-threaded RMVPE F0 extraction for Kaggle
import torch, numpy as np, os, glob

# Failsafe: download rmvpe.pt if missing
if not os.path.exists("assets/rmvpe/rmvpe.pt"):
    os.system("wget -P assets/rmvpe/ <rmvpe_url>")

# Load RMVPE model on CUDA
from rvc.lib.rmvpe import RMVPE
model = RMVPE("assets/rmvpe/rmvpe.pt", device="cuda:0")

# Process all sliced 16kHz audio files sequentially
wav_files = sorted(glob.glob(f"./logs/{EXP_NAME}/sliced_audios_16k/*.wav"))
for wav_path in wav_files:
    audio = librosa.load(wav_path, sr=16000)[0]

    # Extract raw F0
    f0 = model.get_f0(audio, thred=0.03)

    # Convert to mel-scale coarse codes (256 bins, 50-1100 Hz range)
    f0_mel = 1127.0 * np.log(1.0 + f0 / 700.0)
    f0_mel_min = 1127.0 * np.log(1.0 + 50.0 / 700.0)
    f0_mel_max = 1127.0 * np.log(1.0 + 1100.0 / 700.0)
    f0_coarse = np.clip(
        (f0_mel - f0_mel_min) * 254 / (f0_mel_max - f0_mel_min) + 1,
        1, 255
    ).astype(int)

    # Save as .npy arrays
    np.save(wav_path.replace(".wav", "_f0.npy"), f0)
    np.save(wav_path.replace(".wav", "_f0_coarse.npy"), f0_coarse)
```

A separate Cell 2b (rvc_cell2b_f0_fix.py) was also created as a standalone re-extraction script for cases where Cell 2's inline extraction failed partway through.

S3.5 Deployment Details: Complete Bug Registry

S3.5.1 Dependency and Environment Bugs (9 resolved)

Appendix Table 15: *Dependency and Environment Integration Bugs Registry and Resolutions*

#	Bug	Root Cause	Resolution
1	unhashable type: 'dict' on Gradio startup	Gradio 5.x API schema change creates unhashable dict objects	Pinned gradio==4.44.1
2	500 Internal Server Error on ASGI health check	Starlette version mismatch with Gradio's internal ASGI app	Pinned starlette==0.38.6
3	Jinja2 TemplateNotFound	Jinja2 3.2+ removed template rendering APIs that Gradio depends on	Pinned jinja2==3.1.4
4	gradio_client bool schema crash	Pydantic 2.10+ changed boolean schema validation	Pinned pydantic<2.10.0
5	omegaconf metadata rejection by pip	pip ≥24.1 enforces strict package metadata validation; omegaconf 2.0.6 has broken metadata	Downgraded pip to 23.3.2 during RVC install
6	ImportError: cannot import 'compose' from 'hydra'	fairseq installs old hydra-core<1.1 which lacks compose/initialize needed by VQGAN	Force-reinstall hydra-core>=1.3 after RVC install
7	RVC numpy.bool AttributeError	NumPy 2.0 removed deprecated numpy.bool, numpy.int, numpy.float aliases	Pinned numpy<2.0
8	RVC pyworld/fairseq C extension crash	PyTorch 2.6+ changed internal C API used by pyworld and fairseq	Pinned torch>=2.0,<2.6.0
9	HfFolder AttributeError	huggingface_hub>=0.31 removed deprecated HfFolder class	Added compatibility shim: HfFolder = type('HfFolder', (), {'get_token': lambda: HfApi().token})

S3.5.2 Inference and Audio Quality Bugs (7 resolved)

Appendix Table 16: *Inference Latency and Synthesis Audio Quality Bugs Registry and Resolutions*

#	Bug	Symptom	Root Cause	Resolution
10	VQGAN silent failure	No audio output, no error	VQGAN defaults to --device cuda, but HF Space runs CPU-only	Added explicit --device cpu flag
11	8-minute inference for short sentences	Simple 5-word sentence takes 8+ minutes	Without voice prompt, T2S generates 1024 noise tokens before producing speech	Re-enabled voice prompting with curated .npy reference files
12	First word cut off	"Akkam" becomes "kkam"	Phonetic blending at voice prompt boundary causes initial phonemes to merge with prompt tail	Prepend sacrificial prefix `. ` (period + space) to force pause boundary
13	Aggressive silence trimming	Natural pauses within sentences removed	silence_thresh=0.015 amplitude threshold too high	Reduced to 0.002, then disabled trimming entirely
14	"Frightened/cold" voice quality	Pitch instability, shaky intonation	Temperature 0.6 too high for Oromo phonology + filter_radius 3 too low	Temperature reduced to 0.3; filter_radius increased to 7
15	Background buzzing sound	Persistent low-frequency buzz under speech	VQGAN decoder produces faint spectral hiss; RVC amplifies it during conversion	Added noisereduce spectral gating (prop_decrease=0.8) before RVC
16	Base voice leaking through clone	Output sounds 50% original + 50% target	index_rate=0.5 insufficient for full voice conversion	Tested 0.95 (too robotic), settled on 0.75 as optimal balance

S3.5.3 RVC Parameter Tuning Journey

The RVC inference parameters were iteratively tuned through systematic A/B testing:

Appendix Table 17: *RVC Inference Parameter Hyperparameter Tuning and Empirical Findings*

Parameter	Initial	Tested Values	Final	Finding
index_rate	0.5	0.5, 0.75, 0.85, 0.95	**0.75**	0.95 caused robotic artifacts; 0.5 had base voice leakage
rms_mix_rate	0.25	0.0, 0.25, 0.5, 1.0	**1.0**	0.25 broke naturalness; 1.0 preserves original energy envelope
filter_radius	3	3, 5, 7, 9	**7**	Lower values caused "pitch shivering" (rapid F0 oscillation); 7 smoothed pitch
protect	0.5	0.1, 0.33, 0.5, 0.8	**0.33**	Consonant protection ratio — lower = more conversion but risks consonant distortion
f0_method	rmvpe	rmvpe, crepe, harvest	**rmvpe**	Most accurate for Afaan Oromo; crepe was slower with no quality gain
pitch (male)	0	-6, 0, +6	**0**	No pitch shift needed for male base → male target
pitch (female)	0	0, +6, +12, +18	**+12**	Base model is male (~120 Hz F0); +12 semitones reaches female F0 range (~240 Hz)

S3.5.4 Fish Speech + RVC Web Application File Structure

```
hf_clone/
├── app.py                      # Main application (535 lines)
├── config.json                 # Speaker profiles, inference params,
mastering config
├── Dockerfile                  # Docker build with RVC dependency
workarounds
├── requirements.txt            # Pinned Python dependencies (35 entries)
├── README.md                   # HF Space metadata
├── fish_speech/                # Fish Speech 1.5 source code
│   ├── models/text2semantic/inference.py  # T2S inference (1118 lines)
│   ├── models/vqgan/inference.py          # VQGAN decoder (125 lines)
│   ├── configs/                           # Hydra configuration files
│   ├── conversation.py                    # Prompt encoding
│   └── tokenizer.py                       # FishTokenizer
├── text2semantic/              # Fine-tuned T2S weights (downloaded at
startup)
├── vqgan/                      # VQGAN decoder checkpoint (downloaded at
startup)
├── voices/default/             # Voice prompt files (.npy semantic codes,
.lab text)
├── models/rvc/                 # Downloaded RVC models (runtime, on-demand)
├── output/                     # Generated audio (auto-cleaned)
└── temp/                       # Request-isolated temp dirs (auto-cleaned
every 10 min)
```

S3.5.5 Iterative Deployment Timeline (15+ iterations)

Appendix Table 18: *Continuous Deployment Iterations and Deployment Stages for the TTS Application*

#	Deployment	Change
1	Initial Fish Speech deployment	Dialect dropdown, basic text input
2	Production rewrite	UUID-isolated requests, auto device detection, config-driven mastering
3	Anti-hallucination	Adaptive code truncation, tighter token budget
4	RVC integration	5 speakers, 3 dialects, Docker SDK migration
5	Dependency pinning round 1	gradio, jinja2, starlette
6	VQGAN device fix	--device cpu flag, hydra-core reinstall
7	NumPy/PyTorch pinning	numpy<2.0, torch<2.6.0 for RVC compatibility
8	Audio mastering v1	Basic silence trimming (later revised)
9	Voice prompting restored	Fixed 8-minute hallucination bug
10	Text preparation fix	Prepend `. ` to prevent first-word slicing
11	RVC parameter tuning	index_rate, rms_mix_rate, filter_radius optimization
12	Noise reduction added	noisereduce spectral filtering pre-RVC
13	Pitch stabilization	Temperature 0.3, filter_radius 7
14	Pydantic fix	pydantic<2.10.0 for gradio_client
15	HfFolder compatibility	Shim for huggingface_hub ≥0.31

S3.5.6 Gradio Web Interface Features

The production web interface includes:

- **Text Input:** Afaan Oromoo input with full Qubee script support
- **Speaker Dropdown:** 6 options: "Base (No Clone)", "AOTTS Male Standard", "SPKR005 Female Durba", "SPKR210 Male Borena", "SPKR227 Female Siiq Shawaa", "SPKR234 Male Siiq Shawaa"
- **Pitch Shift Slider:** ±12 semitones (auto-set to +12 for female speakers)
- **Dual Output Tabs:** "Final Output" (post-RVC) vs. "Base Fish Speech" for A/B comparison
- **Pipeline Status:** Real-time generation stage indicators, timing for each stage, and warning messages
- **Example Sentences:** Pre-configured demo sentences for each speaker/dialect combination

▪ **Custom CSS Theme:** Dark gradient background, technology badges, Inter typography from Google Fonts

S3.6 ASR Implementation: Extended Details

S3.6.1 Data Augmentation Implementation

Speed Perturbation:

```
def speed_perturb(audio, sr, factor):
  """Time − stretch audio by factor (0.9 = slower, 1.1 = faster)"""
  return librosa.effects.time_stretch(audio, rate = factor)

# Generate two speed variants per original sample
audio_slow = speed_perturb(audio, sr, factor = 0.9) # Slower speaking
audio_fast = speed_perturb(audio, sr, factor = 1.1) # Faster speaking
```

Additive Noise:

```
def add_noise(audio, noise, snr_db):
  """Add noise at specified signal − to − noise ratio"""
  audio_power = np.mean(audio ** 2)
  noise_power = np.mean(noise ** 2)
  target_noise_power = audio_power / (10 ** (snr_db / 10))
  noise_scaled = noise * np.sqrt(target_noise_power / noise_power)
  return audio + noise_scaled[: len(audio)]

# SNR randomly sampled from [10, 20] dB range
snr = np.random.uniform(10, 20)
audio_noisy = add_noise(audio, noise_sample, snr)
```

Noise sources: MUSAN noise corpus (preferred) with synthetic pink noise fallback generated via np.random.randn() with frequency-domain pink filtering.

S3.6.2 Shard-Based Feature Extraction

The augmented dataset (101,518 pairs) was too large to process in a single Kaggle session. A chunked, shard-based approach was implemented:

• **Sharding:** The augmented pairs JSON was split into shards of ~10,000 pairs each

• **Per-shard extraction:** Each shard was independently processed to extract Whisper log-Mel features

• **Resume support:** Completed shards were tracked via a status file; interrupted sessions resumed from the last incomplete shard

• **Lazy loading:** During training, shards were loaded on-demand rather than all at once, preventing memory overload

S3.6.3 Decoder Configuration for Oromo

Three critical decoder settings were modified from the Whisper defaults:

```
# Force Oromo − specific decoder behavior
model.config.decoder_start_token_id = tokenizer.convert_tokens_to_ids("
                    <|startoftranscript|>")
model.config.forced_decoder_ids = None   # Clear forced language/task tokens
model.config.suppress_tokens = []        # Don't suppress any tokens
```

▪ **forced_decoder_ids = None:** The default Whisper model forces English language and transcription task tokens. Clearing this allows the model to learn Oromo-specific token patterns.

▪ **suppress_tokens = []:** Removes the default suppression of certain punctuation and special characters that may be valid in Oromo text.